

Lab of WEB Technologies

Course introduction

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

About the teacher

- Professor: Zeynep Yucel
- Affiliation: Ca' Foscari University of Venice (DAIS)
- e-mail: zeynep.yucel@unive.it
- Office hours: Meetings are arranged upon booking.
- Webpage: yucelzeynep.github.io

Calendar

- Duration: 30 hours (i.e. 15 frontal lessons)
- Timetable in AY 2025/2026 (days and number of academic hours):
 - Week-1: Nov 11 Tue (4 hours), NOV 12 Wed (2 hours)
 - Week-2: Nov 18 Tue (4 hours), Nov 19 Wed (2 hours)
 - Week-3: Nov 25 Tue (4 hours), Nov 26 Wed (2 hours)
 - Week-4: Dec 02 Tue (4 hours), Dec 03 Wed (2 hours)
 - Week-5: Dec 09 Tue (4 hours), Dec 10 Wed (2 hours)
 - ▶ On Tuesdays 09:30~11:00 and 11:30~13:00
 - ▶ On Wednesdays 11:30~13:00

Moodle page

- <https://moodle.unive.it/>
- The Moodle page is your primary reference to obtain information about the course, teaching materials, etc.
- On Moodle page or directly by email, you can ask questions about the course content and receive support from the teacher.

Course pre-requisites

- Formal pre-requisites for the exam:
 - ▶ Introduction to Coding and Data Management
- Suggested requirements/skills:
 - ▶ Basic coding
 - ▶ Basic understanding of computer networks
 - ▶ Being curious

What is the course structured?

- This course provides an overview of the technologies involved in the modern World Wide Web.
- Theoretical side:
 - ▶ Protocols, Patterns, Security, Networking.
- Practical side:
 - ▶ We will learn how to use some technologies to develop web apps!

What will we learn?

- This course is focused on the technologies used to create rich dynamic front-ends.
- We will learn how to
 - ▶ Use and integrate HTML/CSS/JavaScript
 - ▶ Use modern JavaScript-based frameworks
 - ▶ Understand modern web paradigms and cyber security issues

Course flow

- Part 1: "Languages and technologies for the Web"
 - ▶ HTML
 - ▶ CSS
 - ▶ JavaScript
 - ▶ Asynchronous programming
- Part 2: "How the web works"
 - ▶ The HTTP protocol
 - ▶ Cookies and Sessions
 - ▶ Cyber security principles
 - ▶ Web application vulnerabilities

Reference materials

- Slides and exercises
- Thomas A. Powell, "HTML & CSS: the complete reference", McGraw-Hill Education; 5th edition, 2010. ISBN-10: 0071496297
- Eric A. Meyer, Estelle Weyl, "CSS: The Definitive Guide", O'Reilly Media; 3rd edition (2006). ISBN-10 0596527330
- David Flanagan, "Javascript The Definitive Guide", O'Reilly, 2011. ISBN-10: 0596805527
- Brian Totty, David Gourley, Marjorie Sayer, Anshu Aggarwal, Sailu Reddy, "HTTP: The Definitive Guide", O'Reilly Media, 2009. ISBN-10: 9781565925090
- Dane Cameron, "HTML5, JavaScript, and JQuery", Wrox Pr Inc; 1st edition, 2010. ISBN-10: 9781119001164

Suggested online courses

- KhanAcademy
<https://it.khanacademy.org/computing/computerprogramming>
- W3
<https://www.w3schools.com/html>
<https://www.w3schools.com/css>
<https://www.w3schools.com/js/>
<https://www.w3schools.com/jquery/default.asp>
<https://www.w3schools.com/bootstrap/default.asp>

Evaluation

- The grading is based on:
 - ▶ A **written test** comprising open-ended and/or multiple choice questions covering the arguments introduced during the course.
 - ▶ A **web-application group project** of 2-3 people each.
 - Your project should demonstrate of what you will have learned in this class.
 - It is best if you could work both individually and together.
 - The topic should be agreed by the group and the professor.
 - ▶ Oral examination for clarification.
- Both written text and group project have 30+1 pts and 50% weight in overall grade. You will need at least 18 pts in both of them to pass the course.

Lab of WEB Technologies

Lecture # 1 Introduction

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

What is the Internet? 1/3

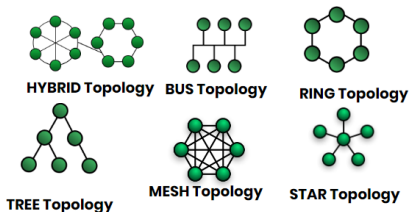
The Internet is the **global** system of interconnected computer networks that use the Internet Protocol Suite (TCP/IP) to link **devices worldwide**.

- It is **global** in a sense that consists of private, public, academic, business, and government networks. All interconnected together.

What is the Internet? 2/3

The Internet is the global system of interconnected computer **networks** that use the Internet Protocol Suite (TCP/IP) to link devices worldwide.

- A computer **network** allows nodes (computers) to share resources (**exchange** data).
- Each node is identified by an address and generally referred as a host.
- Two hosts may exchange data even if there is no direct connection between them (different network topologies!)



What is the Internet? 3/3

The Internet is the global system of interconnected computer networks that use the **Internet Protocol Suite (TCP/IP)** to link devices worldwide .

- A **communication protocol** is a set of rules for exchanging information over a network.
- Protocols are usually layered in a stack
 - ▶ Each protocol leverages the services of the protocol layer below it.
 - ▶ The layer at the lowest level controls the hardware physically sending the information across the media.
- The internet protocol suite
 - ▶ Is a conceptual model for all the protocols used in the Internet.
 - ▶ Commonly known as TCP/IP for the two fundamental protocols: TCP and IP.
 - ▶ Specifies how data should be **packetized, addressed, transmitted, routed, and received.**

More on TCP/IP

- The TCP/IP model stands as the foundation of modern computer networking, facilitating efficient and reliable **communication between diverse systems** worldwide.
- By offering **a standardized method for computers to communicate**, it enables the seamless interaction among all these different devices.
- Additionally, it **simplifies** communication processes by **dividing tasks into manageable stages**, enhancing clarity and efficiency.
- The 5 Layer Protocols in Transmission Control Protocol/Internet Protocol (TCP/IP) are the **Application, Transport, Network, Data Link, and Physical Layers**.
- Each layer has its **own set of protocols** that allow for data transmission and packet switching between different nodes on a network.
- As a result, **complex applications**, such as video streaming and online gaming, can **operate smoothly** without lag or interruption.

The five layers 1/2

Physical, Data link, Network

#	Layer Name	Protocol
5	Application	HTTP, SMTP, etc..
4	Transport	TCP/UDP
3	Network	IP
2	Data Link	Ethernet, Wi-Fi
1	Physical	10 Base T, 802.11

- **Physical layer**: is responsible for the physical transmission of data between two systems (via the cables, connectors, and other hardware). Some standard protocols (e.g. Ethernet and Wi-Fi from the next layer) determine how bits are physically transmitted across the medium.
- **Data link layer** is responsible for packing data into frames and handling their transfer between two nodes. It is responsible for error detection and correction, ensuring data frames are not corrupted during transmission.
- **Network layer** is responsible for routing packets to their destination. It handles addressing, ensuring that each packet is sent to the correct system. In addition, it provides flow control and congestion avoidance mechanisms.

The five layers 2/2

Transport, Application

#	Layer Name	Protocol
5	Application	HTTP, SMTP, etc..
4	Transport	TCP/UDP
3	Network	IP
2	Data Link	Ethernet, Wi-Fi
1	Physical	10 Base T, 802.11

- **Transport layer**: is responsible for providing reliable end-to-end communication between two systems. It handles flow control, ensuring that data packets are sent at a reasonable rate and not dropped during transmission. The Transport layer also provides error checking and correction, allowing errors to be corrected before the packet reaches its destination.
- **Application Layer**: is responsible for providing services and applications that use the underlying protocols for communication. This includes email, web browsing, file transfer, streaming media, and more. It also handles authentication and encryption to ensure data is secure during transmission.

What is the World Wide Web?

- The WWW is an information system in which the items of interest (referred as [resources](#)) are identified by [Uniform Resource Locators \(URL\)](#).
- Resources can be linked by [hypertext](#) and are accessible over the [Internet](#).
- Resources may be accessed by a software application called [web browser](#).

Question

Can you think of any analogies for the pair Internet-WWW?

World Wide Web $\neq$ Internet

- Internet:
 - ▶ The internet is a vast network of interconnected computers and servers that communicate with each other using a standard set of protocols (primarily TCP/IP).
 - ▶ Functionality: Email, file transfer, online gaming, social networking etc.
 - ▶ Infrastructure: Routers, cables, servers, and other technology that facilitate data transmission across global networks.
- WWW
 - ▶ The World Wide Web is a system of interlinked hypertext documents and multimedia content accessed via the internet using web browsers.
 - ▶ Content: Web pages that are created using HTML (Hypertext Markup Language) and can include text, images, videos, and links to other pages.
 - ▶ Protocols: primarily HTTP (Hypertext Transfer Protocol) and HTTPS (HTTP Secure).

Architectural bases of the WWW

- URLs are used to identify resources.
- Web agents communicate using **standardized protocols** that enable interaction through the exchange of messages which adhere to a defined syntax and semantics.
- Resources have a certain **representation** that can be interpreted (and visualized) by web browsers.

Parts of the URL 1/2

Host name and server

- When you want to visit a certain website, everything starts by opening your favorite browser and entering a URL, e.g.
- `http` :// `www.example.org` / `home.html`
- The URL is divided in different parts that together specify the protocol to use, the address of the host machine owning the resource, and the resource name.
 - ▶ **Host name:**
 - The host name is resolved to an Internet Protocol (IP) address using the globally distributed Domain Name System (DNS).
 - After querying the DNS, an IP address like 203.0.113.4 is obtained.
 - ▶ **Server:**
 - The browser will establish a TCP connection with server and start communicating as specified in the HTTP protocol.
 - ▶ **Resource**

Parts of the URL 2/2

Resource

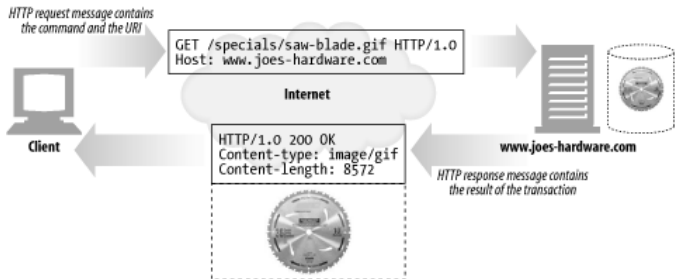
- `http` :// `www.example.org` / `home.html`
 - ▶ Host name
 - ▶ Server
 - ▶ Resource:
 - Using the HTTP protocol, the browser asks for a resource named `home.html`.
 - The resource is not necessarily a file. It can be generated on-the-fly by the server.
 - The resource has an associated representation (MIME-type, or media type or content type).
 - Multipurpose Internet Mail Extensions (MIME) type is a standardized way to indicate the nature and format of a file or data.
 - It is a two-part identifier for file formats and format contents and helps browsers etc. understand how to process and display the received content correctly.
 - For instance, in `text/html`, `text` is type, `html` is subtype.
 - In this case, the resource type is Hyper Text Markup Language (HTML), a standard used to create web pages.
 - The browser interprets the resource and visualizes it graphically.

Browsing the web

- The HTML language can contain [hyperlinks](#) to other web pages or resources.
- Such a collection of useful, related resources, interconnected via hypertext links was called a [web of information](#) by its original inventor: [Tim Berners-Lee](#).

A typical transaction

- A typical transaction looks like:



- In the boxes, you see request and response headers.
- The response header includes an HTTP response status code.
- In this case, it is 200 OK which means that the request was successfully received, understood, and accepted.
- Some other status codes are
 - ▶ 302 Redirect. Go someplace else.
 - ▶ 404 Not Found. Can not find this resource.

More on HTML

- HTML is based on another **markup language** SGML (stands for Standard Generalized Markup Language).
- It **quickly became one of the core building blocks** defining the world wide web as we know it today.
- HTML is a language **interpreted** by web browsers to visualize documents containing text, images, sounds, videos and links to other similar documents.
- It is a **declarative** language, i.e. only allows to define the **document structure** (pages are not interactive).

A bit more on URL

- URI (Uniform Resource Identifier):
 - ▶ A string used to identify a resource either by location, name, or both.
 - ▶ General term that encompasses both URLs and URNs.
 - ▶ Provides a simple and extensible way to identify resources.
 - ▶ Example: `http://example.com`, `urn:isbn:0451450523`,
`ftp://ftp.example.com/file.txt`
- URL (Uniform Resource Locator):
 - ▶ A specific type of URI that not only identifies a resource but also provides a means of locating it by describing its primary access mechanism (such as HTTP, FTP) and network location.
 - ▶ Tells browsers or clients where and how to access a resource.
 - ▶ `https://www.unive.it`, `ftp://ftp.example.com/file.txt`
- URN (Uniform Resource Name):
 - ▶ A type of URI that names a resource in a persistent, location-independent way. It is meant for identifying resources by name within a namespace, without implying where or how to access.
 - ▶ Provides a persistent, location-independent resource identifier, often used for standards like ISBNs, ISSNs, or other persistent identifiers.
 - ▶ `urn:isbn: 3959401035`, `urn:doi:10.1017/CBO9781139568555`

NCSA Mosaic

- NCSA Mosaic is among the first widely available web browsers, instrumental in popularizing the World Wide Web
- It is developed at the National Center for Supercomputing Applications (NCSA) at the University of Illinois at Urbana–Champaign beginning in late 1992 and released in 1993.
- It revolutionized the concept of web browser allowing the visualization of text and images (multimedia) on the same page.



Other browsers and new tools

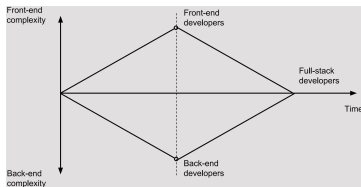
- In 1994–1996, many **browsers were born** (e.g. Internet explorer, Netscape navigator, Opera).



- In 1995, **JavaScript**.
 - ▶ Included in Netscape Navigator 2.
 - ▶ Syntax similar to "Java".
 - ▶ Idea (Marc Andreessen): Allow a web page to dynamically modify its own content (**static pages became dynamic!**).
 - ▶ JavaScript code was embedded in the page itself.
 - ▶ **Not a proper programming language**, but intended for designers.
 - ▶ Many standards. Jscript by Microsoft in 1996.
- In 1996, **CSS**.
 - ▶ Language born to define how **to visualize elements defined by a markup language** (like HTML).
 - ▶ Idea: **Separate content from presentation**.
 - ▶ More flexible, reduced complexity and repetitions.

Back-end and front-end

- In 1996-2000, companies grew interest in better presenting their content.
- CSS and JavaScript became fundamental in developing web pages.
- Contents are now dynamically generated.
- The role of front-end developer is born.
 - ▶ **Back-end developers:** Website "back-stage" (data, content, security, structure, business logic)
 - ▶ **Front-end developers:** User experience and content presentation.
- **Full-stack developers** are expected to be able to work in all the layers of the application.
- The convergence and divergence of developer roles over time.



Asynchronous programming

- In 2005, Jesse James Garrett published a paper "Ajax: A New Approach to Web Applications".
- AJAX = Asynchronous JavaScript And XML
- The term Ajax is coined to describe a **set of technologies** to develop web applications that **communicate asynchronously** with the server without interfering with the display and behavior of the existing page.
- Actually, modern implementations use JSON instead of XML.
- JSON = JavaScript Object Notation
XML = Extensible Markup Language

Web 1.0 vs 2.0

- In 2004, Tim O'Reilly popularized the concept of Web 2.0.
- Web 2.0 is not a standard, but defines the way in which web pages are developed and used.
- Comparison of Web 1.0 and Web 2.0
 - ▶ Web 1.0
 - Web 1.0 is a retronym referring to the first stage of the World Wide Web's evolution, from roughly 1989 to 2004.
 - Static pages.
 - Content is generated by few "publishers" and viewed by many users.
 - ▶ Web 2.0
 - From wikipedia, "Web 2.0 (also known as participative (or participatory) web and social web) refers to websites that emphasize user-generated content, ease of use, participatory culture, and interoperability (i.e., compatibility with other products, systems, and devices) for end users. "
 - Dynamic pages.
 - Content dynamically generated by the users in virtual communities.
 - E.g. social media.

What about the future? Web 3.0?

- The Semantic Web, sometimes referred as Web 3.0, aims to make Internet data machine-readable.
- Tim Berners-Lee:
"I have a dream for the Web [in which computers] become capable of analyzing all the data on the Web – the content, links, and transactions between people and computers. A "Semantic Web", which makes this possible, has yet to emerge, but when it does, the day-to-day mechanisms of trade, bureaucracy and our daily lives will be handled by machines talking to machines. The "intelligent agents" people have touted for ages will finally materialize."
- Web 3.0 is beyond client-server paradigm toward a peer-distributed system.
- It is still not a standard and under active research.

Lab of WEB Technologies

Lecture # 2 HTML 1/2

Zeynep Yücel

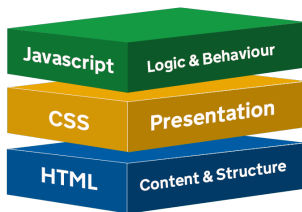
Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

Hypertext and hypermedia

- The web started as a giant **hypertext information system**.
- The idea behind hypertext is that instead of reading text in a rigid, linear structure, one can **jump** from one point to another following the **links**.
- Currently, the content is **not limited to text**, but may also contain images, videos, etc, hence **Hypermedia**.
- Nowadays, entire applications can be developed and used via web, hence **web applications**.

Web pages

- Content is defined in **web pages** that can be retrieved and visualized by a **web browser**.
- Each web page is written in a language called **HyperText Markup Language (HTML)** that includes:
 - ▶ A description of the page **structure**.
 - ▶ Links to other documents, images or media.
 - ▶ Definitions regarding the document **presentation (CSS)** or **additional functionality and interactivity (JavaScript)**.



- HTML is a **markup language**.
- But what is a markup language?

Different language purposes

- **Programming languages** are used to create software applications and operating systems.
- **Scripting languages** are used to automate tasks and add functionalities to application.
- **Markup languages** are used to structure and format text and other contents.

Programming

- Needs to be compiled before executing
- Provides a one-shot version
- C, C++, Java, C#, ...

Scripting

- Interpreted (no explicit compilation)
- Designed for run time systems
- Python, Perl, Ruby, PHP, ...

Markup

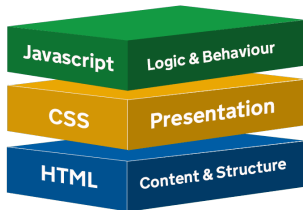
- Interpreted by browser
- For annotating documents
- HTML, XML, Markdown, ...

Markup languages

- A markup language is used to **annotate** a document with a set of **tags**.
- Tags are used to provide additional **meaning** and **structure** to the text of the document.
- Markup focuses on the structural aspects of a document and leaves the visual presentation of that structure to the interpreter.
- In case of web documents, HTML **structures and expresses the content** of a web page in a manner that can be **consistently interpreted by a web browser**.
- The HTML markup language specifies:
 - ▶ What tags can be used in a document.
 - ▶ How the tags should be used.
 - ▶ What additional information (attributes) a tag can contain.

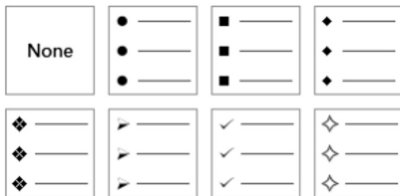
Fundamental concept

- HTML should be **only** used to define the **content and structure** of the document.
- The document formatting (how is visualized) is handled by another technology: **CSS**.
- The document dynamic behavior is described by a scripting language (usually **JavaScript**).



Structure vs. Presentation 1/2

- Structure refers to the parts of a document that authors create (like titles, paragraphs, lists, etc.).
 - ▶ For example, we can define a list of items as an "unordered list".
 - ▶ This definition do not say anything about how it is supposed to look.



Structure vs. Presentation 1/2

- Presentation refers to the way in which various components of a page are actually displayed on a given media device (i.e. computers, printers, etc.)
- For example, in these slides we visualize unordered lists with blue circles at the first level and with blue triangles and a certain indentation at the second level and with blue squares and additional indentation in the third level.
- First level
 - ▶ Second level
 - Third level

Why separating structure from presentation?

1. Authors (especially programmers) usually are not very good designers, vice versa.
 - ▶ Good idea to let authors compose their documents and designers worry about how documents should look.
 - ▶ Designers can easily make sure that every document produced within an organization conforms to a certain style.
2. If markup consists in content and structure alone, the document appearance can be quickly changed.
 - ▶ We can define different presentation settings for different media devices.
 - ▶ We can change the style of an entire website without rewriting the content.
3. A document can be more accessible to people with physical limitations
 - ▶ A document may be presented by a Braille printer for those with limited vision or by a text reader for those with limited hearing.

Creating HTML documents

- An HTML document contains just text!
- Can be created with any text editor.



Notepad



Notepad++



Emacs



TextEdit



Atom



VSCode

- Then it can be visualized simply in a web-browser (locally).



Google Chrome



Microsoft Edge



Mozilla Firefox



Safari

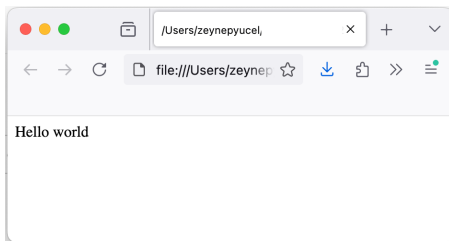
...

- Usually, we create a text file with `.html` extension.

Basic HTML template

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="utf-8" />  
</head>  
<body>  
  Hello world  
</body>  
</html>
```

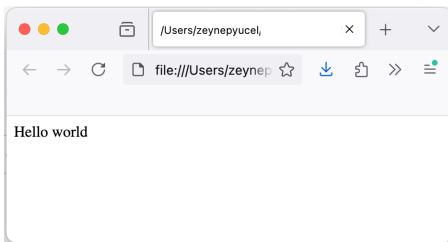
Every HTML compliant page must start with a special tag that defines the HTML version to which the page conforms. For the latest HTML5 standard, the tag is the following:
`<!DOCTYPE html>`



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

All HTML documents have 3 document-level tags in common, delimiting different sections of the page: `<html>`, `<head>`, `<body>`.



Basic HTML template

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="utf-8" />
```

```
</head>
```

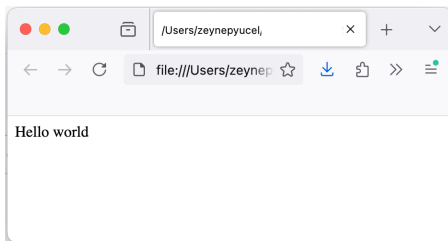
```
<body>
```

```
    Hello world
```

```
</body>
```

```
</html>
```

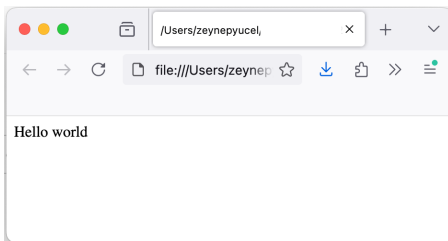
The `<html>` tag surrounds the entire document and tells the browser where the document begins and ends.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
</head>
<body>
    Hello world
</body>
</html>
```

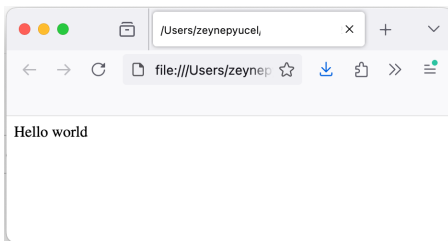
The `<head>` tag delimits the heading of the document. The heading can contain "meta" information like the document title, the character set, optional CSS and JavaScript definitions etc.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

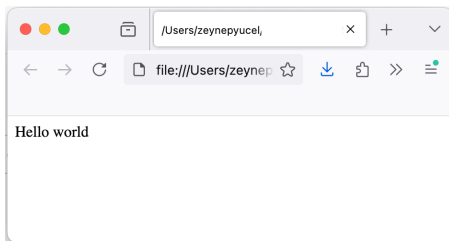
The `<body>` tag delimits the actual page content that will be displayed by the browser. The body section can contain text, images, links, lists, tables and most tags supported by the HTML standard.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

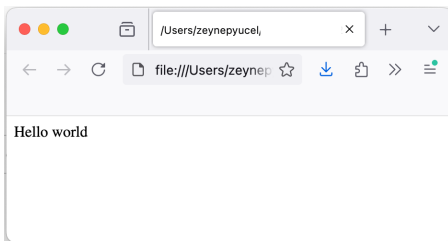
Elements consist in an opening and closing tag.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

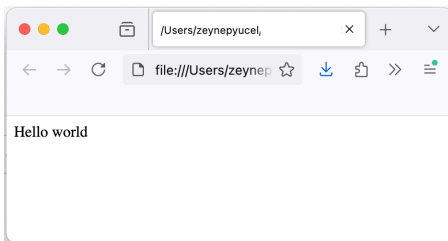
Elements can be nested one inside the other to create a tree-like structure.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

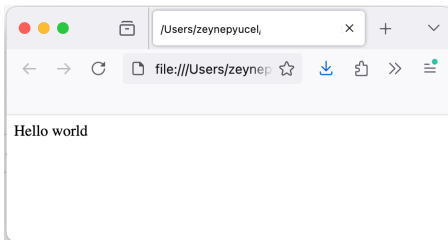
Every element in a document (except html) have exactly one parent element.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

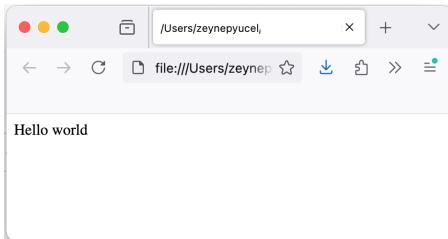
The parent of meta is head.
The parent of head is html.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

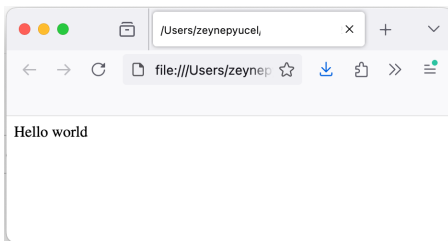
Some elements may have text content.



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

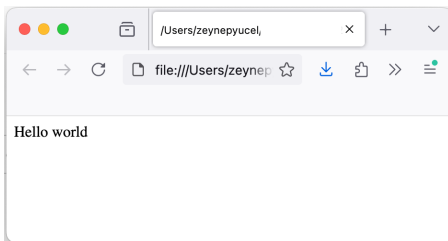
Some elements may contain attributes
in the form attribute="value"



Basic HTML template

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
</head>
<body>
  Hello world
</body>
</html>
```

If the attribute is boolean, it will contain only the attribute, e.g.
<input read-only />



Characteristics of well-formatted HTML

1. Begin with the DOCTYPE declaration (it is not a tag!)
 - ▶ In HTML 5
`<!DOCTYPE html>`
 - ▶ In HTML 4.01
`<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">`
2. Tags should be closed in the opposite order of how they were opened.
 - ▶ Wrong!
`<p> hello <strong> world </p> </strong>`
 - ▶ Right ✓
`<p> hello <strong> world </strong></p>`
3. All non-empty elements must be terminated.
 - ▶ Wrong!
`<p> hello <span> world </p>`
 - ▶ Right ✓
`<p> hello <span> world </span></p>`
4. All attribute values should be quoted.
 - ▶ Wrong!
`<input type=checkbox />`
 - ▶ Right ✓
`<input type="checkbox" />`

Structuring text

- Like a textual document created with a word processor, HTML documents **comprise paragraphs and other structuring sections**.
- Text formatting can be performed **only via tags**.
- In particular, all the white spaces will be condensed.
- Browser will render **one white space**, where the line breaks and where multiple spaces appear.

Block and inline elements

- HTML elements are divided in two classes:
 - ▶ **Block** elements **change** the content flow.
 - They are normally **separated from each other** by line breaks and some vertical space.
 - They **occupy** the full width available and enough height to accommodate the content.
 - Within that area, child text and inline elements are wrapped into lines.
 - They **can contain** other block elements, text or inline elements.
 - They **cannot be contained inside an inline** element.
 - ▶ **Inline** elements **do not change** the content flow.
 - They **are not separated** from each other.
 - They are **always found inside block** elements.
 - They take up **as much width as necessary** for the content (not the full width available).

Some common block elements

- `<div>`: A generic container for grouping elements, used for layout purposes.
- `<h1>` ... `<h6>`: Heading elements, where `<h1>` is the highest level and `<h6>` is the lowest level of headings.
- `<p>`: Represents a paragraph of text.
- `<ol>`: An ordered list, used to create a list with a specific order (numbered).
- `<ul>`: An unordered list, used with bullet points.
- `<li>`: A list item, used within `<ol>` and `<ul>`.
- `<nav>`: Represents a section of navigation links.
- `<form>`: Represents a document section containing interactive controls for submitting data to a web server.
- `<table>`: Represents a table for displaying tabular data.

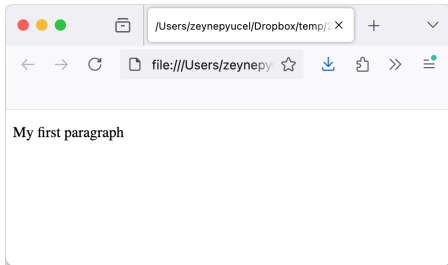
Some common inline elements

- `<span>`: A generic inline container for phrasing content.
- `<a>`: Defines a hyperlink.
- `<img>`: Embeds an image.
- `<em>`: Indicates emphasized text.
- `<b>`: Defines bold text.
- `<i>`: Defines italic text.
- `<br>`: Inserts a line break.
- `<small>`: Renders text in a smaller size.
- `<button>`: Represents a clickable button.

Paragraphs

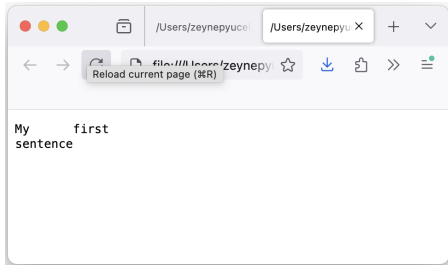
- Paragraphs are delimited by the tag `<p>`.
- Paragraph controls the **line spacing** of the lines **within the paragraph** as well as the line spacing **between paragraphs**.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8"/>
</head>
<body>
  <p>My first paragraph</p>
</body>
</html>
```



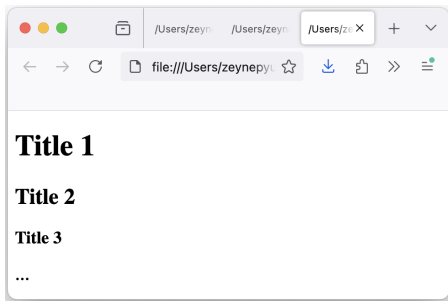
Preserving formatting

- To **preserve text formatting** and let the browser interpret the text **literally** (with spaces, newlines, etc) we can use the tag `<pre>`.
- By default, the browser switches to a **monospace** font to ensure that the formatting appears correctly.



Headings

- HTML has 6 predefined heading tags from the most important to the least:
`<h1>` `<h2>` `<h3>` `<h4>` `<h5>` `<h6>`
- Each heading acts as a paragraph tag, providing automatic line break and extra line spacing before and after the element.



Manual line breaks

- To manually break a line without ending a paragraph, the element `<br>` is used.

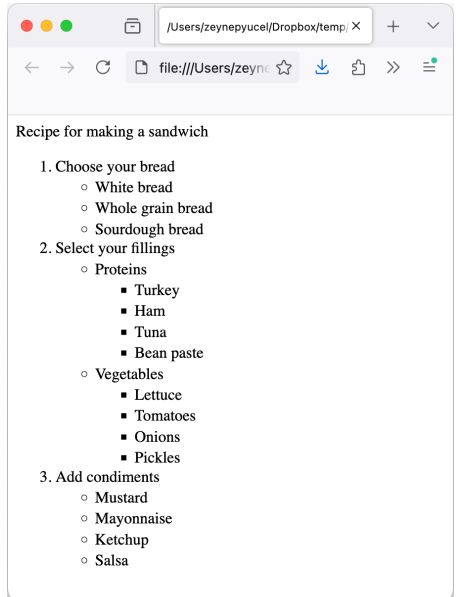
Lists

- A list is a block level element containing a sequence of items.
- HTML provides:
 - ▶ Ordered lists
 - ▶ Unordered lists
 - ▶ Definition lists

Ordered/unordered lists 1/2

- Ordered lists use the tags `<ol></ol>`.
- Unordered lists use the tags `<ul></ul>`.
- Lists contain list items `<li></li>` with the actual content.
- List items are also block elements.
- A list item may contain other lists.
- A list element must contain list items only.

Ordered/unordered lists 2/2

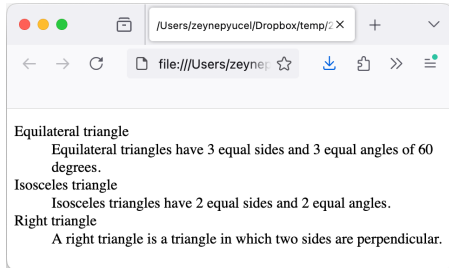


Definition lists 1/2

- The items of a definition list are enclosed in `<dl></dl>`.
- Each item is a pair of:
 - ▶ Definition term `<dt></dt>`.
 - ▶ Definition description `<dd></dd>`.
- Definition list has no restrictions regarding the use of other HTML elements within the definition term or description part.

Definition lists 2/2

```
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='utf-8'>
</head>
<body>
  <dl>
    <dt> Equilateral triangle </dt>
    <dd> Equilateral triangles have 3 equal sides and 3 equal angles
    of 60 degrees. </dd>
    <dt> Isosceles triangle </dt>
    <dd> Isosceles triangles have 2 equal sides and 2 equal
    angles. </dd>
    <dt> Right triangle </dt>
    <dd> A right triangle is a triangle in which two sides are
    perpendicular. </dd>
  </dl>
</body>
</html>
```



Lab of WEB Technologies

Lecture # 3 HTML 2/2

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

Web pages

- Each web page is written in a language called HyperText Markup Language (HTML) that includes:
 - ▶ A description of the page structure
 - ▶ Links to other documents, images or media
- A **mark-up language** is different than a **programming language**, which is again different than a **scripting language**.
- Programming languages are set of instructions or code which tells a computer what it needs to do (e.g. Java, C, C++, C#).
- Scripting languages help us produce the script to perform some certain task. They basically connect one language to one another languages and do not work standalone (e.g. JavaScript, PHP, Perl).
- Markup languages prepare a structure for the data or prepare the look or design of a page. These are presentational languages and they do not include any kind of logic or algorithm (e.g. HTML).

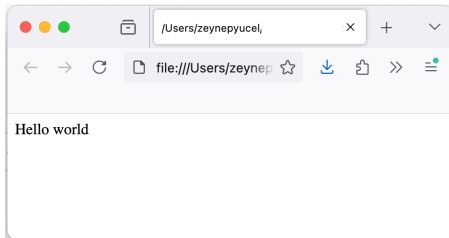
Basic HTML template

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8" />
</head>

<body>
  Hello world!
</body>

</html>
```



HTML elements are divided in two classes:

- **Block** elements **change** the content flow.

- ▶ `<div>`: Division
- ▶ `<h1> ... <h6>`: Heading.
- ▶ `<p>`: Paragraph.
- ▶ `<ol>`: Ordered list.
- ▶ `<ul>`: Unordered list.
- ▶ `<li>`: List item.
- ▶ `<nav>`: Navigation.
- ▶ `<form>`: Form.
- ▶ `<table>`: Tabular data.

Inline elements do **not change** the content flow.

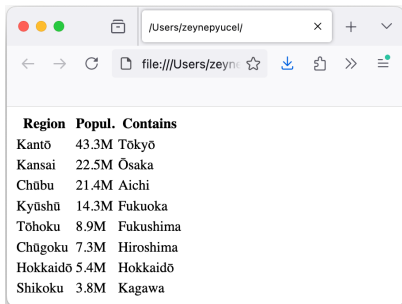
- ▶ `<i>` Italics
- ▶ `<b>` Bold
- ▶ `<u>` Underline
- ▶ `<q>` Quote
- ▶ `<sub>` Subscript text
- ▶ `<em>`: Emphasized text.
- ▶ `<a>`: Hyperlink.
- ▶ `<img>`: Image.

Example for table

Tags `<th></th>`, `<tr></tr>`, `<td></td>` define headers, rows and cells.

```
<table>
  <tr>
    <th> header-1 </th>
    <th> header-2 </th>
  </tr>
  <tr>
    <td> cell-1-1 </td>
    <td> cell-1-2 </td>
  </tr>
  <tr>
    <td> cell-2-1 </td>
    <td> cell-2-2 </td>
  </tr>
</table>
```

The 8 regions of Japan, their populations and important cities.



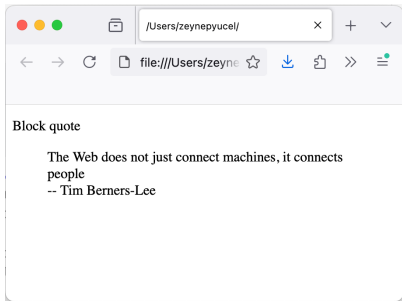
A screenshot of a web browser window. The address bar shows a file path: file:///Users/zeynep/ucel/. The browser displays a table with three columns: Region, Popul., and Contains. The table lists eight regions of Japan with their populations and major cities.

Region	Popul.	Contains
Kantō	43.3M	Tōkyō
Kansai	22.5M	Ōsaka
Chūbu	21.4M	Aichi
Kyūshū	14.3M	Fukuoka
Tōhoku	8.9M	Fukushima
Chūgoku	7.3M	Hiroshima
Hokkaidō	5.4M	Hokkaidō
Shikoku	3.8M	Kagawa

Example for blockquote

```
<p>Blockquote</p>  
<blockquote cite="https://en.wikiquote.org/wiki/Tim_Berners-Lee">  
    The Web does not just connect machines, it connects people.  
    <br />  
    -- Tim Berners-Lee  
</blockquote>
```

- The cite attribute does not render as anything special in any of the major browsers.
- But it can be used by search engines to get more information about the quotation.
- It is a good habit to always add the source of a quotation.



Semantic tags 1/2

- A **semantic element** clearly describes its meaning to both the browser and the developer.
 - ▶ For instance, `<div>` tells nothing about its content.
 - ▶ But `<table>` clearly defines its content.
- Some common semantic tags involve (here given with only their opening tags):
 - ▶ `<em>` : to emphasize the text (default is italics).
 - ▶ `<strong>` : to emphasize even more (default is bold).
 - ▶ `<abbr>` : to indicate the abbreviation of a word. (e.g. `<abbr>CA</abbr>` for California).
 - ▶ `<cite>` : to indicate the cited title of a work (default is italics).
 - ▶ `<code>` : to indicate that the text inside is a code sample (usually in fixed-width font).
 - ▶ `<dfn>` : to indicate a definition.

Semantic tags 2/2

- `<header>`: Represents introductory content, typically containing a logo, navigation, or a heading.
- `<nav>`: Defines a set of navigation links.
- `<article>`: Represents a self-contained piece of content that could be distributed independently, such as a blog post or news article.
- `<section>`: Defines a thematic grouping of content, typically with a heading.
- `<footer>`: Contains footer information, typically including copyright information, contact details, or links.
- `<figure>`: Represents self-contained content, often with a caption, such as an image, illustration, or diagram.
- `<figcaption>`: Provides a caption for the `<figure>` element.
- `<time>`: Represents a specific period in time.
- `<address>`: Contains contact information for the author or owner of a document or article.

Not-semantic tags 1/2

- `<div>`: A generic container for flow content, often used for styling or grouping elements.

```
<div class="container">  
  <h1>Title</h1>  
  <p>Some content here.</p>  
</div>
```

- `<span>`: A generic inline container that is used to apply styles or target specific text without adding any meaning.

```
<p>This is a <span style="color:red">colored</span> text.</p>
```

- `<b>`: Used to make text bold without implying any extra importance.

```
<p> This is <b>bold</b> text.</p>
```

Not-semantic tags 2/2

- `<i>`: Used to italicize text without implying emphasis or importance.
`<p>This is <i>italicized</i> text.</p>`
- `<u>`: Used to underline text without implying any specific meaning.
`<p>This is <u>underlined</u> text.</p>`
- `<font>`: An older tag used to specify font size, color, and face. It is now deprecated in HTML5.
`<font color="red">This text is red.</font>`
- `<center>`: Used to center-align text or other elements. This tag is also deprecated.
`<center>This text is centered.</center>`
- `<link>`: (when used for stylesheets) - While it does have a specific purpose of linking resources, when nested within `<head>` it does not convey semantic meaning about the content.
`<link rel="stylesheet" href="styles.css">`

- The tag has no inherent effect on the text that it is wrapped around.
- It exists solely to be associated with a CSS to modify the text appearance in a [custom way](#).
- Like <div>, but for text instead of other blocks.

Links

- Web links have two basic components:
 - ▶ The **content** (usually text) in the main document that refers to another document.
 - ▶ The **target** document (or location in the document) to which the link leads.
- To create a link, we use the tag `<a> </a>` (the anchor tag).
- `<a>` requires at least the `href` attribute in order to be useful.
- For instance,
`Data has a cat named <a href="spotdata.html">Spot</a>`
- The `href` attribute specifies the target document that will be browsed when clicking on the link.
- The target document can be specified via **relative** or **absolute** URL.

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`



The scheme specifies the protocol to use to retrieve the resource from the server (e.g. http, https, ftp, rtsp, etc.)

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`



User and password can sometimes be requested to authenticate with the server. The default user is "anonymous".

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`



The server address can be a host-name (to be resolved via DNS) or an IP address. Host must always be specified.

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`



The server port on which the server is listening for a connection. Depending on the scheme, the port can be omitted and a default value is used. HTTP default port is 80.

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`



The (local) resource name hosted by the server. Path syntax depends by the scheme. In HTTP the path can be divided by multiple segments separated by / (e.g. seg1/seg2/seg3)

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`



This component allows to specify a list of "name-value" pairs to a path segment. This is useful to provide additional information to the webserver depending on the resource requested.

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`

Similar to `<params>`, `<query>` is frequently used to restrict the context of a certain resource.

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`

`<query>`



Ex:

`http://ww.a.com/inventory.html?item=1234`

In this case, the resource `inventory.html` is restricted to a hypothetical item number 1234.

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`

`<query>`



Ex:

`http://ww.a.com/inventory.html?item=1234&num=3`

Name-value pairs can be separated by
the & character

Links

- A Uniform Resource Locator (URL) is a standard to define and identify resources inside the web.
 - ▶ Usually represent the "entry point" from which the users start browsing the web.
 - ▶ You encode the resource name, the location where to find it and how the browser can retrieve it.
 - ▶ The general URL format comprises of 9 distinct parts (not all are mandatory).
 - ▶ For instance,
`http://www.joes-hardware.com/seasonal/index-fall.html`
 - ▶ `<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>`



A fragment (a.k.a. anchor) is used to identify a fragment inside the specified resource. For example it can refer to a specific paragraph inside an HTML page.

Absolute and relative URLs

- Concerning links, we have examined so far only absolute URLs, i.e. they contain everything needed to access a specific resource.
- Sometimes is simpler to use relative URLs (containing just the resource name).
- Base URL can be obtained in 2 ways:
 - ▶ By specifying it explicitly using the tag `<base>` in the document header, e.g.
`<base href="http://www.mysite.com"/>`
 - ▶ Using the absolute URL of the resource (HTML page) in which the relative URL is contained.
- The latter is more common.

Absolute and relative URLs

- Here is an example:

```
<html>
<head>
  <title>Joe's Tools</title>
</head>
<body>
  <h1> Tools Page </h1>
  <p> Joe's Hardware Online has the largest selection of
  <a href="./hammers.html" />hammers </a>
  </p>
</body>
</html>
```

This URL is relative
(there is no host and
no schema specified)

- Assume that the following is the url of the current page.

<http://www.joes-hardware.com/tools.html>

Base URL

- Then the link will refer to

<http://www.joes-hardware.com/hammers.html>

New absolute URL

The URL character set and escaping

- According to the HTTP protocol, URLs can contain symbols **only from the standard ASCII character set**.
- If the resource name contains characters outside the ASCII set it is necessary to **escape** the name with a set of valid characters.
- Character escaping in a URL is performed in the following way:
%<2 digits hex code>
- The hex code to insert depends on the browser's character set.
- Default is UTF8.
- Here are some escape characters:

!	#	\$	&	'	(	)	*	+
%21	%23	%24	%26	%27	%28	%29	%2A	%2B
- And some others:

,	/	:	;	=	?	@	[	]
%2C	%2F	%3A	%3B	%3D	%3F	%40	%5B	%5D
- The ones after percent-encoding are reserved.

Internationalized Resource Identifier (IRI)

- The Internationalized Resource Identifier (IRI) was defined in 2005 to extend the URL with all the characters contained in the universal character set (Unicode).
- It is backward-compatible via the standard URL escaping.
- Browsers designed to support IRI will visualize the correct characters.
- Here is an example:

<https://en.wiktionary.org/wiki/Πόδος>
https://en.wiktionary.org/wiki/Πόδος

- The corresponding escaped URL is:
<https://en.wiktionary.org/wiki/%E1%BF%AC%CF%8C%CE%B4%CE%BF%CF%82>
- One of the biggest problems of IRI is the Internationalized domain name (IDN) homograph attack.
- The idea is to use the characters similar to common ASCII symbols to redirect a user to a malicious website
- For instance, paypaλ.it instead of paypal.it

Images

- Images can be inserted with the tag `<img>`.
- Two attributes are required:
 - ▶ `src` specifies the image URL.
 - ▶ `alt` specifies an alternate text (if the image cannot be displayed or if a user with a visual impairment visits the page with a screen reader).
- Example:
``

Some examples with images

- Similar to links, images do not define their own text elements.
- So the `<img>` tag has to go inside a paragraph or heading element.

`<p>`

``

`</p>`

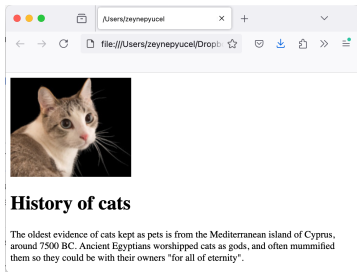


Image formats in the Web

- **Raster files** are images built from pixels — tiny color squares that, in great quantity, can form highly detailed images such as photographs.
 - ▶ The more pixels an image has, the higher quality it will be, and vice versa.
 - ▶ The number of pixels in an image depends on the file type. (e.g. *.jpeg, *.gif, or *.png).
- **Vector files** use mathematical equations, lines, and curves with fixed points on a grid to produce an image.
 - ▶ There are no pixels in a vector file.
 - ▶ A vector file's mathematical formulas capture shape, border, and fill color to build an image.
 - ▶ A vector image scales up or down without impacting its quality (fe.g. *.svg, *.eps, or *.emf)

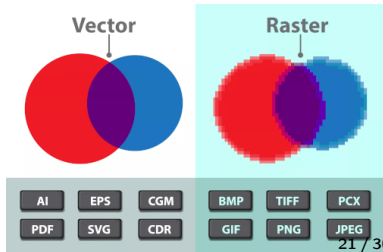
Differences in resolution, use and file size

Raster

- Resolution is in DPI (dots per inch) or PPI (pixels per inch).
- Zooming in or expanding the size, one can see individual pixels.
- They have a wider array of colors, permit greater color editing, and show finer light and shading.
- They are used for editing images, photos, and digital drawings.

Vector

- One can resize, rescale vectors without losing image quality.
- They are popular for images that need to appear in a wide variety of sizes (e.g. a logo that needs to fit on both a business card and a billboard).
- They are much more lightweight than raster files.



Differences in file size

Raster

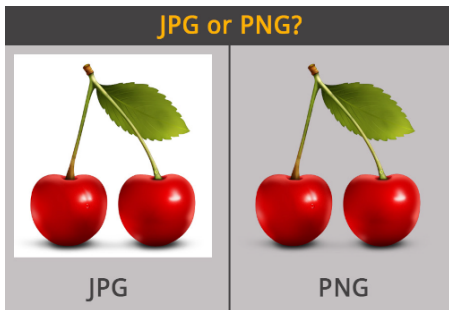
- It is generally larger than a vector file.
- They can contain millions of pixels and incredibly high levels of detail.
- Their large size can impact device storage space and slow down page loading speeds on the web.

Vector

- It is much more lightweight than a raster file.
- Because it contains only the mathematical formulas that determine the design.

Other consideration about rasters

Image format	Available colors	Compression	File size	Best for
Raw	Billions	No	Very big	Editing
JPG	16.1M	Lossy	Small	Website, storage
GIF	256	Lossless	Small	Animation
PNG	16.1M + transp	Lossless	Big	Website, editing, storage
TIFF	Variable	Variable	Big	Editing, printing
BMP	Variable	Lossless	Big	Windows apps



The document <head>

- The information in the head element of an HTML document is very important because it is used to describe or augment the content of the document.
- The most important child elements of head are
 - ▶ title
 - ▶ meta
 - ▶ base
 - ▶ link
 - ▶ style
 - ▶ script

<title>

- **title**: A single title element is required in the head element and is used to set the text that most browsers display in their title bar.
`<title>Simple HTML Title Example</title>`
- Only one title element should appear in every document, and most browsers will ignore subsequent tag instances.
- Make sure that a title element is well formed because omitting the close tag can cause many browsers to not load the document.

<meta>

- A <meta> tag can be used to specify values that are equivalent to HTTP response headers:

```
<meta http-equiv="Content-Type" content="text/html;  
charset=ISO-8859-1"/>
```

- Most of the time, <meta> is used to specify the character set of the document.

<base> and <link>

- A <base> tag specifies an absolute URL address that is used to provide server and directory information for partially specified URL addresses (relative links) used within the document:

```
<base href="http://htmlref.com/basexample" >
```

- Because of its global nature, a <base> tag is often found right after a <title> tag as it may affect subsequent <script>, <link>, <style>.

<link>

- A <link> tag specifies a special relationship between the current document and another document.
- Most commonly, <link> is used to specify a style sheet used by the document

```
<link rel="stylesheet" media="screen"  
href="global.css" type="text/css" >  
<link rel="stylesheet" media="print"  
href="print.css" type="text/css" >
```

Common attributes

- A number of HTML attributes can be used with any HTML elements:
 - ▶ `id` specifies a unique identifier that references the element in CSS and scripts
 - ▶ `class` specifies a semantic class that the element should be considered a member of
 - ▶ `style` specifies CSS style rules that should be applied to the element
 - ▶ `hidden` specifies whether the user agent should hide the element's content.

Custom attributes

- In HTML5, a web developer can create his own attributes to add to any HTML element as long as:
 - ▶ the attribute name begins with the characters "data-"
 - ▶ has at least one more letter or number following the dash
- For example,

```
<li class="book-title" data-borrower="Vonnegut,  
K." data-status="overdue">Venus On The Half-Shell</li>
```
- The purpose of data attributes is to provide extra information about the contents of an element that can be accessed by client-side scripts.

Lab of WEB Technologies

Lecture # 4 HTML assignment

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

Assignment description

We will prepare a simple web site for a travel agency "Discover Italy!". The content should be as follows.

- There should be [links](#) on each page to [navigate](#) between these four: "Home", "Travel plans", "Contact us", and "Tourism in Italy".
- In "Home", there should be a description of Italy together with an [image](#).
- In "Travel plans", there should be a [table](#) of available plans, each with a picture, name of place and price.
 - Each plan ([picture](#)) has a [link](#) to a dedicated page with the same picture, a description, [list](#) of destinations, price and a link to go [back](#).
- In "Contact us", there should be a [form](#) for sending mails (it may redirect to the same page).
- In "Tourism in Italy", there should be a picture and the provided text ([quoted](#) from Wikipedia).

See next slides for text contents and download the zip file from Moodle for images.

Home

Discover Italy!

Italy (Italian: Italia) is a country in Southern Europe. Together with Greece, it is acknowledged as the birthplace of Western culture. Not surprisingly, it is also home to the greatest number of UNESCO World Heritage Sites in the world. High art and monuments are to be found everywhere around the country.

It is famous worldwide for its delicious cuisine, its trendy fashion industry, luxury sports cars and motorcycles, diverse regional cultures and dialects, as well as for its beautiful coast, alpine lakes and mountain ranges (the Alps and Apennines). No wonder it is often nicknamed the Bel Paese (the Beautiful Country).

Check one of our travels plans to discover Italy!

Travel plans 1/2

- Aosta

- Description: Where Nature Meets History

- Destinations:

- Monte Bianco: The highest peak in the Alps, perfect for trekking and skiing.

- Aosta Ancient Roman Ruins: Well-preserved Roman ruins.

- Gran Paradiso National Park: Italy's first national park.

- Price : 600€

- Lazio

- Description: Heart of Italy's Heritage

- Destinations:

- Rome: The Colosseum, Vatican City, and St. Peter's Basilica.

- Tivoli: Villa Adriana and an ancient Roman archaeological complex.

- Sperlonga: A picturesque seaside town known for its beautiful beaches.

- Price : 700€

Travel plans 1/2

- Sicily

- Description: The Heart of the Mediterranean
- Destinations

Palermo: Ballarò and the stunning Palermo Cathedral.

Mount Etna: Europe's most active volcano.

Valle dei Templi: An archaeological site in Agrigento.

- Price : 500€

- Veneto

- Description: Feel the Magic of Veneto
- Destinations

Venice: The romantic city of canals, St. Mark's Basilica, and the Doge's Palace.

Verona: Roman Arena, Juliet's House, Ponte Pietra.

Valdobbiadene: Vineyards producing Italy's famous sparkling wine.

- Price : 800€

"Contact us" and "Tourism in Italy"

Contact Us

Name:

Email:

Message:

Send (button)

Tourism in Italy

Ac. 1760 painting of James Grant, John Mytton, Thomas Robinson and Thomas Wynne on the Grand Tour by Nathaniel Dance-Holland

Tourism in Italy is one of the largest economic sectors of the country. With 60 million tourists per year (2024), Italy is the fifth-most visited country in international tourism arrivals.

According to 2018 estimates by the Bank of Italy, the tourism sector directly generates more than five percent of the national GDP (13 per cent when also considering the indirectly generated GDP) and represents over six per cent of the employed.

People have visited Italy for centuries, yet the first to visit the peninsula for tourist reasons were aristocrats during the Grand Tour, beginning in the 17th century, and flourishing in the 18th and 19th centuries. This was a period in which European aristocrats, many of whom were British and French, visited parts of Europe, with Italy as a key destination. For Italy, this was in order to study ancient architecture, local culture and to admire the natural beauties.

From Wikipedia, the free encyclopedia.

Lab of WEB Technologies

Lecture # 5 Cascading Style Sheets (CSS) 1/2

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

Fundamental concept

- Cascading Style Sheets (CSS) is a powerful tool that **transforms the presentation** of a document (or a collection of documents).
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media.
- CSS **saves a lot of work**, since it can control the layout of **multiple web pages all at once**.
- It can also **account for** design, layout and variations in display for **different devices and screen sizes**.

Who creates and maintains CSS?

- CSS is created and maintained through a group of people within the World Wide Web Consortium (W3C) called the CSS Working Group.
- W3C is a group that makes recommendations about how the Internet works and how it should evolve.
- The CSS Working Group creates documents called specifications.
- When a specification has been discussed and officially ratified by the W3C members, it becomes a recommendation.
- These ratified specifications are called recommendations because the W3C has no control over the actual implementation of the language.

CSS Versions

- Cascading Style Sheets level 1 (CSS1) came out of W3C as a recommendation in December 1996.
 - This version describes the CSS language as well as a simple visual formatting model for all the HTML tags.
- CSS2 became a W3C recommendation in May 1998 and builds on CSS1.
 - This version adds support for media-specific style sheets e.g. printers and aural devices, downloadable fonts, element positioning and tables.
- Unlike previous versions, CSS3 was not released as a single, monolithic specification (2011).
 - Instead, it was divided into several separate modules (selectors, text effects etc.), each introducing new features and capabilities.

Associate CSS with an HTML document

- One can **associate a CSS** file to a document in one of the three ways
 - using the `<link>` tag
 - using the `<style>` element
 - using the `style` attribute

Using the link tag 1/4

- The style definitions are normally saved in **external** .css files.
- With the <link> tag, the browser **locates and loads the stylesheet** and uses it to render the HTML document.

```
<link rel="stylesheet" type="text/css"  
href="sheet1.css" media="all"/>
```

- The href attribute is the **URL of the CSS**, which can be absolute or relative.
- The file extension is not required, but some older browsers do not recognize the file, unless it actually ends with .css, even if you do include the correct type of text/css in the link element.
- The **media attribute** contains one or more media descriptors which are rules regarding media types and features on those media.

Using the link tag 2/4

Media types

- With the `media` attribute, we can apply different styles for different kind of media.
- Some media types are:
 - `all`: use the linked CSS for all presentation media.
 - `print`: use when printing the document and when displaying a print preview.
 - `screen`: used when presenting the document in a screen medium (like a desktop browser).
 - `aural`: intended for speech synthesizers.
 - `embossed`: intended for paged braille printers.
 - `projection`: intended for projected presentation (i.e. projectors or print to transparencies).
 - `tv`: Intended for television-type devices.

Using the link tag 3/4

Linking multiple CSS and applying multiple rules at once

- There can be more than one linked stylesheet associated with a document:

```
<link rel="stylesheet" type="text/css" href="basic.css">  
<link rel="stylesheet" type="text/css" href="splash.css">
```
- This will cause the browser to load both stylesheets, combine the rules from each, and apply them all to the document.
- Why “Cascading”?
 - Multiple styles will cascade into one whether they are defined inside the head section of an HTML page, or in an external CSS file, or inside an HTML element.
- Generally speaking, all the styles will **cascade** into a new **virtual** style sheet by the following **certain rules**.

Using the link tag 4/4

- Here is an example for linking the following (1) index.html and (2) style.css (under the same directory) with the `<link>` tag.

(1) `<!DOCTYPE html>`

```
<html>
```

```
<head>
```

```
  <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

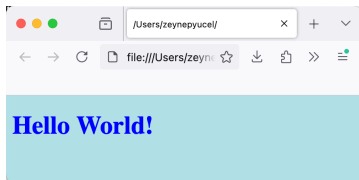
```
  <h1>Hello World!</h1>
```

```
</body>
```

```
</html>
```

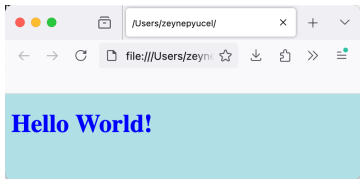
(2) `body {`
 `background-color: powderblue;`
`}`

```
h1 {  
    color: blue;  
}
```



Using the style element

- The `style` element is one way to include a stylesheet.
- It appears in the head section of the document itself and includes the contents of the css file.
- The styles between the opening and closing `style` tags are referred to as the **document stylesheet** or **embedded stylesheet**.
- How would you turn the previous document with external styling into one with embedded style?
 - → Putting the css contents into a style element. The outcome will be identical.



Using the style attribute

- To assign styles to one individual element (without the need for external or embedded stylesheets), we can use the `style` attribute.
- The `style` attribute can be associated with any HTML tag whatsoever, except for those tags that are found outside the body.
- For instance, the following piece of script in the css file sets the color of a paragraph.

```
<style>
  p {
    color: blue;
  }
</style>
```

- One may also set the color of a paragraph directly in the html file as follows:

```
<p style="color: blue;">Here is my text.</p>
```

- How would you turn the previous document into one with inline styling?
- Generally, the use of the `style` attribute is **not recommended**.

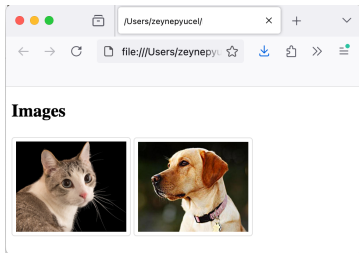
CSS with non-/replaced elements 1/2

- CSS can distinguish between and act on:
 - **Replaced elements**: are those for which the element's content is replaced by something that is not directly represented by document content, e.g.
``
`<input type="text" name="username" placeholder="Enter username">`
 - **Non-replaced elements**: are presented by the browser inside a box generated by the element itself. These are the majority of elements in a document.

CSS with non-/replaced elements 2/2

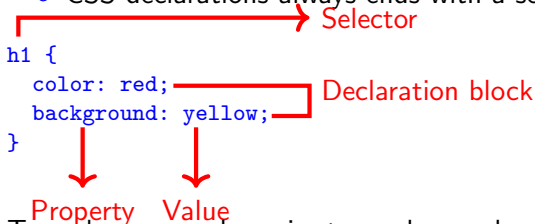
Example with replaced elements

```
<!DOCTYPE html>
<html>
<head>
  <style>
    img {
      border: 3px solid #ddd;
      border-radius: 10px;
      padding: 10px;
      width: 150px;
    }
  </style>
</head>
<body>
  <h2>Images</h2>
  
  
</body>
</html>
```



Basics of CSS

- CSS are composed by list of rules.
 - Each rule has two fundamental parts: the **selector** and the **declaration block**.
 - The declaration block (inside {}) is composed of one or more **declarations**.
 - Each declaration is a pairing of a **property** and a **value**.
 - CSS declarations always ends with a semicolon.



The diagram shows a CSS rule set: `h1 { color: red; background: yellow; }`. A red bracket above the `h1` is labeled "Selector". A red bracket to the right of the `color: red;` and `background: yellow;` lines is labeled "Declaration block". Two red arrows point down from the `color: red;` line to the words "Property" and "Value" respectively.

```
h1 {  
  color: red;  
  background: yellow;  
}
```

Property Value

- To make your code easier to read, you should use spaces, indentation, and blank lines within a rule set.
- CSS comments begin with `/*` and end with `*/`. A CSS comment can be coded on a single line, or it can span multiple lines.

Basics of CSS

- CSS allows to create rules that are simple to change, edit, and apply to all the elements in a document.
- In addition to setting a style for an **HTML element**, CSS allows you to specify **your own selectors** with **ids** and **classes**.
- So there are 3 important types of selectors to be used:
 - Element selectors
 - Class selectors
 - ID selectors
- Other ways of selecting are
 - Descendant selectors
 - Pseudo-class selectors
 - Child selectors
 - Attribute selectors

Element selectors

- The elements of an HTML document serve as the most basic selectors.
- The rules specified in the declaration block will be applied to any document element of that type:

```
h1 {color: gray;}  
h2 {color: silver;}
```

- Selectors can be grouped together (with comma) so that the same rules are applied to all of them:

```
h2, p {color: gray;}
```

- The universal selector * matches all the elements in a document:

```
* { color: red; }
```

Class selectors 1/2

- To associate the styles of a class selector with an element, we must assign a **class attribute** (in the HTML document) with the **appropriate value**.
- Then, the selector is built by preceding the class name with the period (.)
- It is possible to have a space-separated list of words in a single **class value**. But you need a . in the css.
Here is the excerpt from a css file

```
.urgent.warning {  
    font-weight: bold;  
}
```

And this is how it is used in the HTML file

```
<p class="urgent warning">  
    You have unpaid invoice!  
</p>
```

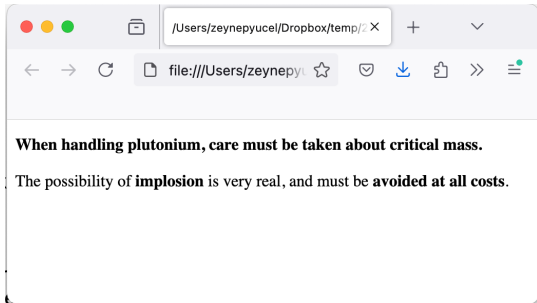
Class selectors 2/2

- A class selector (always preceded by the period) can be combined with an element selector to restrict the selection.
- For instance, the below applies the rules to any p elements that have a class attribute with value warning, but no other elements of any kind, classed or otherwise.

```
p.warning {  
    font-weight: bold;  
}
```

Example for class selector

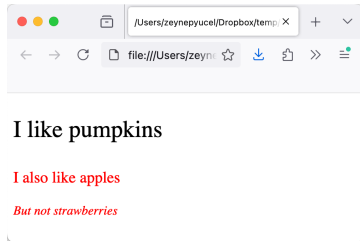
```
<!DOCTYPE html>
<html>
<head>
  <style>
    .warning {
      font-weight: bold;
    }
  </style>
</head>
<body>
  <p class="warning">
    When handling plutonium, care must be taken about critical mass.
  </p>
  <p>
    The possibility of <span class="warning">implosion</span> is very real
    and must be <span class="warning">avoided at all costs</span>.
  </p>
</body>
</html>
```



Multiple classes

- By chaining two class selectors together, you can select only those elements that have both class names, in any order.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .big {font-size: 30px;}
    .small {font-style: italic;}
    .red {color: red;}
    .medium.red {font-size: 20px;}
  </style>
</head>
<body>
  <!--matching classes are given in comments-->
  <p class="big">I like pumpkins</p> <!--.big-->
  <p class="red medium"> Also apples</p><!-- .red, .medium.red-->
  <p class="small red">But not strawberries</p><!-- .red, .small-->
</body>
</html>
```



ID selectors

- ID selectors are similar to class selectors, but with the following differences:
 - ID selectors are preceded by #.
 - ID selectors refer to values found in `id` attributes instead of `class` attributes.
 - IDs should be used once, and only once, within an HTML document.
 - ID selectors cannot be combined with other IDs, since ID attributes do not permit a space separated list of words.
 - Rules carried by an ID selector have precedence over classes (see the specificity rules).
 - ID selectors are case-sensitive.

Example for ID selectors

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
p {  
    color: blue;  
}
```

```
#para1 {  
    color: red;  
}
```

```
</style>
```

```
</head>
```

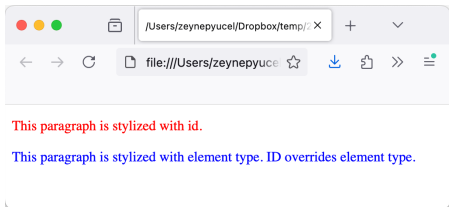
```
<body>
```

```
<p id="para1">This paragraph is stylized with id.</p>
```

```
<p>This paragraph is stylized with element type.  
ID overrides element type.</p>
```

```
</body>
```

```
</html>
```



Descendant selectors

- Descendant selectors are rules that operate in certain structural circumstances of the HTML document, but not others.
- The selector side of a rule is composed of two or more space separated selectors that can be read as "is a descendant of" when read from right to left.
- Example: `div p {color: red;}` will match "Any p element that is a descendant of a div element".
- Note that the closeness of two elements within the document tree has no bearing on whether a rule applies or not.

Example for descendant selectors

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <style>
```

```
    div p {
```

```
      color: red;
```

```
    }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <div>
```

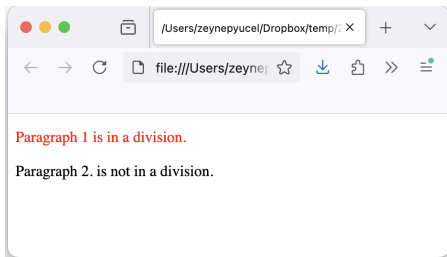
```
    <p>Paragraph 1 is in a division.</p>
```

```
  </div>
```

```
  <p>Paragraph 2. is not in a division.</p>
```

```
</body>
```

```
</html>
```



Child selectors

- Child selector is very similar to descendant selector, but has different functionality.
- Consider the following example:

```
body > p {  
    color: #000000;  
}
```

- This rule will render all the paragraphs in black, if they are a direct child of the <body> element.
- Other paragraphs put inside other elements like <div> or <td> would not have any effect of this rule.

Pseudo-class selectors

- These selectors are used to assign styles to "phantom classes" that are inferred by the state of certain elements, or markup patterns within the document, or even by the state of the document.
- For example,
 - The below will match unvisited links.
`a:link { color blue; }`
 - The below will match visited links.
`a:visited { color red; }`
 - The below will match links over which the mouse hovers.
`a:hover { color: yellow; }`
 - The below signifies an element on which the user is clicking.
`a:active { color: black; }`

Attribute selectors

- You can also apply styles to HTML elements with particular attributes.
- The style rule below will match all the input elements having a type attribute (possibly with a value).
- There are following rules applied to attribute selector.
 - `p[lang]` selects all paragraph elements with a `lang` attribute.
 - `p[lang="fr"]` selects all paragraph elements whose `lang` attribute has a value of exactly `fr`.
 - `p[lang~="fr"]` Selects all paragraph elements whose `lang` attribute contains the word `fr` (French (Belgium) `fr-BE`, French (Canada): `fr-CA`).

CSS measurement units

- CSS supports a number of measurements including absolute units such as inches, centimeters, points, and so on, as well as relative measures such as percentages and em units.
- You need these values while specifying various measurements in your style rules: `<p style="border: 1px solid red;">Content</p>`
- All CSS measurement units along with examples follow:

Unit	Description	Example
%	Relative to another value	<code>p {line-height:125%;}</code>
cm	centimeters	<code>div {margin-bottom: 2cm;}</code>
em	Size of a given font	<code>p {letter-spacing: 7em;}</code>
ex	relative to height of letter x	<code>p {line-height: 3ex;}</code>
in	inches	<code>p {word-spacing: .15in;}</code>
mm	millimeters	<code>p {word-spacing: 15mm;}</code>
pc	picas (1in=6picas)	<code>p {font-size: 20pc;}</code>
pt	points (point=1in/72)	<code>body {font-size: 18pt;}</code>
px	screen pixels	<code>p {padding: 25px;}</code>

Font Size

- To avoid the resizing problem with Internet Explorer, many developers use `em` instead of `px`.
- The `em` size unit is recommended by the W3C.
- `1em` is equal to the current font size.
- The default text size in browsers is `16px`. So, the default size of `1em` is `16px`.
- The size can be calculated from pixels to `em` using `pixels/16=em`

```
h1 {font-size:2.5em;} /* 40px/16=2.5em */
```

```
h2 {font-size:1.875em;} /* 30px/16=1.875em */
```

Font Family

- The font family of a text is set with the font-family property.
- The font-family property should hold several font names as a **fallback** system.
- If the browser does not support the first font, it tries the next font.

```
p{  
    font-family:"Times New Roman", Times, serif;}  
}
```

CSS color definitions

- CSS uses color values to specify a color.
- Typically, these are used to set a color either for the foreground of an element (i.e. its text) or for the background of the element.
- They can also be used to affect the color of borders and other decorative effects.
- You can specify your color values in various formats.

Format	Syntax	Example
Hex Code	#RRGGBB	p{color:#FF0000;}
Short Hex Code	#RGB	p{color:#6A7;}
RGB %	rgb(rr%,gg%,bb%)	p{color:rgb(5%,0%,0%);}
RGB Absolute	rgb(rrr,ggg,bbb)	p{color:rgb(0,0,255);}
keyword	aqua, black, etc.	p{color:teal;}

Lab of WEB Technologies

Lectures # 6

Cascading Style Sheets (CSS) 2/2

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

CSS: Color properties

- The CSS color properties are used to add color effects for elements:
 - ▶ `color: DodgerBlue;`
 - ▶ `background-color: Tomato;`
 - ▶ `border:2px solid BlueViolet;`
- Colors can be specified using different formats:
 - ▶ Name (+100 color names)
`Red, Green, Aqua, Tomato, DodgerBlue, ...`
 - ▶ RGB representation (RED, GREEN, and BLUE)
`rgb(255, 0, 0), rgb(255, 255, 255), rgb(100, 10, 15),...`
 - ▶ Hexadecimal color (`#RRGGBB`)
`#ff0000, #84d210, #846210, ...`
 - ▶ HSL representation (Hue, Saturation, and Lightness Form)
`hsl(0, 100%, 50%), hsl(248, 53%, 58%),...`
 - ▶ ...
- The color property is **inherited** by all the descendants of a certain element.
 - ▶ Since color is inherited, it is theoretically possible to set all of the ordinary text in a document to a color, such as red, by declaring `body{color: red;}`.

CSS: Background color and image

- Default background-color is "transparent" (i.e. no background).
- It affects the background of any element and extends all the way to the outer edge of the border.
 - ▶ So any border with transparent parts (like dashed) will have the background color fill into those transparent parts.
- Some CSS background properties follow.
 - ▶ Setting a background color.
`background-color: Tomato;`
 - ▶ Using an image as background.
`background-image: url("cumulus.png")`
 - ▶ Further possibilities for background images.
 - ▶ Repeating images only horizontally (x) or vertically (y). e.g.
`background-repeat: repeat-x;`
 - ▶ Setting the starting position of a background image, e.g.
`background-position: right top;`
 - ▶ Setting a background image which scrolls with the page
`background-attachment: scroll;`
 - ▶ Setting a fixed background image (i.e. not scrolling)
`background-attachment: fixed;`

CSS: Background shorthand property

- The CSS allows also a short declaration for the background.
- For instance, instead of the following long version

```
body {  
    background-color: #ffffff;  
    background-image: url("cat.png");  
    background-repeat: no-repeat;  
    background-position: right top;  
}
```

one can use the below short version.

```
body {  
    background: #ffffff url("cat.png") no-repeat right top;  
}
```

CSS: Fonts and font families

- Setting fonts and their properties is among the most common uses of CSS.
- Setting the property can be tricky, because not all browsers have access (by default) to any existing font.
- Users can add additional fonts in their OS.
- To maximize the compatibility, CSS allows to define fonts in terms of **Font Families**.
 - ▶ A "font" is usually composed of many variations to describe bold text, italic text, and so on.
 - ▶ Each of these variants is an actual font face, the group of all the font faces of the same type (like Times) is called a font family.
 - ▶ CSS defines 5 generic font families: **Serif**, **Sans-serif**, **Monospace**, **Cursive**, **Fantasy**.

CSS: More on font families

- **Serif**: The font is proportional (all characters have different widths) and contain serifs (decorations on the ends of strokes within each character, such as little lines at the top and bottom of a lowercase l).
- **Sans-Serif**: The font is proportional and does not contain the serifs ("sans" = "without" in French).
- **Monospace**: is not-proportional (each character uses the same amount of space) and is often used to display code or data.
- **Cursive**: attempts to emulate human handwriting or lettering.
- **Fantasy**: All other fonts. Browsers are likely to put any fonts they cannot classify as serif, sans-serif, monospace, or cursive into this.

Georgia is an example of a Serif font

Arial is an example of a Sans-Serif font

Courier new is an example of monospace font

Comic Sans is an example of a cursive font

- **font-family** property can be used to specify the font face and the "fallback" font family to use, if the specified font face is not available:

```
h1 {font-family: Georgia, serif;}
```

CSS: Other font properties 1/2

- `font-style` property can be set to normal, italic or oblique.

Normal font style

Italic font style

Oblique font style

- `font-variant` property can create a small-caps effect (or normal).

font-weight: bold

font-style: italic

FONT-VARIANT: SMALL-CAPS

- `font-weight` property sets how bold or light a font appears.
 - ▶ Possible values are normal, bold, bolder, lighter, 100, 200, ..., 900.

CSS: Other font properties 2/2

- **font-size** property sets the size of a font.
 - ▶ Possible values could be xx-small, x-small, small, medium, large, x-large, xx-large, smaller, larger (relative to parent element's font-size), in pixels or in %.
- **font-size-adjust** property adjusts the x-height to make fonts more legible. Possible value could be any number.
- **font-stretch** property displays expanded or condensed version of the font.
 - ▶ Possible values could be normal, wider, narrower, ..., ultra-expanded.
- **font** property is shorthand to specify several properties at once.

```
<p style="font:italic small-caps bold 15px georgia;">  
Applying all the properties on the text at once.  
</p>
```


CSS: Basic text properties

- `color` property is used to set the color of a text.
- `direction` property is used to set the text direction (with `ltr` or `rtl`, e.g. for hebrew, arabic etc).
- `letter-spacing` property is used to add or subtract space between the letters that make up a word (e.g. `letter-spacing:5px;`).
- `word-spacing` property is used to add or subtract space between the words of a sentence.
- `text-indent` property is used to indent the text of the first line of a paragraph.
- `text-align` property is used to align the text of a document (`left`, `right`, `center`, `justify`).

CSS: Text decoration

- `text-decoration` property is used to underline, overline, and line-through (strikethrough) text.
- `text-transform` property is used to convert text to uppercase or lowercase or capitalize it (with only first letters as uppercase).
- `white-space` property is used to control the flow and formatting of text (`normal` for wrapping, `pre` for preserving, `nowrap` without wrapping.).
- `text-shadow` property is used to set the text shadow around a text (additional settings are offset, blur radius, and color).

CSS: Images

- You can set the following image properties using CSS.
 - ▶ `border` property is used to set the width of an image border.
 - ▶ `height` and `width` properties are used to set the dimensions of an image.
 - ▶ `-moz-opacity` property is used to set the opacity of an image and used to create a transparent image in Mozilla.

CSS: Tables

- You can set the following properties of a table:
 - ▶ The `border-collapse` specifies whether the browser should control the appearance of the adjacent borders that touch each other or whether each cell should maintain its style (`collapse` and `separate`).
 - ▶ The `border-spacing` specifies the width that should appear between table cells.
 - ▶ The `caption-side` captions are presented in the `<caption>` element. You use the `caption-side` property to control the placement of the table caption (`top`, `bottom`).
 - ▶ The `empty-cells` specifies whether the border should be shown if a cell is empty (`show`, `hide`).

CSS: Cursor

- The cursor property of CSS allows you to specify the type of cursor that should be displayed to the user.
- One good usage of this property is in using images for submit buttons on forms.
- By default, when a cursor hovers over a link, the cursor changes from a pointer to a hand.
- In contrast, the cursor remains unchanged, when hovering over a submit button within a form.
- As a result, when users hover over an image used as a submit button, it provides a visual cue indicating that the image is clickable.
- Some options are `crosshair` (plus sign), `default` (arrow), `pointer` (a pointing hand), `move` ("I" bar).

CSS: Lists

- With the following CSS properties, one can control lists:
 - ▶ `list-style-type` allows you to control the shape or appearance of the marker (many options like `square`, `lower-alpha`, `lower-hebrew`, `katakana-alpha` etc.).
 - ▶ `list-style-position` specifies whether a long point that wraps to a second line should align with the first line or start underneath the start of the marker (e.g. `inside`).
 - ▶ `list-style-image` specifies an image for the marker rather than a bullet point or number.
 - ▶ `list-style` serves as shorthand for the preceding properties.
 - ▶ `marker-offset` specifies the distance between a marker and the text in the list.

Specificity

- It is possible that the same element could be selected by two or more rules, each with its own selector.
- How do we know which rule will win?
- For every rule, the user agent evaluates the specificity of the selector and attaches it to each declaration in the rule.
- When an element has two or more conflicting property declarations, the one with the highest specificity will win.

Specificity rules

- There are four categories which define the specificity level of a selector:
 - ▶ Elements (e.g. `h1`)
 - ▶ Classes, pseudo-classes, attribute selectors (e.g. `.test`, `:hover`, `[href]`)
 - ▶ IDs (e.g. `#navbar`)
 - ▶ styles (e.g. `<h1 style="color: pink;">`)
- How to calculate specificity?
 - ▶ Start at 0
 - ▶ Add 1 for each element selector
 - ▶ Add 10 for each class value (or pseudo-class or attribute selector)
 - ▶ Add 100 for each ID value.
 - ▶ Add 1000 for inline style. This always given the highest priority!

Examples for calculating specificity

- `h1 {color: red;}`
specificity = ?
- `p em {color: purple;}`
specificity = ?
- `.grape {color: purple;}`
specificity = ?
- `*.bright {color: yellow;}`
specificity = ?
- `p.bright em.dark {color: maroon;}`
specificity = ?
- `#id216 {color: blue;}`
specificity = ?
- `<h3 style = "color: blue;">`
specificity = ?

Examples for calculating specificity

- `h1 {color: red;}`
specificity = 0,0,0,1
- `p em {color: purple;}`
specificity = ?
- `.grape {color: purple;}`
specificity = ?
- `*.bright {color: yellow;}`
specificity = ?
- `p.bright em.dark {color: maroon;}`
specificity = ?
- `#id216 {color: blue;}`
specificity = ?
- `<h3 style = "color: blue;">`
specificity = ?

Examples for calculating specificity

- `h1 {color: red;}`
specificity = 0,0,0,1
- `p em {color: purple;}`
specificity = 0,0,0,2
- `.grape {color: purple;}`
specificity = ?
- `*.bright {color: yellow;}`
specificity = ?
- `p.bright em.dark {color: maroon;}`
specificity = ?
- `#id216 {color: blue;}`
specificity = ?
- `<h3 style = "color: blue;">`
specificity = ?

Examples for calculating specificity

- `h1 {color: red;}`
specificity = 0,0,0,1
- `p em {color: purple;}`
specificity = 0,0,0,2
- `.grape {color: purple;}`
specificity = 0,0,1,0
- `*.bright {color: yellow;}`
specificity = ?
- `p.bright em.dark {color: maroon;}`
specificity = ?
- `#id216 {color: blue;}`
specificity = ?
- `<h3 style = "color: blue;">`
specificity = ?

Examples for calculating specificity

- `h1 {color: red;}`
specificity = 0,0,0,1
- `p em {color: purple;}`
specificity = 0,0,0,2
- `.grape {color: purple;}`
specificity = 0,0,1,0
- `*.bright {color: yellow;}`
specificity = 0,0,1,0
- `p.bright em.dark {color: maroon;}`
specificity = ?
- `#id216 {color: blue;}`
specificity = ?
- `<h3 style = "color: blue;">`
specificity = ?

Examples for calculating specificity

- `h1 {color: red;}`
specificity = 0,0,0,1
- `p em {color: purple;}`
specificity = 0,0,0,2
- `.grape {color: purple;}`
specificity = 0,0,1,0
- `*.bright {color: yellow;}`
specificity = 0,0,1,0
- `p.bright em.dark {color: maroon;}`
specificity = 0,0,2,2
- `#id216 {color: blue;}`
specificity = ?
- `<h3 style = "color: blue;">`
specificity = ?

Examples for calculating specificity

- `h1 {color: red;}`
specificity = 0,0,0,1
- `p em {color: purple;}`
specificity = 0,0,0,2
- `.grape {color: purple;}`
specificity = 0,0,1,0
- `*.bright {color: yellow;}`
specificity = 0,0,1,0
- `p.bright em.dark {color: maroon;}`
specificity = 0,0,2,2
- `#id216 {color: blue;}`
specificity = 0,1,0,0
- `<h3 style = "color: blue;">`
specificity = ?

Examples for calculating specificity

- `h1 {color: red;}`
specificity = 0,0,0,1
- `p em {color: purple;}`
specificity = 0,0,0,2
- `.grape {color: purple;}`
specificity = 0,0,1,0
- `*.bright {color: yellow;}`
specificity = 0,0,1,0
- `p.bright em.dark {color: maroon;}`
specificity = 0,0,2,2
- `#id216 {color: blue;}`
specificity = 0,1,0,0
- `<h3 style = "color: blue;">`
specificity = 1,0,0,0

Inheritance

- Inheritance is the mechanism by which some styles are applied not only to a specified element, but also to its descendants.

- Consider the below:

```
h1 {color: gray;}  
<h1>Meerkat <em>Central</em></h1>
```

- Here, both the h1 text and em are colored gray, because em is a descendant of h1.
- All the box-model properties (see next slides) are not inherited to avoid undesirable outcomes.
- Inherited values **have no specificity at all**, not even zero specificity.
- This has important implications...

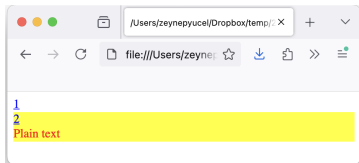
No specificity example 1/2

- Suppose to have:

```
a:link {color: blue;}  
#toolbar {color: red; background: yellow;}
```

- We expect that all text in a "toolbar" to be red on yellow.
- However, this will work only when the element with an id of toolbar contains nothing but plain text.
- Consider the below

```
<a href="1.html">1</a>  
<div id="toolbar">  
  <a href="2.html">2</a>  
  <br>  
  Plain text  
</div>
```

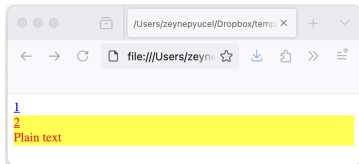


- For the above case, the rule has no specificity for a, thus the a:link rule takes over. How to fix?

No specificity example 2/2

- Here is the fix:

```
<head>
  <style>
    a:link {color: blue;}
    #toolbar {
      color: red;
      background: yellow;
    }
    #toolbar a:link {
      color: red;
    }
  </style>
</head>
<body>
  <a href="1.html">1</a>
  <div id="toolbar">
    <a href="2.html">2</a>
    <br>
    Plain text
  </div>
</body>
```



Cascade

- What happens when two rules of equal specificity apply to the same element?
- For example, the below have the same specificity of 0,0,0,1

```
h1 {color: red;}  
h1 {color: blue;}
```

- The one that is defined last in the stylesheet is the one that will be used (in this case blue).

Recap of (simplified) cascade rules

- CSS is based on a method of causing styles to cascade together, which is made possible by combining inheritance and specificity:
 1. Find all rules that contain a selector that matches a given element.
 2. Sort all declarations applying to the given element by specificity. Those elements with a higher specificity have more weight than those with lower specificity.
 3. Sort all declarations applying to the given element by order. The later a declaration appears in the style sheet or document, the more weight it is given.

!Important rule

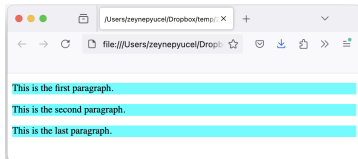
- !important rule allows one to override all other styling rules for that specific property on that element.

```
#myid { background-color: yellow;}  
.myclass { background-color: pink;}  
p { background-color: cyan !important;}
```

```
<p>This is the first paragraph.</p>
```

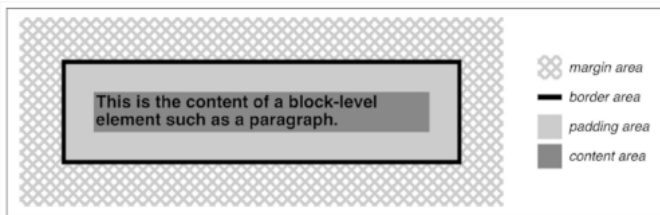
```
<p class="myclass">This is the second paragraph.</p>
```

```
<p id="myid">This is the last paragraph.</p>
```

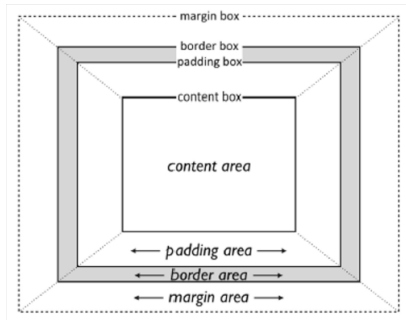


Element boxes

- CSS assumes that every element generates one or more rectangular boxes, called **element boxes**.
- Each element box has a **content area** at its center.
- Content area is surrounded by optional amounts of **padding**, **borders**, **outlines**, and **margins**.

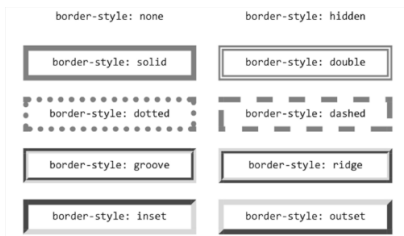
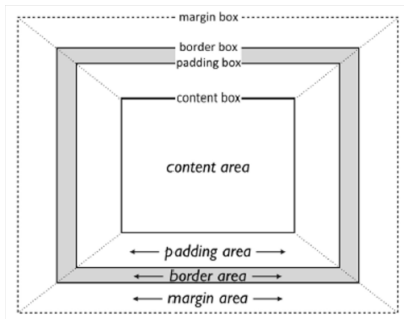


Content and padding



- Size of content area can be as follows:
`#box {width:60px; height:15px; }`
- Padding area size can be adjusted affecting all the sides at once setting the property `padding`
- Or adjusting it on each side separately setting `padding-top, padding-bottom, padding-right, padding-left`

Border and margin



- The border of an element is just one or more lines that surround the content and padding of an element.
- `border-width` is used to set the border thickness.
- `border-style` is used to change the style.
- Setting a margin creates extra blank space around an element.
- Margin area size can be changed with the property `margin` (affecting all the sides) or each side separately:

`margin-top`, `margin-bottom`,
`margin-right`, `margin-left`

Block positioning

- By default, a visually rendered document is composed of a number of rectangular boxes that are distributed so that they do not overlap.
- The `position` and `offset` properties allow us to manually position a certain box with respect to the containing block.

Five properties of position

1. **Static**: is generated as normal (i.e. the box is not positioned). Block-level elements generate a rectangular box that is part of the document's flow.
2. **Relative**: The element's box can be offset by some distance (i.e. the box is positioned). The element retains the shape it would have had were it not positioned, and the space that the element would ordinarily have occupied is preserved.
3. **Absolute**: The element's box is completely removed from the flow of the document and positioned with respect to its containing positioned block. Whatever space the element might have occupied in the normal document flow is closed up, as though the element did not exist.
4. **Fixed**: The element's box behaves as though it was set to absolute, but its containing block is the browser window itself.
5. **Sticky**: It is positioned based on the user's scroll position. It is left in the normal flow, until the conditions that trigger its stickiness.

Offset

- relative, absolute, sticky, and fixed positioning use four distinct properties to describe the offset of a positioned element's sides with respect to its containing block: left, right, top, bottom
- Values of the properties can be a length (in px, mm, etc.) or a percentage.
- top, left, right, bottom describe an offset from the sides of the containing block.
- For example, top describes how far the top margin edge of the positioned element is to be placed from the top of its containing block.
- Positive values move the top margin edge of the positioned element downward, while negative values move it above the top of its containing block.
- By using negative offset values, it is possible to position an element outside its containing block

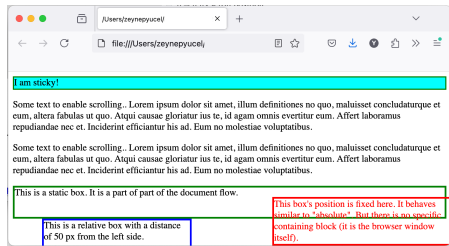
Width and height

- Width and height properties can be used to modify the size of a box.

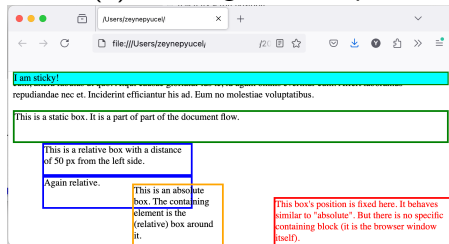
```
div {  
  height: 100px;  
  width: 50%;  
  border: 1px solid #ffffff;  
}
```

Example for various block properties

```
div.static { position: static;
height: 50px; border: 3px solid green;
}
div.relative { position: relative;
left: 50px;
height: 50px; width: 250px;
border: 3px solid blue;
}
div.fixed { position: fixed;
bottom: 0; right: 0;
width: 300px; color:red;
border: 3px solid red;
}
div.absolute { position: absolute;
top: 10px; left: 150px;
width: 150px; height: 100px;
border: 3px solid orange;
}
div.sticky { position: sticky;
top: 0; background-color: cyan;
border: 2px solid green;
}
```



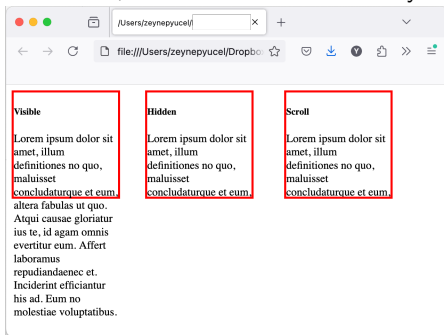
(a) scrolling to the top



(b) scrolling to the bottom

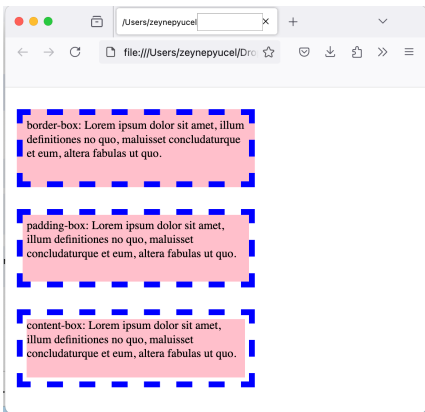
Overflow

- If the content of an element is too much for the element's size, it will overflow the element itself.
- The overflow property controls how to handle this situation.
 - ▶ **visible** Default. The overflow is not clipped. The content renders outside the element's box.
 - ▶ **hidden**: The content is clipped, and the rest will be invisible.
 - ▶ **scroll**: The content is clipped, and a scrollbar is added to see the rest.
 - ▶ **auto**: Similar to scroll, but it adds scrollbars only when necessary.



Background-clip

- This property specifies how far the background should extend within an element.



Possible values

- ▶ **border-box**: Default value. Background extends behind the border.
- ▶ **padding-box**: Background extends to the inside edge of the border.
- ▶ **content-box**: Background extends to the edge of the content box.
- ▶ **inherit**: Inherit this property from the parent element.

Lab of WEB Technologies

Lecture # 7 CSS assignment

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

Assignment description

We have prepared a simple web site for a travel agency. This time we will use CSS to make it look more appealing.

- You may work on the navigation bar and design it in different ways.
- You can work on the text, place it in containers, center and justify etc.
- You may work on how your images, list items, links etc look.
- You may work on the form to resize the form fields, apply shadowing effect, highlight active fields etc.
- You may use our knowledge from the lectures, but you are also welcome to get creative and discover additional features.

Lab of WEB Technologies

Lecture # 8 JavaScript 1/2

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

Fundamental concept

- HTML is a simple text markup language.
- HTML **can not handle behaviors** of web applications (e.g. respond to the user, make decisions, or automate tasks and actions).
- These require a programming or script language (e.g. JavaScript)
- Examples of JavaScript usage
 - ▶ Display/hide dynamically messages or parts of a web page.
 - ▶ Animate images or create images that change with the mouse movements.
 - ▶ Validate the content of forms and make calculations.
 - ▶ ...

Introduction 1/2

- JavaScript is the most common dialect (implementation) of a language standardized by Netscape to the European Computer Manufacturer's Association known as [ECMAScript](#).
- Almost all browsers support ECMAScript version >5 .
- Despite its standard name, in practice just about everyone calls the language "JavaScript".
- JavaScript = ECMAScript
- JavaScript $\neq$ Java

Introduction 2/2

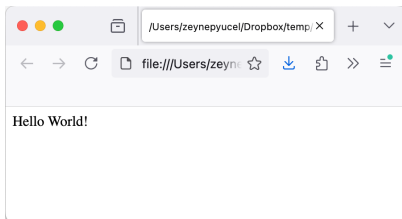
- JavaScript is a **high-level, dynamic, untyped, interpreted** programming language.
 - ▶ **High-level** : A high-level programming language provides **abstraction from the machine's hardware**, enabling developers to write code that is easier to read and maintain, typically using syntax closer to human languages.
 - ▶ **Dynamic**: A dynamic programming language allows types to be determined at **runtime** rather than at compile time, enabling more flexible coding practices.
 - ▶ **Untyped**: An untyped language does not enforce data types on variables, allowing values to be treated in a more fluid manner without strict type constraints.
 - ▶ **Interpreted**: An interpreted language is executed line-by-line by an interpreter **at runtime**, rather than being compiled into machine code beforehand, which can facilitate debugging and immediate feedback.
- JavaScript is well-suited to object-oriented and functional programming styles.

Hello World in JavaScript

- Let us take an example to print out "Hello World".
- One way of doing this is to call a function `document.write` which writes a string into our HTML document.

```
<body>  
  <script language="javascript" type="text/javascript">  
    document.write ("Hello World!")  
  </script>  
</body>
```

- If your browser supports JavaScript, it should produce a result as below:



JavaScript syntax 1/4

Whitespaces and indentation

- JavaScript is a **case-sensitive** language. This means that variables, function names etc. must be typed with a consistent capitalization.
- JavaScript **ignores whitespace** characters such as spaces, tabs, and newlines.
- You can use these freely to **format and indent** your code, making it **more readable** and easier to understand.

JavaScript syntax 2/4

Comments

- JavaScript supports C-style and C++-style comments.
- Text following `//` until the end of the line is a comment.
- Text between `/*` and `*/` is also a comment and can span multiple lines.
- JavaScript recognizes HTML comment opening `<!--` as a single-line comment.

```
<script language="javascript" type="text/javascript">  
  <!--  
    // This is a comment in C++ style  
    /*  
      * This is a multiline comment in JavaScript  
      * It is very similar to comments in C Programming  
      */  
    //--> HTML closing comment is not recognized, so use this instead.  
</script>
```

JavaScript syntax 3/4

Variable declaration and assignment

- To create a variable in JavaScript, use the `let` keyword.
- In older scripts, you may also find `var` instead of `let`.
- In JavaScript, simple statements **typically end with a semicolon**, similar to C, C++, and Java.

```
<script language="javascript" type="text/javascript">  
    let a = 1 // OK  
    let b = 2; // Also OK  
</script>
```

- If each statement is on a **separate line**, you can **skip the semicolon**. But if they are on the same line, you must include semicolons.

```
<script language="javascript" type="text/javascript">  
    let a = 1; // OK  
    let c = 3; let d = 4; // Also OK  
</script>
```

JavaScript syntax 4/4

Variable declaration and assignment

- Before you use a variable in a JavaScript program, you must declare it.

```
<script language="javascript" type="text/javascript">  
  let a = 1 // OK  
  b = 2; // NOT OK. b is still not declared.  
  let c = 3, d = 4; // OK. declares and assigns values to c and d  
  let e = 5; f = 6; // We have a problem!  
  // OK to declare and assign a value to e  
  // NOT OK to assign a value to f without declaring  
</script>
```

- In general, it is a good programming practice to use semicolons and new lines for each statement.

Variable types

- JavaScript is **untyped** language.
- This means that you **do not have to tell** JavaScript during variable declaration **what type** of value the variable will hold.
- The value **type of a variable can change** during the execution of a program and JavaScript takes care of it automatically.
- If we attempt to use (read) an undeclared variable, an error is generated (this kind of error is called exception).

JavaScript variable scope

- The **scope of a variable** is the region of your program in which it is defined.
- JavaScript variables have only two scopes.
 - ▶ **Global Variables**: A global variable has global scope which means it can be defined anywhere in your JavaScript code.
 - ▶ **Local Variables**: A local variable will be visible only within a function where it is defined.
- Function parameters are always local to that function.
- Within the body of a function, a **local variable takes precedence** over a global variable with the same name.
- In other words, if you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable.

JavaScript variable names

- While naming your variables in JavaScript, keep the following rules in mind.
- You should **not use** any of the JavaScript **reserved keywords** as a variable name.
 - ▶ For example, break, boolean, while, class etc are invalid as variable names.
- JavaScript variable names should **not start with a numeral** (0-9). They must begin with a letter or an underscore character.
 - ▶ For example, 123test is an invalid variable name but `_123test` is a valid one.
- JavaScript variable names are case-sensitive (e.g. Name and name are two different variables).

Input-output functionality

- The core JavaScript language defines a minimal API for **working with text, arrays, dates and regular expressions**, but not include any input-output functionality.
- In other words, the language itself **does not provide built-in mechanisms** for tasks like reading from files, writing outputs to files, or handling direct user input from devices (like keyboards or mice) in a native way.
- Input-output operations usually **require interaction with the environment** in which JavaScript is running.
 - ▶ In a web browser, you might use the Document Object Model (DOM) to interact with the user interface (like handling button clicks or form submissions).
 - ▶ In server-side environments, JavaScript can perform I/O operations through various modules or libraries specifically designed for file system access and network requests.

Data types and values

- The kind of values that can be represented and manipulated in a programming language are known as **types**.
- The set of types it supports is one of the most important characteristics of a programming language.
- When a program needs to **retain a value** for future use, or manipulate it, it **assigns the value to a variable**.
- A variable defines a **symbolic name** for a value so that such value can be referred by that name.

Data types

- JavaScript types can be divided in two categories
 1. Primitive types:
 - ▶ Numbers
 - ▶ Strings (text)
 - ▶ Booleans (true/false)
 - ▶ null and undefined
 2. Object types: (everything that is not primitive)
 - ▶ An object is a collection of properties, where each property has a **name and a value**.
 - ▶ An array is an ordered collection of numbered values
 - ▶ A function is an object with executable code associated to it.
- JavaScript converts values liberally from one type to another.
 - ▶ **Example:** If a program expects a string and we give it a number, it will automatically convert the number to a string.
 - ▶ The rules for value conversion are in general not trivial and sometimes prone to errors.

Operators

JavaScript supports the following types of operators.

- Arithmetic Operators
- Comparison Operators
- Logical (or Relational) Operators
- Assignment Operators
- Conditional (or ternary) Operators

Arithmetic operators

JavaScript supports the following arithmetic operators:

- + Addition
- - Subtraction
- * Multiplication
- / Division
- % Modulus
- ++ Increment
- -- Decrement

Comparison operators

JavaScript supports the following comparison operators:

- `==` (Equal)
- `!=` (Not Equal)
- `>` (Greater than)
- `<` (Less than)
- `>=` (Greater than or Equal to)
- `<=` (Less than or Equal to)

Logical operators 1/3

JavaScript supports the following logical operators:

- `&&` (Logical AND) If both the operands are non-zero, then the condition becomes true.
- `||` (Logical OR) If any of the two operands are non-zero, then the condition becomes true.
- `!` (Logical NOT) Reverses the logical state of its operand.

Bitwise logical operators 1/2

Binary operators

- Javascript supports logical AND, OR and XOR operations.

		AND	OR	XOR
1	1	1	1	0
1	0	0	1	1
0	1	0	1	1
0	0	0	0	0

- Assume $A = 2$ and $B = 3$ (i.e. 10 and 11, respectively).
 - ▶ $\&$ (Bitwise AND) performs an AND operation on each bit of its integer arguments.
 - ▶ $(A \& B)$ is 2, since the bit representations of A and B are 10 and 11, respectively.
 - ▶ $|$ (BitWise OR) performs an OR operation on each bit of its integer arguments.
 - ▶ $(A | B)$ is 3.
 - ▶ $\sim$ (Bitwise XOR) performs an exclusive OR operation on each bit of its integer arguments.
 - ▶ $(A \sim B)$ is 1.

Bitwise logical operators 2/2

Unary operator of NOT

- JavaScript supports also the unary operator of NOT.
- $\sim$ (Bitwise NOT) is a unary operator, reversing all the bits in the operand.
 - ▶ If $B = 3$, then $(\sim B)$ is -4.
- JavaScript represents numbers in 32-bit signed binary format.
- So $B=3$ is represented with
0000 0000 0000 0000 0000 0000 0000 0011.
- The bitwise NOT operator flips each bit:
1111 1111 1111 1111 1111 1111 1111 1100.
- The most significant bit (MSB), leftmost, acts as the sign bit.
 - ▶ If MSB is 0, then positive or zero. If MSB is 1, then negative.
- A binary number with an MSB of 1 can be converted to decimal by:
 - ▶ Inverting the bits (change all 1s to 0s and all 0s to 1s).
 - ▶ Adding 1 to the inverted binary number.
 - ▶ Converting that from binary to decimal
 - ▶ Adding a negative sign

Shift operators

- $\ll$ (Left Shift) moves all the bits in its first operand to the left by the number of places specified in the second operand. New bits are filled with zeros.
 - ▶ Shifting a value left by one position is equivalent to multiplying it by 2, shifting two positions is equivalent to multiplying by 4, and so on.
 - ▶ If $A = 2$, then $(A \ll 1)$ is 4.
- $\gg$ (Right Shift) Binary Right Shift Operator.
 - ▶ The left operand's value is moved right by the number of bits specified by the right operand.
 - ▶ If $A = 2$, then $(A \gg 1)$ is 1.

Assignment operators

JavaScript supports the following assignment operators:

- `=` (Simple Assignment) assigns values from the right side operand to the left side operand
- `+=` (Add and Assignment) adds the right operand to the left operand and assigns the result to the left operand.
- `-=` (Subtract and Assignment)
- `*=` (Multiply and Assignment)
- `/=` (Divide and Assignment)
- `%=` (Modules and Assignment) takes modulus using two operands and assigns the result to the left operand.
 - ▶ `C %= A` is equivalent to `C = C % A`.
- Same logic applies to Bitwise operators, so they will become `<<=`, `>>=`, `&=`, `|=` and `^=`.

Other miscellaneous operators

JavaScript supports the following assignment operators:

- (`? :`) (Conditional Operator) first evaluates an expression for a true or false value and then executes one of the two given statements depending upon the result of the evaluation.
 - ▶ For the below, if `a=10` and `b = 20`, then `result = 200`
`result = (a > b) ? 100 : 200;`
 - ▶ `typeof` is a unary operator that is placed before its single operand, which can be of any type.
 - ▶ The `typeof` operator evaluates to `"number"`, `"string"`, or `"boolean"` if its operand is a number, string, or boolean value. In other words, its value is a string.

Numbers

- JavaScript does not make a distinction between integer values and floating-point values. All numbers in JavaScript are represented as floating-point values.
- Numbers can be manipulated with the common arithmetic operators (+ - */) and modulus (%), increment (++) and decrement -- operators.
- Note that addition operator (+) works for numbers as well as strings, e.g. "a" + 10 will give "a10", casting 10 as a string.

Math operators

- To manipulate numbers, you can also use some of the useful operators provided by the Math object.

```
Math.pow(2,53) // 2 to the power of 53
Math.round(0.6) // round to the nearest integer
Math.ceil(0.6) // round up to an integer
Math.floor(0.6) // round down to an integer
Math.random() // a random number 0<=x<1
Math.sqrt(3) //The square root of 3
Math.PI // The value of PI
Math.min(x,y,z) // The smallest value between x, y, z
Math.max(x,y,z) // The largest value between x, y, z
```

Strings

- A string is an immutable sequence of characters used to represent a text.
- To include a string literally in JavaScript, simply enclose the characters of the string in a matched pair of single or double quotes:

```
"hello world"
```

```
'hello world'
```

- The `+` operator applied to two strings is used to obtain a new string in which the two operands are joined.

```
"hello" + "world"
```

- To determine the length of a string we can use the `length` property:
For example:

```
"hello".length
```

String functions

- Suppose `s` is a variable containing a string value:

`s.charAt(n)` // returns the character in position `n`

`s.substring(m,n)` // returns the substring from `m`th to `n`th characters

`s.indexOf(c)` // returns the position of the first occurrence
of `c` in `s`

`s.lastIndexOf(c)` // returns the position of the last occurrence of
`c` in `s`

`s.split(c)` // splits `s` into substrings using `c` as separator

`s.replace(c1,c2)` // replaces every occurrence of `c1` with `c2`

`s.toUpperCase()` // converts `s` to uppercase

Boolean values

- A Boolean value represents truth or falsehood.
- This type has only two values: `true` and `false`.
- A Boolean value is usually the result of a comparison performed with a comparison operator.
- For example: `5 == 4` results in `false`.
- Values that are coerced into true and false are called `truthy` and `falsy`.
- Some of JavaScript's falsy values are `false`, `0`, `-0`, `0n` (`BigInt`), `""`, `''` (empty strings), `null`, `undefined`, `NaN`, `document.all`.
- Also, an empty array is coerced to false.
`[] == false; // -> true`
- All other arrays, strings etc. are `truthy` (but not `true`).

null / undefined

- `null` and `undefined` are two types used to indicate the absence of a value.
- What is the difference between the two?
- `undefined` represent a system-level, unexpected or error-like absence of a value.
- `null` represent a program-level, normal or expected absence of a value.

Some JavaScript quirks 1/2

- `[] == 0; // true`

An empty array is coerced to a number (without becoming a boolean first). So it is coerced to 0, which is equivalent to 0, thus true.

- `![] == 0; // true`

An empty array (i.e. truthy) is inverted (i.e. false), which is then coerced to 0. 0 is equivalent to 0, thus true.

- `typeof NaN // "number"`

NaN just a number that a computer can not calculate, so it is a number.

- `true === 1 // false`

The operator `===` is called strict equality. Neither value is implicitly converted to some other value before being compared. If the values have different types, the values are considered unequal.

- `true == 1 // true`

This is an example of implicit type coercion. Implicit type coercion is being triggered when you apply operators to values of different types (e.g. `2+'2'`, `'true'+false`). In this case, the boolean value `true` is coerced to the number 1.

Some JavaScript quirks 2/2

- `Math.max() // -Infinity`

`-Infinity` is an identity element of `Math.max()`. So when no arguments are passed, it returns `-Infinity`.

- `Math.min() // Infinity`

Similarly, when no arguments are passed, `Math.min()` returns `Infinity`.

- `[] + [] // ""`

Both arrays return an empty string. Thus the result is a concatenation of these (check primitive conversions in JS).

- `{ } + [] // 0`

The first `{ }` is interpreted as a code block. So it is ignored. What is left is `+[]`. The `+` is an unary prefix operator that converts the operand (`[]`) into a number (`0`).

- `(!+[]+[]+![]).length // 9`

Breaking it down at unary prefix, we get `!0 + [] + false`, which is `true + [] + false`. That is `true + '' + false`. Concatenating we get `truefalse` of length 9.

Lab of WEB Technologies

Lecture # 9 JavaScript 2/2

Zeynep Yücel

Ca' Foscari University of Venice
zeynep.yucel@unive.it
yucelzeynep.github.io

JavaScript

Appeals

- Browser Support:
 - ▶ JavaScript is the only programming language **natively supported** by all web browsers, making it the standard for client-side web development.
- Interactivity:
 - ▶ JavaScript is used to create **dynamic and interactive** user interfaces. It can manipulate HTML and CSS to update the content and design of web pages without needing to reload the page.

Expressions

- Expressions are evaluated to produce a value; statements are executed to make something happen.
- An **expression** is a phrase of JavaScript language that the interpreter can evaluate to **produce a value**.
- Constants, literal values and variable references are simple expressions.

10

"hello"

a

- Complex expressions can be created by combining simple expressions with operators:

3+a

"hello"+"world"

Statements

- **Statements** are JavaScript sentences or commands.
- The most useful kind of statements are:
 - ▶ **Expression statements** (for example assignment statements, declarations, function invocations, etc.)
 - ▶ **Conditionals** (makes the interpreter execute or skip other statements depending on the value of an expression)
 - ▶ **Loops** (used to execute statements repetitively)

Conditionals

- The `if` statement is the fundamental control statement that allows JavaScript to execute statements conditionally:

```
if (expression)
    statement (or {statement block })
else
    statement (or {statement block })
```

- `if` statement example:

```
if (n == 5)
    m = 10;
else {
    m = 20;
    f = 3;
}
```

Loops

- The while statement is JavaScript's basic loop:

```
while( expression )  
    statement (or { statement block } )
```

- In the do/while loop, the body is always executed at least once:

```
do  
    statement (or { statement block } )  
while( expression );
```

- The for statement simplifies loops that follow a common pattern

```
for( initialize ; test ; increment )  
    statement (or { statement block } )
```

- for loop example:

```
for(var i=0; i < mystring.length; i++)  
{  
    var c = mystring[i];  
    console.log(c);  
}
```

initialize

test

increment

- Usually, we use a variable to count the number of loops (i in this example).

for, while, do/while loops

- In a for loop, the increment is evaluated and the test expression is tested again.

```
for (let i = 0; i < 10; i++) {  
    text += "<br>The number is " + i;  
}
```

- In a while loop, the condition is tested again.

```
while (i < 3) {  
    text += "<br>The number is " + i;  
    i++;  
}
```

- In a do/while loop, the execution skips at the bottom of the loop where the condition is tested again.

```
do {  
    text += "<br>The number is " + i;  
    i++;  
}  
while (i < 3);
```

For/in (Foreach loop), break and continue, switch

- The for/in loop can be used to execute some statements for each property of a certain object:

```
for ( var c in string )  
    console.log(c);
```

- The break statement causes the innermost enclosing loop or switch statement to exit immediately.
- continue restarts a loop at the next iteration:

```
switch( n ) {  
    case 1:  
        statement block  
        break;  
    case 2:  
        statement block  
        break;  
    default:  
        statement block  
        break;  
}
```

Functions

- A function is a block of JavaScript code that is defined once but may be executed, or **invoked**, any number of times.
- Functions are **parametrized**, namely the function definition may include a **list of parameters** that work as **local variables** for the body of the function.
- Functions can be assigned to variables and passed to other functions as arguments (fundamental for **event programming**).
 - ▶ An **event** is an action or occurrence recognized by software, often originating asynchronously from the external environment, that may be handled by the software.
 - ▶ Event-driven programming is a programming paradigm in which the flow of the program is determined by **external events** (e.g. UI events from mice, keyboards, touchscreens)
- Functions can be defined at **runtime**.

Defining and invoking functions

- Defining a function (examples):

- ▶ Function declaration

- ```
function sum(a,b) { return a+b; }
```

- ▶ Function expression

- ```
var sum = function(a,b) { return a+b; }
```

- ▶ Using arrow functions (ES6):

- ```
var sum = (a,b) => { return a+b; }
```

- Function code is not executed when the function is defined but when it is invoked.

- Functions can be invoked different ways:

- ▶ As an invocation expression: `f(a)`

- ▶ As a method invocation, if the function is inside an object:

- ```
o.f(a); //Dot notation. More common, if method name is known
```

- ```
o["f"](a); // Bracket notation. Useful, if method name is dynamic
```

# JavaScript: Client-side vs Server-side

- JavaScript is more commonly associated with client-side applications.
- But its use on the server side has grown significantly in recent years.
- With both ways of use of the language, client-side builds and updates user interfaces and server-side serves data required for the client to work.
- This enables developers to use a single programming language for both sides.

# Examples of JavaScript usage

## Client-side examples

- **Form Validation:**

JavaScript can validate user input in forms before it is submitted to the server (e.g. email, birthday in the correct format).

- **Interactive Content:**

JavaScript can create interactive elements on web pages (e.g. image slider in a photo album).

- **Asynchronous Requests:**

JavaScript can fetch or send data without reloading the page (e.g. update a shopping cart).

- **DOM Manipulation:**

JavaScript can dynamically change the structure and content of a web page (e.g. add/remove elements).

- **Event Handling:**

JavaScript can handle user events (e.g. clicks, keyboard input).

# Examples of JavaScript usage

## Server-side examples

- **Authentication and Authorization:**

JavaScript can handle user authentication (verifying identity) and authorization (determining permissions) on the server side.

- **Real-Time Applications:**

JavaScript enables the creation of real-time applications (e.g. chat).

- **File Manipulation:**

JavaScript can handle file operations on the server, such as reading,

- **Database Interactions:**

JavaScript can interact with databases (e.g. SQL) and handle database queries, retrieval operations etc.

- **Task Scheduling:**

JavaScript can schedule and run background tasks (e.g. periodic emails, cleaning up databases).

## Loading from external file 1/2

- One way to embed JavaScript code into an HTML file is to load from an external file (referenced in <head> or <body>).
- You can load from an external file by:

- ▶ using a file path

```
<script src="js/myScript.js"></script>
```

- ▶ using a full URL

```
<script src="https://example.com/js/myScript.js"></script>
```

- In this way, the <script> tag specified with the src attribute behaves exactly as if the contents of the specified JavaScript file appeared directly between the script tags.

```
<script>
...JavaScript code here...
</script>
```

- Any content between the opening and closing script tags is ignored

```
<script src="https://example.com/js/myScript.js">
I am ignored
</script>
```



# Loading from external file 2/2

## Advantages of loading from an external file

- Placing scripts in external files has some advantages:
  - ▶ It separates HTML and code
  - ▶ It makes HTML and JavaScript easier to read and maintain
    - ▶ When multiple web pages share the same JavaScript code, using the `src` attribute allows to maintain only a single copy of that code.
  - ▶ We can use JavaScript code provided by other web servers.
  - ▶ Cached JavaScript files can speed up page loads.
    - ▶ If a file of JavaScript code is shared by more than one page, it only needs to be downloaded once

# Inline JavaScript

- You can also hold the code inline.
  - ▶ Inline within the HTML page (in the head or body section, or both) between `<script>...</script>` tags

```
<script>
document.getElementById("someID").innerHTML = "Hello World";
</script>
```
  - ▶ Inline inside an HTML event handler attribute (e.g. `onclick` etc).

```
<button type="button" onclick="myFunction()">Click me</button>
<p id="someID"></p>
<script>
function myFunction() {
 document.getElementById("someID").innerHTML = "Hello World";
}
</script>
```
- Since JavaScript is the default scripting language in HTML, the `type` attribute is actually not required.

```
<script type="text/javascript">
...javascript code here...
</script>
```

## The window object

- JavaScript was designed to turn static HTML document into interactive web applications.
- When running inside the browser, the JavaScript global object is a window object.
- What can we do with that?
- E.g. Changing the currently displayed document.

## Document Object Model 1/3

- The window object has a document property that refers to a Document object.
- Through window.document we can access the Document Object Model (DOM) which is the in-memory browser representation of a web page.
- The elements of an HTML document are represented in the DOM as a tree of objects.

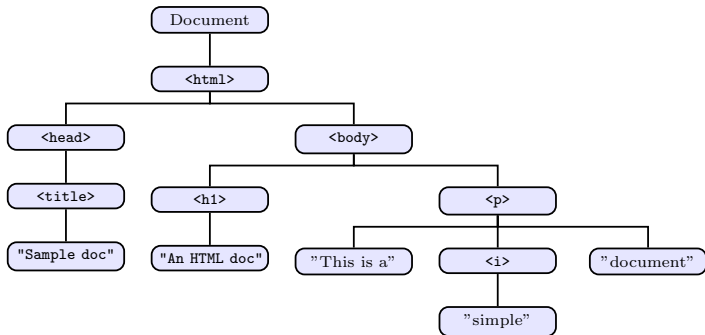
## Document Object Model 2/3

- The DOM represents **each node** as a JavaScript **object**.
- Every object consist of a set of **properties and methods**.
- HTML DOM defines:
  - ▶ The HTML elements as objects
  - ▶ The properties of all HTML elements
  - ▶ The methods to access all HTML elements
  - ▶ The events for all HTML elements

# Document Object Model 3/3

An example for HTML code and relating tree.

```
<html>
<head>
 <title>Sample doc</title>
</head>
<body>
 <h1>An HTML doc</h1>
 <p>This is a <i>simple</i> document.</p>
</body>
</html>
```



# Objects in the DOM

There are 4 important kinds of objects in the DOM:

- **document**: A document object represents the entire document. This can be accessed within the browser using `window.document`.
- **element**: used to represent the elements (tags) in the document.
- **attr**: represents attributes. Each element can have zero to many attr objects as its children.
- **text**: represents the text contained in an element. In some cases an element can have multiple text nodes.

For instance:

```
<p>Left side inner right side</p>
```

The `<p>` tag has two text nodes, one for the text on either side of the `<em>` tag, while the `<em>` tag also has a text node.

# DOM API

- The functions used for interacting with, and manipulating all these types of nodes are called the DOM Application Programming Interface (API).
- The purpose is mainly to allow the DOM to be manipulated **after** the web page has loaded.
- Why after? We will see soon.
- In other words, the objective is to **implement dynamic functionality** within web pages **without loading a new page**.



# DOM API

- The DOM API allows to:
  - ▶ **Select** nodes from the DOM
  - ▶ **Traverse** from a node to another set of nodes
  - ▶ **Manipulate** the nodes (add, replace or delete nodes).
  - ▶ **Respond** to events generated by the nodes
- Quite complex to use directly. Frameworks are often used instead (e.g. JQuery).

# Why after?

```
<!DOCTYPE html>
<html>
<head>
 <meta charset="utf-8" >
 <script>
 document.getElementById("myp").innerHTML = "Hello world";
 </script>
</head>
<body>
 <p id="myp"></p>
</body>
</html>
```

How would you expect the above to work?

# Possible Solution

```
<!DOCTYPE html>
<html>
<head>
 <meta charset="utf-8" />
</head>
<body>
 <p id="myp"></p>
 <script>
 document.getElementById("myp").innerHTML = "Hello world";
 </script>
</body>
</html>
```



Move after tag creation.

## More Elegant Solution

```
<!DOCTYPE html>
<html>
<head>
 <meta charset="utf-8" />
 <script>
 window.onload = function () {
 document.getElementById("myp").innerHTML = "Hello world";
 }
 </script>
</head>
<body>
 <p id="myp"></p>
</body>
</html>
```



Event driven approach

When the whole page is loaded, run the function.

# Elements of window object

- **Timers:** `setTimeout()` and `setInterval()` allow one to register a function to be invoked once or repeatedly after a specified amount of time has elapsed.
- **Dialog Boxes:**
  - ▶ `alert()` displays a message to the user and waits for the user to dismiss the dialog  
`alert("Hello, " + name); // Display a plain message`
  - ▶ `confirm()` displays a message, waits for the user to click an OK or Cancel button and returns a boolean value.  
`var choice = confirm("Are you sure?"); // Get a boolean`
  - ▶ `prompt()` displays a message, waits for the user to enter a string, and returns that string.  
`var name = prompt("What is your name?"); // Get a string`

# Finding, changing and adding/deleting HTML Elements

## Finding HTML Elements

- `document.getElementById("myId1")`
- `document.getElementsByTagName("p")`
- `document.getElementsByClassName("myClass")`
- `document.querySelectorAll("p.myClass")`

## Changing HTML Elements

- `element.innerHTML = "Hello World"`
- `element.attribute = "color : red;"`
- `element.style.property.color = "red";`
- `element.setAttribute("hidden", false)`

## Adding/deleting HTML Elements (examples)

- `document.createElement("p")`
- `document.removeChild(document.getElementById("myId"))`
- `document.appendChild(document.createTextNode("Hello"))`
- `document.replaceChild(newNode, element.childNodes[0])`

# HTML Forms

- Forms are special HTML elements used to **collect information** from people visiting a page.
- Such information can be sent to a server for processing or can be accessed by scripts running inside the page.
- Each form is defined by a form element that may contain **special controls like buttons, text fields, check boxes, etc.**

# Input and label

- The two most used elements inside a form are label and input:
  - ▶ `<label>` is used to enter some text to describe a form field
  - ▶ `<input>` specifies how to create the data entry field (what type of form control to use)
- Here is a simple example (to be explained in the next slide).

```
<form>
 <label for="petname">Enter your pet's name:</label>

 <input type="text" id="petname" name="petname" value="Daisy" required>
 <input type="submit" value="Submit">
</form>
```



## Simple example explained

```
<form>
```

```
 <label for="petname">Enter your pet's name:</label>

```

```
 <input type="text" id="petname" name="petname" value="Daisy" required>
```

```
 <input type="submit" value="Submit">
```

```
</form>
```

- The `for` attribute of the `<label>` tag should be equal to the `id` attribute of the `<input>` element to bind them together.
- `label for="petname"`: associates the label with the input field that has the matching `id` attribute. In this case, it links to the input field where the user will enter their pet's name.
- `id="petname"`: IDs must be unique within a page, and this allows JavaScript or CSS to reference this specific input field directly.
- `name="petname"`: The `name` attribute is used to specify the name of the data that will be sent to the server when the form is submitted (to be sent in a key-value pair where the key is `petname`).
- `value="Daisy"`: This sets a default value for the input field. When the form is loaded, the input field for `petname` will initially show "Daisy".

# Input field types

- Some other common input field types are:
  - ▶ **password:** similar to text field except that the data is masked so that characters typed are not visible on screen
  - ▶ **email:** allows the insertion of email addresses. The browser will automatically validate the content and (on mobile) the keyboard will change to fill the email address
  - ▶ **datetime:** used to collect a time and date (showing a calendar)
  - ▶ **range:** displays a horizontal slider to enter a numerical value in a range
  - ▶ **button:** used to display a button to execute a certain script function when clicked

# HTML Forms

- Action specifies the URL to which the form is submitted. If omitted, the form is submitted to the current URL.

- In the general form:

```
<form action="URL" method="get or post">
... form content ...
</form>
```

- A simple example for Google search

Search your pets name in Google:

```
<form action="http://www.google.com/search" method="get">
 <input type="text" name="q" value="Daisy"/>

 <input type="submit" value="Search" />
</form>
```

- Method indicates how the form data should be packaged in the request that is sent to the server.
- We will study HTTP methods (e.g. get) in the following lessons.

## HTML event handler examples

- JavaScript code can register an event handler by assigning JavaScript statements to a property of an HTML element:
- `<input type="button" value="test" onclick="alert('button clicked');">`
- Event handler attributes defined in HTML may include any number of JavaScript statements, separated from each other by semicolons

## HTML event handlers

HTML element	Type property	Event handler	Description and events
<code>&lt;input type="button"&gt;</code> or <code>&lt;button type="button"&gt;</code>	"button"	onclick	A push button
<code>&lt;input type="checkbox"&gt;</code>	"checkbox"	onchange	A toggle button without radio button behavior
<code>&lt;input type="file"&gt;</code>	"file"	onchange	An input field for entering the name of a file to upload to the web server; value property is read-only

## HTML event handlers

HTML element	Type property	Event handler	Description and events
<code>&lt;input type="password"&gt;"password"</code>		onchange	An input field for password entry—typed characters are not visible
<code>&lt;input type="radio"&gt;</code>	"radio"	onchange	A toggle button with radio button behavior—only one selected at a time
<code>&lt;input type="reset"&gt;</code> or <code>&lt;button type="reset"&gt;</code>	"reset"	onclick	A push button that resets a form

## HTML event handlers

HTML element	Type property	Event handler	Description and events
<code>&lt;select&gt;</code>	"select-one"	onchange	A list or drop-down menu from which one item may be selected
<code>&lt;select multiple&gt;</code>	"select-multiple"	onchange	A list from which multiple items may be selected
<code>&lt;input type="submit"&gt;</code> or <code>&lt;button type="submit"&gt;</code>	"submit"	onclick	A push button that submits a form

## HTML event handlers

HTML element	Type property	Event handler	Description and events
<code>&lt;input type="text"&gt;</code>	"text"	onchange	A single-line text entry field
<code>&lt;textarea&gt;</code>	"textarea"	onchange	A multiline text entry field
<code>&lt;input type="hidden"&gt;</code>	"hidden"	none	Data submitted with the form but not visible to the user



# Lab of WEB Technologies

## Lecture # 10 JavaScript assignment

Zeynep Yücel

Ca' Foscari University of Venice  
zeynep.yucel@unive.it  
yucelzeynep.github.io

## Assignment description

We will build a page to play the "Dots-and-Boxes" game.

- You will need HTML to hold the header, a message division (to show the current player and the current score), a game container with a 3x3 table and a button (to reset the game).
- You will need CSS to style table cells (size, alignment, border, background), text (color, font etc.) and button (size, color, border, font etc.).
- You will need JavaScript to control the game, e.g. starting the game, switching turns (current player and their color), handling clicks on the lines, and determining win or draw.
- The professor will make a demonstration of the correct solution to clarify details.

# Lab of WEB Technologies

## Lecture # 11

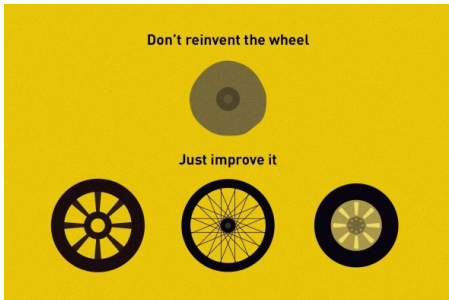
### jQuery, Bootstrap, AJAX

Zeynep Yücel

Ca' Foscari University of Venice  
zeynep.yucel@unive.it  
yucelzeynep.github.io

## Third Party Libraries 1/2

- There are libraries that allow one to simplify and improve the development of web applications.
- They are crucial for modern web development, offering **pre-built** functions and components that streamline the development process, enhance functionality, and improve efficiency.



## Third Party Libraries 2/2

- Front-End Libraries (jQuery, React)
- CSS Frameworks (Bootstrap)
- Back-End Libraries (Django, Flask)
- Database Libraries (Sequelize)
- Testing Libraries (Jest)
- Template engines (Jinja2)
- ...

Popular libraries are often maintained and widely used by various communities.

# jQuery

# jQuery

- jQuery is an **open source library** written entirely in JavaScript.
- It is used to **simplify the interaction with the DOM API**.
- It can ease many other useful things (like AJAX).
- It provides an **intuitive and easy to learn API** for performing the most common tasks in website development.
- It provides cross browser support.
  - ▶ Consistent user experience regardless of the browser being used (e.g., Google Chrome, Mozilla Firefox, Safari, Microsoft Edge, etc.).
- It has become the **de facto standard** for web applications development.

# Adding jQuery to Your Web Pages 1/3

- Download the jQuery library and place it in the same directory as the pages where you wish to use it.

`<head>`

`<script src="jquery-3.7.1.min.js"></script>`

`</head>`

- Or pick a different version from [jQuery API Documentation](#)



## Adding jQuery to Your Web Pages 2/3

- You can also include it from a CDN (Content Delivery Network).

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"
 integrity="sha256-/JqT3SQfawRcv/BIHPThkBs00EvtFFmqPF/lYI/Cxo ="
 crossorigin="anonymous">
</script>
```

- ▶ integrity ensures the script has not been tampered with.
  - ▶ It contains a cryptographic hash (e.g., SHA-256) of the file.
- ▶ crossorigin specifies the mode of the cross-origin request, enabling proper integrity validation.
  - ▶ anonymous: Sends a cross-origin request without credentials.
  - ▶ use-credentials: Sends credentials with the request.

## Adding jQuery to Your Web Pages 3/3

- Google is another jQuery host.

```
<head>
```

```
 <script src="https://ajax.googleapis.com/ajax/
 libs/jquery/3.7.1/jquery.min.js"></script>
```

```
</head>
```

- Since many users already have downloaded jQuery from Google when visiting another site, it will be loaded from cache when they visit your site, which leads to faster loading time.

# jQuery Syntax

- jQuery exports one single function named jQuery that is aliased with a single dollar sign \$.
- The jQuery (or \$) function is used to:
  - ▶ Select elements from the DOM
  - ▶ Invoke some "global" functions provided by jQuery like (\$.ajax())
- The jQuery syntax is **taylor-made for selecting** HTML elements and **performing some action** on the elements.
- Basic syntax is: **\$ (selector).action()**
  - ▶ A \$ sign to define/access jQuery
  - ▶ A (selector) to "query (or find)" HTML elements
  - ▶ A jQuery action() to be performed on the element(s)

## Selecting elements

- Element selection in jQuery works almost the same way as CSS selectors.
- For example, we can do the following:
  - ▶ `$("td");` // all td tags (table data cells)
  - ▶ `$("#contacts");` // element with id contacts
  - ▶ `$(".controls");` // elements with class controls
  - ▶ `$("body h1");` // all h1 tags in body
- Under the hood, jQuery wraps the matching document elements and provides additional functions to them.

## Performing actions on elements

- Here are some examples for actions:

- ▶ Hiding all <p> elements.

```
$("#p").hide()
```

- ▶ Hiding all elements with class="test".

```
$(".test").hide()
```

- ▶ Hiding the element with id="test".

```
$("#test").hide()
```

- ▶ Returning the text of an element

```
$("#h1").text()
```

- ▶ If there are multiple h1 elements in the document, e.g.

```
<h1>Heading 1</h1>
```

```
<h1>Heading 2</h1>
```

it returns "Heading 1Heading 2".

- ▶ Setting the text of an element

```
$("#h1").text("hello");
```

- It is more readable and concise than HTML

```
// Using jQuery
```

```
$("#myElement").hide();
```

```
// Using getElementById in HTML
```

```
document.getElementById("myElement").style.display = "none";
```

## Performing actions on elements

- The keyword (`this`) is another way to select elements.
- In jQuery (and JavaScript in general), the keyword `this` refers to the current context or the current object that is being referenced.
- Its meaning can change depending on how it is used (we will explain more later).

# Manipulating the DOM 1/2

## Adding new nodes

- We can simply add new DOM nodes with the functions (example):
  - ▶ `$("...").before(...html string...)`
  - ▶ `$("...").after(...html string...)`
  - ▶ `$("...").append(...html string...)` (as last child)
  - ▶ `$("...").prepend(...html string...)` (as first child)
    - ▶ before and after methods insert content adjacent to the selected element, before or after it in the DOM, but do not alter the element's existing children.
    - ▶ append and prepend methods add content as children to the selected element.

# Manipulating the DOM 2/2

## Changing the text of an element

- We can change the text of an element or the child subtree with:

- ▶ `$("...").text(...string...)`
- ▶ `$("...").html(...html string...)`

- For example:

```
// with jQuery
$("#myid").html("Content Updated!");
```

```
// with HTML
document.getElementById("#myid").innerHTML = "Content Updated!";
```



# Manipulating the style

- We can add and remove classes to any DOM node with:

- ▶ `$("...").addClass("classname")`
- ▶ `$("...").removeClass("classname")`

- You can also change any CSS property using jQuery:

- ▶ 

```
// using CSS
p {
 background-color: red;
}
```

```
// using jQuery
$("p").css("background-color", "yellow");
```

- ▶ If you have both of the above in your project, what will be the background color of `<p>` elements?
  - ▶ In CSS, when the same property is applied multiple times, the one that is applied last takes precedence, if they have the same specificity.
  - ▶ If the jQuery command runs after the CSS is applied, it effectively overrides the red background color, resulting in the final background color being yellow.

## Adding event callbacks

- Some common functions are
  - ▶ `$("...").click( function(evt) {} );`
  - ▶ `$("...").mousemove( function(evt) {} );`
  - ▶ `$("...").change( function(evt) {} );`
  - ▶ `$("...").hover( function(evt) {} );`
  - ▶ `$(document).ready( function() { } );`
- Some other examples involving the event object (evt):  
`$(".button-class").click( function(evt) {  
 console.log("Clicked element:", evt.target);  
});`

## this and event target

- The keyword `this` can be used inside the function handler to access the element generating the event:

```
$("#button.changeButton").click(function() {
 $(this).text("Button Clicked!");
});
```

- `this` may work in an unexpected way in arrow functions `() => {}`
  - ▶ Arrow functions do not have their own `this`. Instead, they inherit `this` from the surrounding lexical context at the time they are defined.
  - ▶ This means that if you use `this` inside an arrow function, it will refer to the `this` value of the enclosing scope, not the context where the arrow function is called.
  - ▶ This is often a point of confusion for developers.
  - ▶ Instead of relying on `this`, we access the target of the event using `event.target` (example).

```
$("#myButton").on("click", (event) => {
 console.log("hello");
 console.log(event.target); // Logs the clicked element
});
```

# Animations

- With jQuery, you can hide and show HTML elements with the `hide()` and `show()` methods:
  - ▶ `$("...").show()`
  - ▶ `$("...").hide()`
- You can also toggle between hiding and showing an element with the `toggle()` method.
  - ▶ `$("...").toggle()`
- With jQuery you can fade an element in and out of visibility.
  - ▶ `$("...").fadeIn`
  - ▶ `$("...").fadeOut`
- With jQuery you can create a sliding effect on elements.
  - ▶ `$("...").slideUp`
  - ▶ `$("...").slideDown`
- For finding out more, refer to
  - ▶ [jQuery documentation](#)
  - ▶ [w3schools jQuery Tutorial](#)

# Bootstrap

# Bootstrap

- Bootstrap is a free and open source CSS framework directed at responsive, mobile first, front-end web development.
- It contains HTML, CSS and (optionally) JavaScript based design templates
  - ▶ Typography, forms, buttons, navigation, ...
- Bootstrap 5 released in 2021 is the newest version (with last stable release on 25 Aug 2025).
- The main differences between Bootstrap 5 and Bootstrap 3 and 4, is that Bootstrap 5 removed dependence on jQuery.
  - ▶ To align it with contemporary web development practices, for improving performance, reducing dependencies, and encouraging the use of modern JavaScript features.

# Responsive web design

- Responsive web design (RWD) is an approach to web design that makes web pages **render well on a variety** of devices and window or screen sizes.
- A site designed with RWD adapts the layout to the viewing environment by using **fluid, proportion based** grids, flexible images, and CSS media queries.

# RWD principles

- **Fluid grid concept**: page elements should be sized using relative units (percentages).
- **Images** should also be sized in **relative units**.
- **Media query** should be used to apply different CSS based on characteristics of the device.
  - ▶ **Media queries** are a CSS technique that lets you apply different styles **based on specific conditions** (e.g. the width, height, orientation, resolution, etc. of the device's viewport).
  - ▶ By using media queries, you can set **breakpoints** at which your CSS will change to accommodate different screen sizes and resolutions.
  - ▶ When a browser renders a web page, it **evaluates the defined media queries** against the characteristics of the user's device.
  - ▶ If a specified condition in a media query is met, the CSS rules contained within that query will be applied.
- Responsive layouts automatically adjust and adapt to any device screen size.



# Adaptive vs Responsive

- Adaptive generates templates **optimized for every device class**:
  - ▶ **Multiple Layouts**: Specific layouts are designed for different screen sizes or devices (e.g., mobile, tablet, desktop).
  - ▶ **Device Detection**: The website uses a server-side technology to detect the device and serve the appropriate layout.
  - ▶ **More Control Over Design**: Designers can create tailored experiences for specific devices.
- Responsive is a **universal design reflowing accross displays**.
  - ▶ **Fluid Grids**: Layouts are based on a percentage of the screen size rather than fixed pixels, allowing the content to resize fluidly.
  - ▶ **Media Queries**: CSS allows you to apply styles based on the characteristics of the device.
  - ▶ **Single Codebase**: Just one version of the site works across all devices, which simplifies development and maintenance.



# Where to get Bootstrap?

- You can include Bootstrap from a CDN (Content Delivery Network).

```
<head>
```

```
 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/
 bootstrap.min.js">
```

```
 </script>
```

```
</head>
```

- For instance, jsDelivr provides CDN support for Bootstrap.
- Many users already have downloaded Bootstrap 5 from jsDelivr when visiting another site. As a result, it will be **loaded from cache** when they visit your site, which leads to faster loading time.

# How to use a CDN

- Just add these tags in the <head> section:

```
<head>
```

```
 <meta charset="utf-8">
```

```
 <meta name="viewport" content="width=device-width,
 initial-scale=1">
```

```
 <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/
 bootstrap/3.4.1/css/bootstrap.min.css">
```

```
 <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/
 jquery.min.js"></script>
```

```
 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/
 js/bootstrap.min.js"></script>
```

```
</head>
```

- The <meta> tag is used in mobile devices to allow a correct handling of the pinch-to-zoom feature.

# Bootstrap grid system

- The basis of the Bootstrap framework relies on a **grid system**.
- It is based on **containers, rows, and columns** to layout and align content.
- Containers are the most basic layout element in Bootstrap. All the page content needs to be placed inside a container.
- Two available classes:
  - ▶ **.container** is fixed width with a certain default size for each display class.
  - ▶ **.container-fluid** is a full width container, spanning the entire width of the viewport.

## Responsive breakpoints

- Bootstrap provides sensible breakpoints based on minimum viewport widths and allows the grid system to scale up elements as the viewport changes.
  - ▶ Extra small: width < 576px
  - ▶ Small (landscape phones): 576px and up
  - ▶ Medium (tablets): 768px and up
  - ▶ Large (desktops): 992px and up
  - ▶ Extra large (large desktops): 1200px and up
- The container width (max width) will change according to the different screen sizes:

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra Large ≥1200px	XXL ≥1400px
max width	100%	540px	720px	960px	1140px	1320px

# How the grid system works

- Bootstrap's grid system allows **up to 12 columns** across the page.
- If you do not want to use all 12 columns individually, **you can group the columns together** to create wider columns.
- Containers provide a means to center and horizontally pad the contents.
- Rows are wrappers for columns. Each column has a horizontal padding for controlling the space between them.
- Content **must be placed within columns** and only columns may be immediate children of rows.
- Column classes indicate the number of columns you would like to use out of the possible 12 per row.
- This can be used to specify the width of each column.

span1	span1	span1	span1	span1	span1	span1	span1	span1	span1	span1	span1
span4				span4				span4			
span4				span8							
span6						span6					
span12											

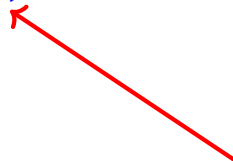
## How the grid system works

- The grid system is responsive, and the columns will **rearrange automatically** based on the screen size.
- To make the grid responsive, there are five grid breakpoints: (extra small), small, medium, large, and extra large.
- Grid breakpoints are based on **minimum width media queries**, meaning they **apply to that one breakpoint and all those above it**.

## Specifying columns

- To identify a column, we must follow the template of `.col-*-*`.
  - ▶ The first asterisk means the viewport for the column.
  - ▶ The second asterisk means the size of the column. It is a number between 1 to 12 (i.e. container width).
- Suppose that we want to create a header section in our page with a single column filling the entire container width. We will write:

```
<header class="container">
 <div class="row">
 <div class="col-md-12">
 <h1>HEADER</h1>
 </div>
 </div>
</header>
```



For medium viewport (and up) we want a 12-column width column (full size). For smaller viewports the default is applied



## Specifying columns

- Here is three equal width columns starting at small viewports and scaling to extra large desktops (col-sm).

```
<div class="row">
 <div class="col-sm-4">col 1</div>
 <div class="col-sm-4">col 2</div>
 <div class="col-sm-4">col 3</div>
</div>
```

- On mobile phones or screens that are less than 576px wide, the columns will automatically stack on top of each other.

## col-\*-\* options

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra large ≥1200px
Max container width	None (auto)	540px	720px	960px	1140px
Class prefix	.col-*	.col-sm-*	.col-md-*	.col-lg-*	.col-xl-*



## Some references and Bootstrap components

- Bootstrap provides a plethora of pre-defined components and themes to help you creating professional web pages.
- Examples include: Badges, Buttons, Cards, Carousel, Navs, Scrollspy, etc.
- Refer to the below for more information
  - ▶ [w3cschools Tutorial](#)
  - ▶ [Bootstrap documentation](#)
- You may also refer to the below
  - ▶ [Examples](#)
  - ▶ [components](#)
  - ▶ [themes](#)

# Ajax

# Why Ajax?

## Evolution of the web

- The trend in web design is towards moving the application code **forward in the stack**.
- AJAX allows to push the application logic **from the server to the client** (web browser).
- With Ajax, you can
  - ▶ Update a web page without reloading the page
  - ▶ Request data from a server - after the page has loaded
  - ▶ Send data to a server - in the background
- Thus, you can **reduce the server load** and hence the costs.

## Server or client side?

Server side  Client side

Server manages every aspect of a web application:

- Application logic
- User input validation
- Webpage structure generation (HTML, etc.)

Client side (Single page app) manages both the presentation and application logic. Interactions with the server are limited to a pure application data exchange

# Single Page Application (SPA)

- SPA is a web application or website that interacts with the user by dynamically rewriting the current page rather than loading entire new pages from the server.
- Key characteristics of SPAs
  - ▶ **Dynamic content loading:** SPAs load content dynamically without requiring a full page reload.
  - ▶ **Single HTML file:** Typically, an SPA consists of a single HTML file that serves as the shell of the application.
  - ▶ **Improved user experience** Since SPAs load data asynchronously and update only parts of the page, users experience faster interactions.
  - ▶ **Heavy Use of JavaScript:** SPAs rely on JavaScript for rendering views, handling user input, communicating with server.
- Gmail is an example of SPA (also Google maps, Twitter, Facebook..).



# Web Flow

- In the classical web, there was no method to either send or download new data (i.e. make an HTTP request) **without loading a new page**.
- This was a serious limitation for any nontrivial web application.  
Example: how would we implement a chat?

# Asynchronous programming

- Asynchronous programming is a programming paradigm that allows a program to **perform tasks concurrently** with other tasks, **without blocking the execution flow**.
  - ▶ **Concurrency** is the ability of a program to manage multiple tasks at the same time. In asynchronous programming, tasks can start, run, and complete in overlapping time periods.
  - ▶ **Non-Blocking** means that the execution of the program continues even when an operation is still in progress. This is crucial for maintaining responsiveness in applications, especially in UI development.
- This is particularly useful for operations that can take a long time to complete, such as network requests, file I/O, and database queries.
- Important benefits of asynchronous programming are improved performance and responsiveness.

# AJAX

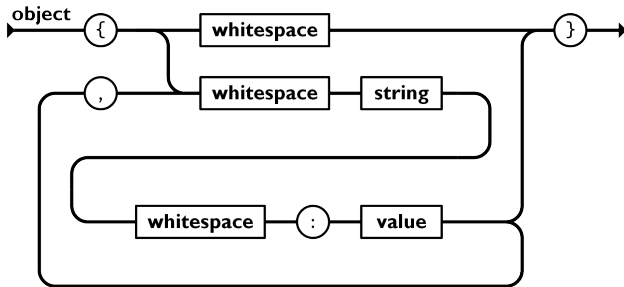
- AJAX stands for **A**synchronous **J**avaScript **A**nd **X**ML.
- It identifies a set of techniques (in the browser) to create asynchronous web applications.
- With AJAX, an application **can retrieve or send data** to a web server asynchronously (in background) without interfering with the display and behavior of the existing page (i.e. **without loading a new page**).
- In the past, this technique was primarily used to send and retrieve data in XML format (hence the X in the name).
- Nowadays it is common to use the JSON format for its simplicity and because a JSON string can be easily converted to a JavaScript object.

# JSON

- JavaScript Object Notation (JSON) is a lightweight data exchange format based on conventions commonly used in languages like C++, JavaScript, Java, etc.
- It is very popular, since it is **easy to read** for a human and **to parse** by the JavaScript interpreter.
- It is based on JavaScript, but it allows us to represent data structures **common to the vast majority of high level programming languages**.
- What can be represented?:
  - ▶ Dictionaries (i.e. Lists of properties composed by a name and a value)
  - ▶ Ordered lists of elements (i.e. Arrays, vectors, etc)

## JSON: Object/Dictionary

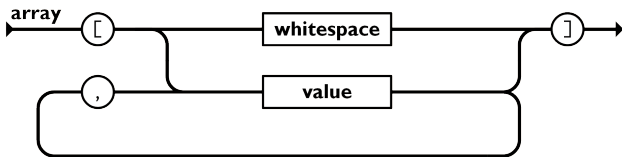
- An object is an unordered set of name/value pairs.
- An object begins with left brace and ends with right brace.
- Each name is followed by colon (:) and the name/value pairs are separated by comma (,).



<https://www.json.org/json-en.html>

## JSON: Array

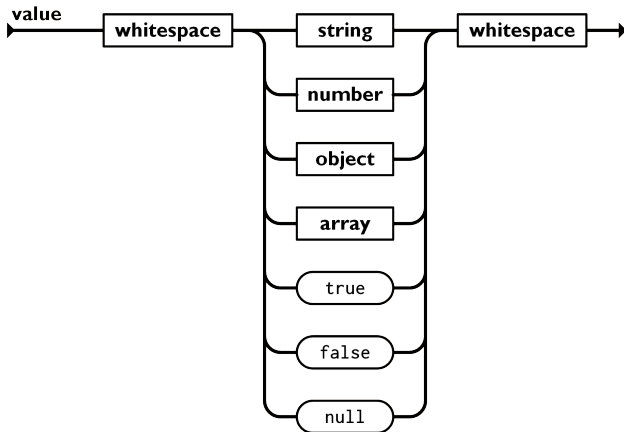
- An array is an ordered collection of values.
- An array begins with left bracket ([) and ends with right bracket (]).
- Values are separated by comma (,).



<https://www.json.org/json-en.html>

# JSON: Value

- A value can be a string in double quotes, or a number, or true or false or null, or an object or an array.
- These structures can be nested.



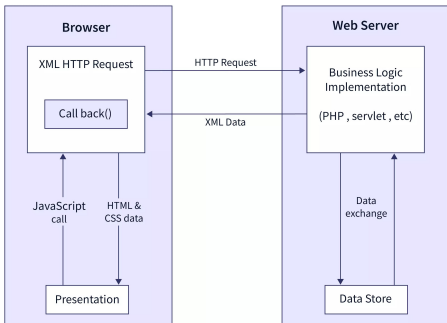
<https://www.json.org/json-en.html>

# JSON: Object/Dictionary example

```
{
 "books": [
 {
 "title": "To Kill a Mockingbird",
 "author": "Harper Lee",
 "year_published": 1960,
 "genre": "Fiction"
 },
 {
 "title": "1984",
 "author": "George Orwell",
 "year_published": 1949,
 "genre": "Dystopian"
 },
 {
 "title": "The Great Gatsby",
 "author": "F. Scott Fitzgerald",
 "year_published": 1925,
 "genre": "Classic"
 }
]
}
```



# Going back to Ajax



- An event occurs in a web page (e.g. the page is loaded, a button is clicked).
- An XMLHttpRequest object is created by JavaScript.
- The XMLHttpRequest object sends a request to the server.
- Server processes the request.
- Server sends a response back to the web page.
- The response is read by JavaScript.
- Proper action (e.g. page update) is performed by JavaScript.

# AJAX and jQuery

- Implementing AJAX without external libraries can be difficult for several reasons:
  - ▶ Cross browser compatibility not trivial
  - ▶ JSON serialization/deserialization
  - ▶ Error codes
- jQuery offers a few functions to send and receive data asynchronously to a remote web server.

# Templating languages

- A general purpose templating language is a type of programming or markup language designed to generate dynamic content by combining static templates with variable data.
  - ▶ It provides a way to define a structured template (containing placeholders or markup) where specific data can be inserted to produce customized output.
  - ▶ Templating languages are commonly used in web development to generate HTML pages.

# Web template engines

- In web publishing, [web template systems](#) are commonly used.
- These allow web designers and developers to work with web templates to automatically generate custom web pages, such as the results from a search.
  - ▶ Web templates support static content, providing basic structure and appearance.
  - ▶ They reuse static web page elements and define the dynamic elements based on web request parameters.
  - ▶ [Jinja2](#) is a library for Python that is designed to be flexible, fast and secure.
- We will get back to Jinja2, after we will have studied web servers.

# Lab of WEB Technologies

## Lecture # 12

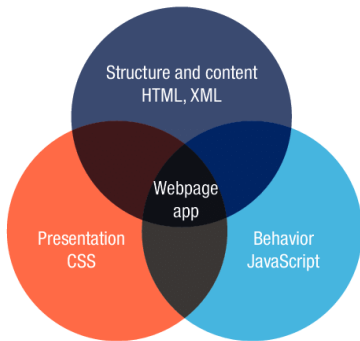
### HTTP Methods and Web Server

Zeynep Yücel

Ca' Foscari University of Venice  
zeynep.yucel@unive.it  
yucelzeynep.github.io

# Anatomy of a Web Page

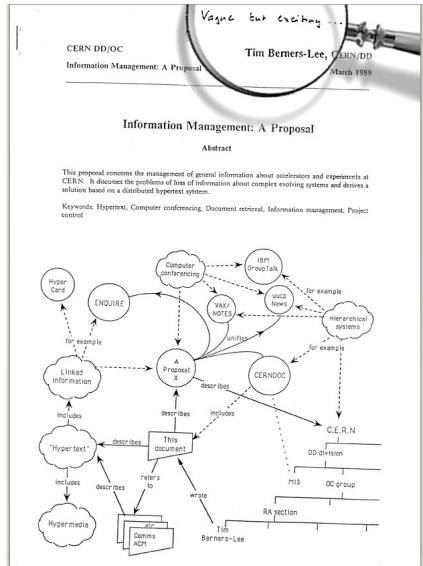
- Triad of Web technologies
  - ▶ HTML establishes the content and its structure
  - ▶ Cascading Style Sheets (CSS) gives styling attributes for presentation
  - ▶ JavaScript adds functional behavior to these Web elements.



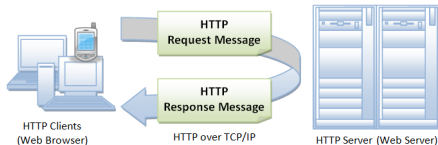
How can I access it in the World Wide Web?

# Hyper-text transfer protocol (HTTP) 1/2

- In March 1989, Tim Berners-Lee submitted a proposal for an information management system to his boss, Mike Sendall, on which he wrote 'Vague, but exciting'.
- Protocol **request-response** originally designed to exchange hypertext (now used to exchange any kind of resource)



## Hyper-text transfer protocol (HTTP) 2/2

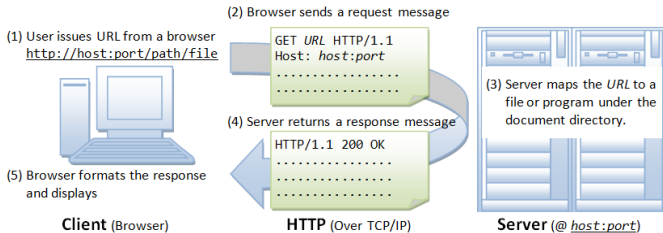


- A [Request for Comments \(RFC\)](#) 2616: "The Hypertext Transfer Protocol (HTTP) is an application-level protocol for distributed, collaborative, hypermedia information systems. It is a generic, stateless protocol..."
- HTTP is an [asymmetric](#) request-response client-server protocol. A client sends a request message to a server, which in turn returns a response message.
- In other words, HTTP is a [pull protocol](#), where the client pulls information from the server (instead of the server pushing information to the client).
- HTTP is [stateless](#), since the current request does not know what has been done in the previous requests.



# The communication between client and server

- Suppose that you **issue a URL** from your browser to get a web resource using HTTP, e.g. `http://www.example.com/index.html`,
- The browser **turns the URL into a request** message and sends it to the HTTP server.
- The HTTP **server interprets the request** message.
- It returns an appropriate response message, which is either the resource you requested or an error message.



# Uniform Resource Locator (URL) 1/2

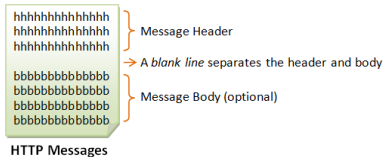
- A URL is used to **uniquely identify a resource** over the web.
- The URL **syntax** is **protocol:// hostname: port/ path-and-file-name**
- There are 4 parts in this URL:
  - ▶ **Protocol**: The application-level protocol used by the client and server, e.g., HTTP, FTP, and telnet.
  - ▶ **Hostname**: The DNS domain name (e.g. www.example.com) or IP address of the server (e.g. 192.128.1.2).
  - ▶ **Port**: The TCP port number that the server is listening for incoming requests from the clients.
  - ▶ **Path-and-file-name**: The name and location of the requested resource, under the server document base directory.

## Uniform Resource Locator (URL) 2/2

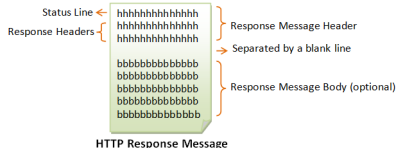
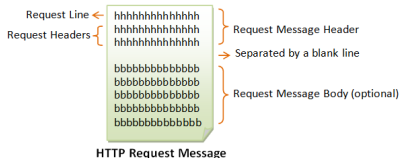
- In the URL `http://www.example.com/docs/index.html`,
  - ▶ The communication protocol is HTTP.
  - ▶ The hostname is `www.example.com`.
  - ▶ The port number was not specified in the URL, and takes on the default number, which is TCP port 80 for HTTP.
  - ▶ The path and file name for the resource to be located is `"/docs/index.html"`.
- Other examples of URL are:
  - ▶ `ftp://www.ftp.org/docs/test.txt`
  - ▶ `mailto:user@test101.com`
  - ▶ `telnet://www.nowhere123.com/`

# HTTP Request and Response Messages

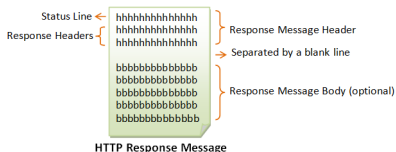
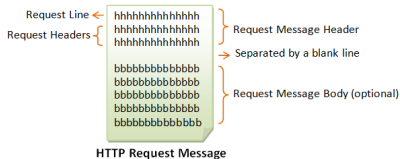
- HTTP client and server communicate by text messages.
- The client sends a request message to the server. The server, in turn, returns a response message.
- An HTTP message consists of a message header and an optional message body, separated by a blank line:



- The format of an HTTP request and response messages is as follows



# HTTP Request Line and Status Line 1/2



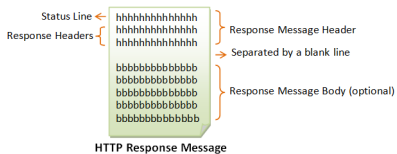
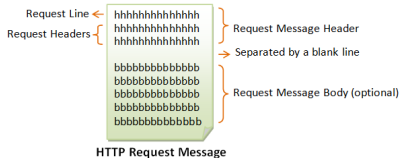
## Syntax of request line

- request-method-name request-URI  
HTTP-version
  - ▶ request-method-name: The client can use GET, POST, HEAD, and OPTIONS etc. methods to send a request to the server.
  - ▶ request-URI: specifies the resource requested.
  - ▶ HTTP-version: HTTP/1.0 or HTTP/1.1

## Syntax of status line

- HTTP-version status-code  
reason-phrase
  - ▶ HTTP-version: HTTP/1.0 or HTTP/1.1
  - ▶ status-code: a 3-digit number generated by the server to reflect the outcome of the request.
  - ▶ reason-phrase: gives a short explanation to the status code.

# HTTP Request Line and Status Line 2/2



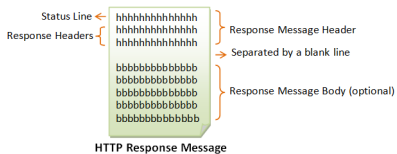
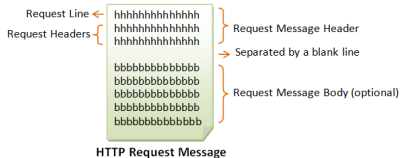
- Examples of request line:

- ▶ GET /test.html HTTP/1.1
- ▶ HEAD /query.html HTTP/1.0
- ▶ POST /index.html HTTP/1.1

- Examples of status line:

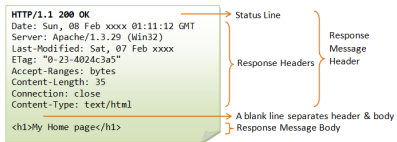
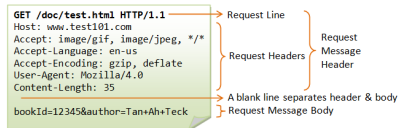
- ▶ HTTP/1.1 200 OK
- ▶ HTTP/1.0 404 Not Found
- ▶ HTTP/1.1 403 Forbidden

# HTTP Request Header and Response Header



- The request headers are in the form of name:value pairs.
- Example of a request header

- The response headers are in the form of name:value pairs:
- Example of a response header



## Common HTTP methods and status codes

- The most commonly-used HTTP methods are POST, GET, PUT, and DELETE.
- If the requests with these methods are a success, they return the data asked by the web client along with the status code 200 (SUCCESS).
- Otherwise, return with the status code 404 (PAGE NOT FOUND) or returns status code 500 (Server error).



## GET Method

- GET is read only, it is used to read the data, retrieve it, and return that to the client.
- It is the simplest among all the requests since it provides the required resource without any modifications.
- For example, it allows one to get a web page.
- Multiple identical GET requests return the same result (i.e. GET is idempotent).

## POST Method

- POST requests are used to create or add a new item to the requested URL.
- For example, creating a new account or posting a new message on blog.
- Once done, it responds with the status code 201 (CREATED), along with the location link of the posted data.
- Making two identical POST requests will duplicate the resource (i.e. POST is not idempotent).

## PUT Method

- PUT requests are used to modify or replace the current data with the requested data.
- Difference between PUT and POST
  - ▶ POST is for creating resources, not idempotent, usually used in HTML forms.
  - ▶ PUT is for updating or creating a specified resource, idempotent, generally used in programmatic APIs rather than in traditional form submissions.
- It can also be used to create a new item like POST, but POST is used for that purpose.
- For example, change the password on a website.
- Multiple identical requests will update the same resource.

## DELETE Method

- DELETE is used to delete all the data from the target location requested by the client.
- It is a risky operation because once deleted, it cannot be retrieved again.
- Multiple identical requests will delete same resource.
- The second time often returns the status code 404 (PAGE NOT FOUND) or status code 500 (Server error).

## Other HTTP methods

- HEAD: requests the headers of a resource without downloading the actual content, allowing you to check metadata like size or modification date.
- CONNECT: establishes a tunnel to a server, often used for SSL (HTTPS) or proxy connections, allowing secure communication.
- OPTIONS: asks the server which HTTP methods are supported for a particular resource, helping clients understand what actions they can perform.
- TRACE: sends back the exact request it received from the client, primarily used for diagnostic purposes to see what changes might occur along the request path.
- PATCH: applies partial modifications to a resource, allowing updates without needing to send the entire content.

# Properties of HTTP Methods

HTTP Method	Request Has Body	Response Has Body	Safe	Idempotent	Cacheable
GET	Optional	Yes	Yes	Yes	Yes
HEAD	No	No	Yes	Yes	Yes
POST	Yes	Yes	No	No	Yes
PUT	Yes	Yes	No	Yes	No
DELETE	No	Yes	No	Yes	No
CONNECT	Yes	Yes	No	No	No
OPTIONS	Optional	Yes	Yes	Yes	No
TRACE	No	Yes	Yes	Yes	No
PATCH	Yes	Yes	No	No	No

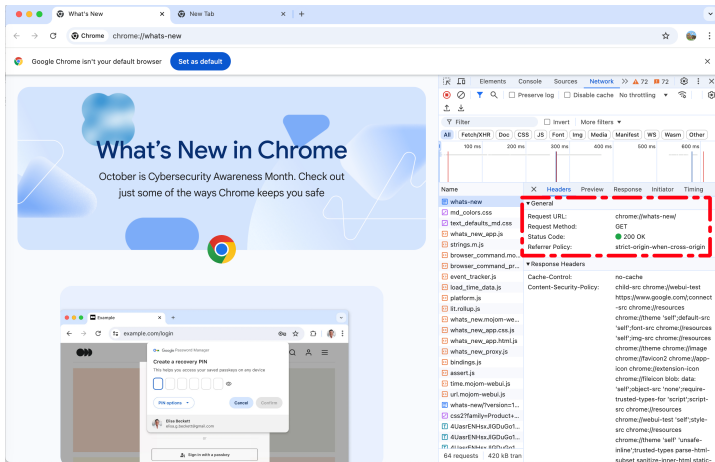
- **Request/Response has body:** Body (message body, or payload body) refers to the data sent between the client and server. For instance, in a response from the server, the body would contain the data that the server is sending back to the client.
- **Safe:** A method is safe if a request with that method has no effect on the server (i.e. it is to be intended as read-only).
- **Idempotent:** A method idempotent if making the same request repeatedly produces the same result.
- **Cacheable:** A method is cacheable if responses to requests with that method may be stored for future reuse.

## How use HTTP Methods

- HTTP methods are declared in HTTP requests
- The browsers can send requests and specify methods in different ways:
  - ▶ Address Bar
  - ▶ `<form>` tags in HTML
  - ▶ jQuery and AJAX functions
  - ▶ ...

# HTTP Request with Address Bar

- When one enters a URL into a web browser's address bar and hits Enter, the browser sends an HTTP GET request to the web server hosting that URL.





# HTTP Request with <form> tags

- `<form action="URL" method="get or post">`  
... form content ...  
`</form>`
- Action specifies the URL to which the form is submitted.
- If omitted, the form is submitted to the current URL.
- Method indicates the HTTP methods to use for the request.
- Example with HTTP request with <form> tags

```
<form action="/register_user" method="POST">
 <label for="fname">First name:</label>
 <input type="text" id="fname" name="fname" required>

 <label for="lname">Last name:</label>
 <input type="text" id="lname" name="lname" required>

 <input type="submit" value="Submit">
</form>
```

First name:

Last name:

# HTTP Request with jQuery and AJAX

- To make an HTTP request with GET method and retrieve data in JSON format, we can use:

```
$.getJSON(url, data, function(d) {
 // callback handler
});
```

- If the request succeeds, d contains the JavaScript object obtained by parsing the received JSON string.
- \$.getJSON() is a shorthand for the more general ajax function:

[jQuery API Documentation](#)

- Example:

```
$.ajax({
 type: "GET", // or "POST" for sending data
 url: url,
 data: null, // data to be sent to the server
 dataType: "json", // expected data type of the response
 success: function(response) {
 // handle successful response
 }
});
```

# Web Server

- A web server is a computer system capable of **delivering web content** (e.g. HTML documents, images, cascaded style sheets, JavaScript files) to end users over the internet via a web browser.
- The primary role of a web server is to **store, process, and deliver** requested information to end users.
- A web **browser initiates** communication by creating a request for a web page or other resource using HTTP.
- The **web server responds** with the content of that resource or an error message.

# How to build a Web Server



- There exist many frameworks and programming languages for server-side programming:

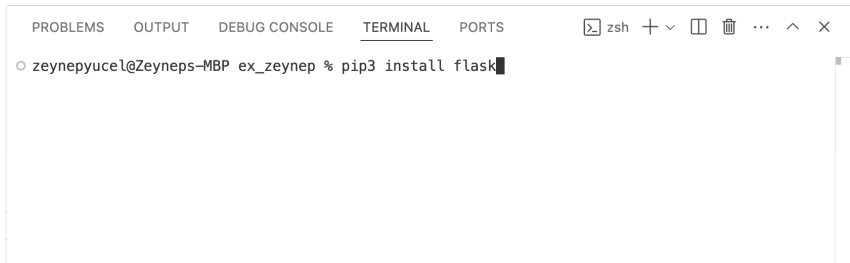
Framework	Programming Language	Famous Use Cases
Django	Python	Instagram, Pinterest, Coursera
ExpressJS	NodeJS	MySpace, GeekList, Storify
Laravel	PHP	Deltanet Travel, Neighborhood Lender
Ruby on Rails	Ruby	ZendDesk, Shopify, GitHub
Flask	Python	Red Hat, Rackspace, Reddit
Asp.NET	C#	Microsoft, Godaddy, Ancestry
Spring Boot	Java	Trivago, Via Varejo, Intuit
Phoenix	Elixir	Financial Times, Fox 10, ABC15

# Flask

- Flask is a microframework for developers, designed to enable them to create and scale web apps quickly and simply.
  - ▶ Pros: Ready to go, lightweight, flexible
  - ▶ Cons: Not many tools, complexity grows with project size
- Refer to [Flask User's Guide](#) for details

# Installation

- Installation Requirements
  - ▶ Flask is a Python framework, so it requires a Python installation and latest versions of Flask work with python  $\geq 3.7.*$ .
  - ▶ [python.org](https://python.org)
- For installing Flask, type "pip install Flask" or "pip3 install Flask" on your terminal (preferably after making a virtual environment).



A screenshot of a terminal window. The window has a title bar with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected), and 'PORTS'. To the right of the tabs are icons for a shell (zsh), window management (plus, minus, square), and other functions (trash, ellipsis, up arrow, close). The terminal content shows a prompt 'zeynepyucel@Zeyneps-MBP ex\_zeynep %' followed by the command 'pip3 install flask' with a cursor at the end.

- Be sure to install version  $\geq 2.2.*$  with python  $\geq 3.7.*$   
`flask --version`

# Quickstart 1/3

- ```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello_world():
    return "<html>
        <head></head>
        <body><p>Hello World!</p></body>
        </html>"
```

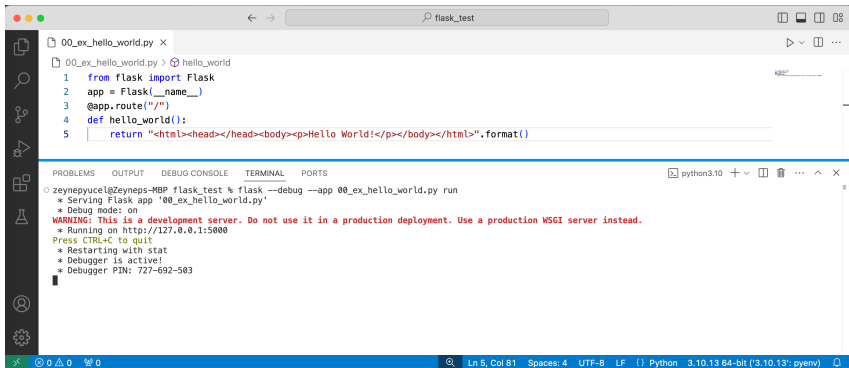
The route() decorator tells Flask what URL should trigger our function

The function returns the HTML code to be rendered by the browser.

- By default, Flask applications listen on the localhost address 127.0.0.1:5000, which means they can only be accessed from the same machine that the application is running on.
- Save this text file in a directory, e.g. with a name "myapp.py".

Quickstart 2/3

- On terminal, run `flask --debug --app myapp.py run`
- Or run `python3 -m flask --debug --app myapp.py run`



The screenshot shows a code editor window titled 'flask_test' with a file named '00_ex_hello_world.py'. The code defines a Flask application with a single route 'hello_world' that returns an HTML response. Below the code editor, the terminal output shows the command being executed and the application running in debug mode on port 5000.

```
00_ex_hello_world.py x
00_ex_hello_world.py > hello_world
1 from flask import Flask
2 app = Flask(__name__)
3 @app.route("/")
4 def hello_world():
5     return "<html><head></head><body><p>Hello World!</p></body></html>".format()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

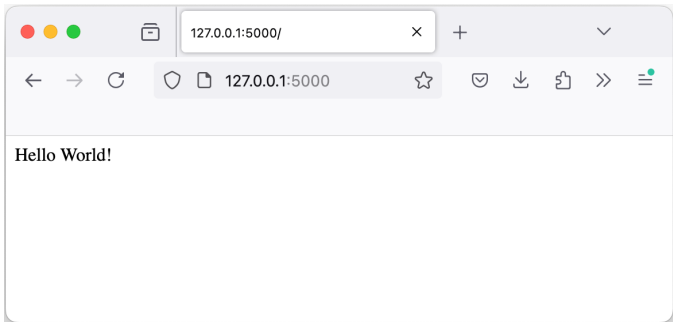
```
zeynepyucel@Zeynep-MBP flask_test % flask --debug --app 00_ex_hello_world.py run
* Serving Flask app '00_ex_hello_world.py'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 727-692-503
```

python3.10 + v ... ^ x

Ln 5, Col 81 Spaces: 4 UTF-8 LF () Python 3.10.13 64-bit ('3.10.13': pyenv)

Quickstart 3/3

- Open the URL `http://127.0.0.1:5000` on your web browser.



- Press `Ctrl+C` on terminal to stop the server.

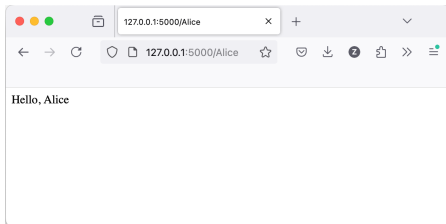
App routing 1/4

Adding arguments

- You can add variable sections to a URL by marking sections with `<variable_name>`.
- Your function then receives the `<variable_name>` as a keyword argument.

```
• from flask import Flask
  app = Flask(__name__)
  @app.route("/<name>")
  def hello_you(name):
      return "<html><head></head><body>
            <p>Hello, {} </p>
            </body></html>".format(name)
```

Whatever you write after the backslash is captured and passed as argument name.



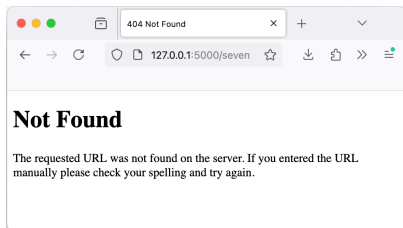
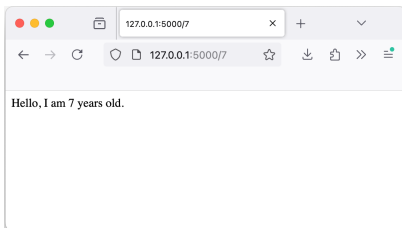
App routing 2/4

Setting the type of an argument

- You can also set the type of an argument specifying the type (string, int, float, path, uuid) with the separator : and after the argument name.

- ```
from flask import Flask
app = Flask(__name__)
@app.route("/<int:age>")
def hello_you(age):
 return "<html><head></head><body>
 <p>Hello, I am {} years old.</p>
 </body>".format(age)
```

The argument age is an integer.



## App routing 3/4

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def index():
 return "<html><head></head><body>This is the homepage</body></html>"

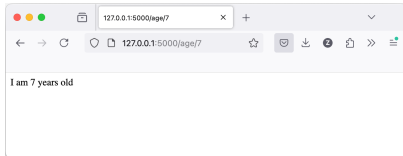
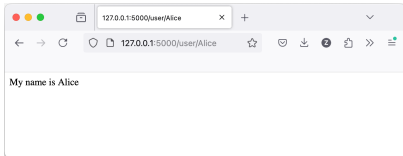
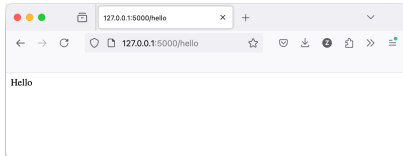
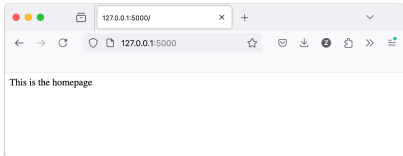
@app.route('/hello')
def hello():
 return "<html><head></head><body>Hello</body></html>"

@app.route('/user/<username>')
def show_user(username):
 return "<html><head></head><body>My name is {}.</body></html>".\
 format(username)

@app.route('/age/<int:age>')
def show_post(age):
 return "<html><head></head><body>I am {} years old.</body></html>".\
 format(age)

if __name__ == "__main__":
 app.run(debug=True)
```

# App routing 4/4



## Static Files

- Dynamic web applications also need static files.
- That is usually where the CSS, JavaScript files and images are coming from.
- Flask will automatically serve as “static” all files located in the “static” subfolder.

## An example for simple static file download server

- You can also define a different subfolder through path variables.
- Consider the following directory structure

```
project-directory
|--- app.py
|--- static
| |--- cat.jpg
```

```
app.py
from flask import Flask, send_from_directory

app = Flask(__name__)

@app.route('/<filename>')
def download_file(filename):
 directory = 'static'
 return send_from_directory(directory, filename, as_attachment=True)
```

- You can download the image by entering in the address line  
`http://127.0.0.1:5000/cat.jpg`

## Rendering Templates 1/2

- Generating HTML from within Python is not straightforward.
- Flask uses the [Jinja2](#) engine to allow you to write Python code inside HTML pages:
  - ▶ We will get back to Jinja2 soon.
- Consider the below directory tree

```
project—directory
|--- app.py
|--- templates
| |--- home.html
```

- app.py

```
app.py
from flask import Flask, render_template
```

```
app = Flask(__name__)
@app.route('/hello/<value>')
def hello(value):
 return render_template('home.html', name=value)
```

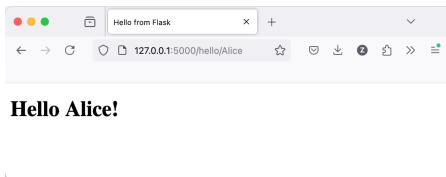


## Rendering Templates 2/2

- And html file

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Hello from Flask</title>
</head>
<body>
 <h1>Hello {{ name }}!</h1>
</body>
</html>
```

- You may view the generated html by entering in the address line `http://127.0.0.1:5000/hello/Alice`



# The Request Object 1/3

- For web applications it is crucial to react to the data a client sends to the server.
- In Flask, this information is provided by the global request object.
- Consider the below directory tree

```
project-directory
|--- app.py
|--- templates
 |--- login.html
 |--- welcome.html
```

## The Request Object 2/3

```
from flask import Flask, request, render_template
app = Flask(__name__)
@app.route('/', methods=['POST', 'GET'])
def main():
 if request.method == 'POST':
 return login()
 return render_template("login.html")
def valid_login(username, password):
 return username == 'Alice' and password == '7'
def login():
 error = None
 username = request.form.get('username')
 password = request.form.get('password')
 if valid_login(username, password):
 return render_template('welcome.html', name=username)
 else:
 error = 'Invalid username/password'
 return render_template('login.html', error=error)
if __name__ == '__main__':
 app.run(debug=True)
```

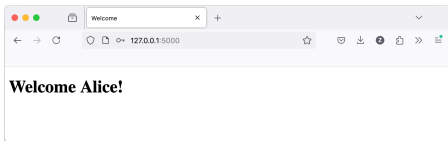
# The Request Object 3/3

## Login form



A screenshot of a web browser window with the title "Login Page". The address bar shows "127.0.0.1:5000". The page content features the heading "Simple Login Form" in bold. Below the heading, there is a form with two input fields: "Username: Enter Username" and "Password: Enter Password". To the right of the password field is a "Login" button.

## Success



A screenshot of a web browser window with the title "Welcome". The address bar shows "127.0.0.1:5000". The page content displays the message "Welcome Alice!" in bold.

## Failure



A screenshot of a web browser window with the title "Login Page". The address bar shows "127.0.0.1:5000". The page content features the heading "Simple Login Form" in bold. Below the heading, there is a form with two input fields: "Username: Enter Username" and "Password: Enter Password". To the right of the password field is a "Login" button. Below the form, there is a red error message: "Invalid username/password".

## File upload 1/2

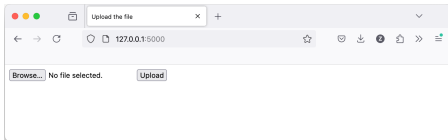
- You can handle uploaded files with Flask easily.
- Here is an example
- Consider the below directory tree

```
project—directory
|--- app.py
|--- templates
| |--- ack.html
| |--- submit.html
|--- uploads
```

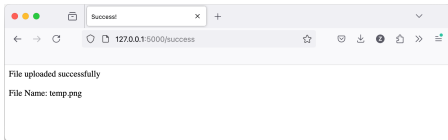
- See examples for the details of app.py, ack.html and submit.html.

# File upload 2/2

Submit



Success (ack)



# Jinja2

- Jinja2 is a general purpose, fast, expressive, extensible [templating language](#).
- Special placeholders in the template allow writing code similar to Python syntax, and then the template is passed some data to render the final document.
- You may install the latest release by typing on your terminal (preferably inside a virtual environment).  
`pip install Jinja2`
- Dependencies will be installed automatically when installing Jinja.

# Jinja2

- Jinja template is simply a text file. Jinja can generate any text-based format (HTML, XML, CSV, LaTeX, etc.).
  - ▶ It does not need to have a specific extension: .html, .xml, or any other extension is just fine.
  - ▶ But in this course, we will use html templates.
- A template contains variables and/or expressions, which get replaced with values when a template is rendered; and tags, which control the logic of the template.
- Let's check a very simple example.



## Very simple example for Jinja2 1/4

- Consider the following directory structure

```
project-directory
|--- jinja2_example_app.py
|--- templates
 |--- jinja2_example_base.html
 |--- jinja2_example_index.html
```

- Let the `jinja2_example_app.py` be:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
 return render_template('index.html', title='My Simple Page')

if __name__ == '__main__':
 app.run(debug=True)
```

## Very simple example for Jinja2 2/4

- Let the templates/jinja2\_example\_base.html be:

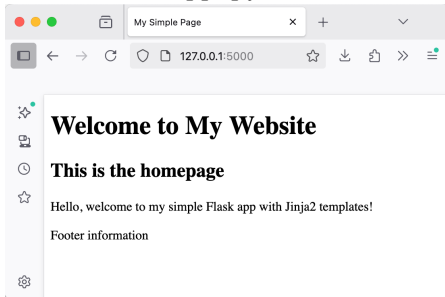
```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <title>{{ title }}</title>
</head>
<body>
 <header>
 <h1>Welcome to My Website</h1>
 </header>
 <main>
 {% block content %}
 <!-- Content from child templates will go here -->
 {% endblock %}
 </main>
 <footer>
 <p>Footer information</p>
 </footer>
</body>
```

# Very simple example for Jinja2 3/4

- Let the `templates/jinja2_example_index.html` be:

```
{% extends "jinja2_example_base.html" %}

{% block content %}
 <h2>This is the homepage</h2>
 <p>Hello, welcome to my simple Flask app with
 Jinja2 templates!</p>
{% endblock %}
```
- When you run it with `python app.py`, you will get:



## Delimiters in Jinja2

- There are a few kinds of delimiters. The default Jinja delimiters are configured as follows:
  - ▶ `{% ... %}` for Statements
  - ▶ `{{ ... }}` for Expressions to print to the template output
  - ▶ `{# ... #}` for Comments not included in the template output
  - ▶ `# ... ##` for Line Statements
- You may see examples of statements, expressions and comments in base file example.
- Line statements mark a line as a statement, e.g. `#` marking the line statement prefix, the following two are equivalent:

```

for item in seq
 item
endfor

{% for item in seq %}
 item
{% endfor %}

```

# Lab of WEB Technologies

## Lecture # 13

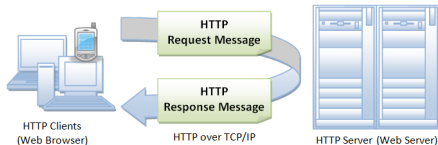
### Session, Cookies and Authentication

Zeynep Yücel

Ca' Foscari University of Venice  
zeynep.yucel@unive.it  
yucelzeynep.github.io

# Hyper-text transfer protocol (HTTP)

Remember some properties of HTML



- A [Request for Comments \(RFC\)](#) 2616: "The Hypertext Transfer Protocol (HTTP) is an application-level protocol for distributed, collaborative, hypermedia information systems. It is a generic, stateless protocol..."
- HTTP is an [asymmetric](#) request-response client-server protocol...
- In other words, HTTP is a [pull protocol](#),...
- HTTP is [stateless](#), since the current request does not know what has been done in the previous requests.

# The communication between client and server

Remember how a typical transaction works

- You **issue a URL** from your browser to get a web resource using HTTP, e.g. `http://www.example.com/index.html`,
- The browser **turns the URL into a request** message and sends it to the HTTP server.
- The HTTP **server interprets the request** message.
- It returns an appropriate response message, which is either the resource you requested or an error message.

## Session: From stateless to stateful

- HTTP was designed to be anonymous and stateless.
- Every request, even if generated by the same client, is independent from the ones performed previously.
- However, to create rich web-applications, we should be able to:
  - ▶ Keep track of each client, identify it and recognize upcoming requests coming from it
  - ▶ Associate application data to each specific client (for example login information, shopping cart, etc.).
- By default, the HTTP protocol is **stateless, but not sessionless**...
- The application data associated with each client, useful to implement the application logic, is called **session**.



# What is a cookie?

- A cookie (also known as a web cookie or browser cookie) is a small piece of data a server sends to a user's web browser.
- Cookies are essentially key-value pairs that the server asks the client browser to store for future usage.
- The browser may store cookies, create new cookies, modify existing ones, and send them back to the same server with later requests.
- Cookies enable web applications to remember state information.

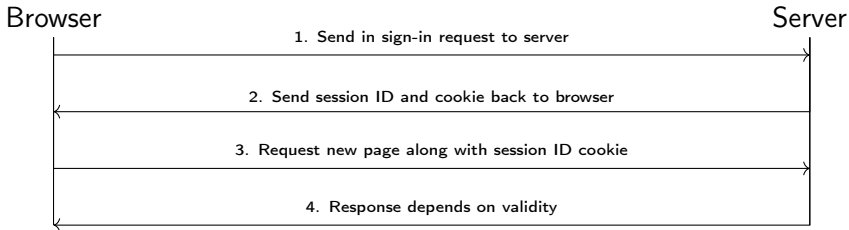
## Purpose of cookies

- Nowadays, cookies became a reliable and widely used method for identifying users and allowing persistent sessions.
- Cookies are mainly used for three purposes.
  - ▶ **Session management:** Session-related details such as user's sign-in status, shopping cart contents, game scores etc.
  - ▶ **Personalization:** User preferences like display language and UI theme.
  - ▶ **Tracking:** Recording and analyzing user behavior.

# Sample use of cookies

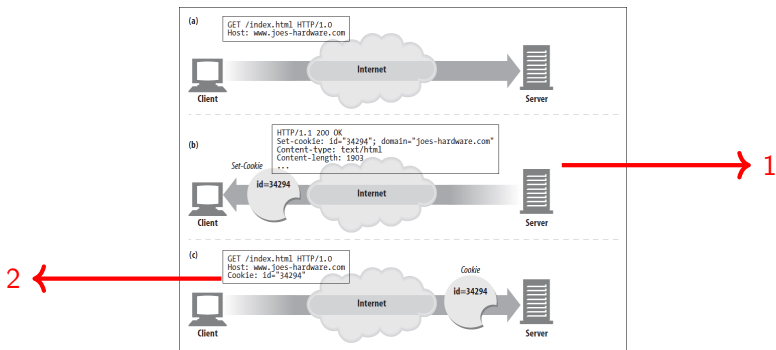
## A very simple user sign-in system

1. User sends sign-in credentials to the server.
2. If the credentials are correct, the server updates the UI to indicate that the user is signed in, and responds with a cookie containing a session ID that records their sign-in status on the browser.
3. At a later time, user moves to a different page on the same site. The browser sends the cookie containing the session ID along with the corresponding request to indicate that the user is signed in.
4. The server checks the session ID. If it is still valid, sends the user a personalized version of the new page. If it is invalid, the session ID is deleted and the user is shown a generic version of the page (or "access denied" message, sign-in page).



# How are cookies transmitted?

Cookies are sent in HTTP headers, so they are invisible to the user.



1. Server asks the client browser to store a certain name=value pair.
2. For each subsequent HTTP request, the client will send all previously stored cookies.

## Cookie types

- Session cookie (or transient or in-memory cookie)
  - ▶ Their lifespan is limited to the browser lifecycle.
  - ▶ When the browser is closed, they are automatically eliminated.
- Persistent cookie (or permanent cookie)
  - ▶ Their lifespan is explicitly defined by the server.
  - ▶ They remain valid after the browser is closed or even after the entire PC is restarted.

## Creating and updating cookies

- After receiving an HTTP request, a server can send one or more Set-Cookie headers with the response, each one of which will set a separate cookie.

- A simple cookie is set by specifying a name-value pair like:

`Set-Cookie: <cookie-name>=<cookie-value>`

- For example, the following HTTP response instructs the receiving browser to store a pair of cookies:

`HTTP/2.0 200 OK`

`Content-Type: text/html`

`Set-Cookie: yummy_cookie=choco`

`Set-Cookie: tasty_cookie=strawberry`

`[page content]`

- When a new request is made, the browser usually sends previously stored cookies for the current domain back to the server within a Cookie HTTP header:

`GET /sample_page.html HTTP/2.0`

`Host: www.example.org`

`Cookie: yummy_cookie=choco; tasty_cookie=strawberry`

# Removing cookies

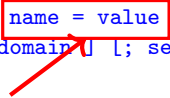
## Types of cookies

- Depending on the attributes set within the Set-Cookie header when the cookies are created, they can be either permanent or session cookies:
- Permanent cookies are deleted
  - ▶ either after the date specified in the Expires attribute:  
`Set-Cookie: id=a3fWa; Expires=Thu, 31 Oct 2021 07:28:00 GMT`
  - ▶ or after the period specified in the Max-Age attribute (in sec):  
`Set-Cookie: id=a3fWa; Max-Age=2592000`
- Session cookies are
  - ▶ without a Max-Age or Expires attribute
  - ▶ deleted when the current session ends.
  - ▶ The browser defines when the "current session" ends.

# Cookie header

- HTTP response header:

```
Set-Cookie: name = value [; expires=date] [; path= path]
 [;domain= domain] [; secure] [; HttpOnly] [; SameSite]
```



name-value pair to store on  
the client user agent

- HTTP request header:

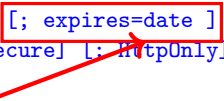
```
Cookie: name1 = value1 [; name2 = value2] ...
```



# Cookie header

- HTTP response header:

```
Set-Cookie: name = value [; expires=date] [; path= path]
[;domain= domain] [; secure] [; HttpOnly] [; SameSite]
```



Cookie expiration date. After expiration date, the cookie is automatically deleted. If this attribute or Max-age is not present, a session cookie is assumed.

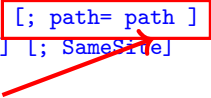
- HTTP request header:

```
Cookie: name1 = value1 [; name2 = value2] ...
```

# Cookie header

- HTTP response header:

```
Set-Cookie: name = value [; expires=date] [; path= path]
[;domain= domain] [; secure] [; HttpOnly] [; SameSite]
```



The cookie is bound to a particular resource on the server (and all the sub-resources in the path tree). If path=/app, then it will only be sent to URLs that match /app or any of its subdirectories (e.g., /app/page1).

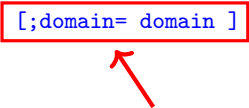
- HTTP request header:

```
Cookie: name1 = value1 [; name2 = value2] ...
```

# Cookie header

- HTTP response header:

```
Set-Cookie: name = value [; expires=date] [; path= path]
[;domain= domain] [; secure] [; HttpOnly] [; SameSite]
```



The cookie is bound to a particular domain or subdomain. If domain=example.com, then it can be sent to example.com and all its subdomains (e.g., sub.example.com).

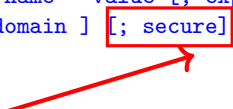
- HTTP request header:

```
Cookie: name1 = value1 [; name2 = value2] ...
```

# Cookie header

- HTTP response header:

```
Set-Cookie: name = value [; expires=date] [; path= path]
 [;domain= domain] [; secure] [; HttpOnly] [; SameSite]
```



The secure attribute instructs the browser to only send the cookie over secure connections (HTTPS).

- HTTP request header:

```
Cookie: name1 = value1 [; name2 = value2] ...
```

# Cookie header

- HTTP response header:

```
Set-Cookie: name = value [; expires=date] [; path= path]
 [;domain= domain] [; secure] [; HttpOnly] [; SameSite]
```



This restricts the cookie from being accessed via JavaScript (e.g., `document.cookie`).

- HTTP request header:

```
Cookie: name1 = value1 [; name2 = value2] ...
```

# Cookie header

- HTTP response header:

```
Set-Cookie: name = value [; expires=date] [; path= path]
 [;domain= domain] [; secure] [; HttpOnly] [; SameSite]
```



This specifies whether cookies are sent along with cross-site requests and helps protect against cross-site request forgery attacks.

- HTTP request header:

```
Cookie: name1 = value1 [; name2 = value2] ...
```

# Security

- Security focuses on protecting systems, networks, and data from unauthorized access, attacks, or damage.
- The primary goal of security is to ensure the integrity, availability, and confidentiality of data.
- This includes protecting the application against threats such as hacking, malware, data breaches, and attacks.
- Security measures include encryption, authentication, authorization, secure coding practices, regular security audits, firewalls, intrusion detection systems, and updates to fix vulnerabilities.

# Privacy

- **Privacy**, in the context of web applications, involves the proper handling of **personal data** and the rights of individuals regarding how their information is collected, used, and shared.
- The primary goal of privacy is to give users control over their personal information and to **protect user data from misuse**.
- Privacy measures may include **data anonymization, data minimization** (collecting only what is necessary), clear user consent mechanisms, privacy policies, and compliance with regulations.



## Cookies, security and privacy 1/2

- Security is about protecting against unauthorized access and attacks.
- Privacy is about managing and protecting personal data and rights.
- These are very important consideration when building websites.
- If done right, it can build trust with your users.
- If done badly, it can completely erode that trust and cause many other problems.
- Security and privacy problems are often underestimated when using cookies... Why?
  - ▶ One reason is that they **can be disabled**, so implicitly cookies become a **user responsibility**.
  - ▶ Another reason is that there exist the conviction that **simple data** "left by the server" on our browsers cannot be harmful.
- But cookies can actually be dangerous for our privacy and security!

## Cookies, security and privacy 2/2

- For instance, **third-party cookies** can be set by third-party content (e.g. an advertisement banner) embedded in sites.
- However, these third-party cookies can also be used to create **invasive user experiences**.
  - ▶ To download an advertisement, our browser may make an HTTP request to a **potentially malicious** external website.
  - ▶ Another classic example is when you search for product information on one site and are then **chased around the web by adverts** for similar products wherever you go.

## Cookie-related regulations

- **Browser vendors** know that users do not like chasing behavior etc., and as a result they have started to **block third-party cookies by default**, or at least make plans to go in that direction.
- There exist also legislation or regulations that cover the use of cookies:
  - ▶ The General Data Privacy Regulation (GDPR) in the EU
  - ▶ The ePrivacy Directive in the EU
  - ▶ The California Consumer Privacy Act
- These regulations apply to any site on the World Wide Web that users from these jurisdictions access. Thus, they have global reach
- They include requirements such as:
  - ▶ Notifying users that your site uses cookies.
  - ▶ Allowing users to opt out of receiving some or all cookies.
  - ▶ Allowing users to use the bulk of your service without receiving cookies.

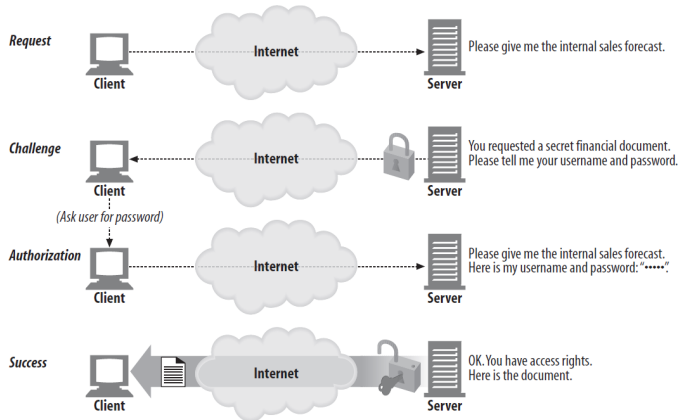
# Authentication

- Cookies allow a web server to store key-value pairs to the user agent requesting a certain resource.
- With this technique we can recognize the same client throughout subsequent requests but not its "real identity".
- HTTP support various mechanisms to **authenticate** a client by providing its own credentials.
- Authentication means showing some kind of evidence of the true physical identity of a certain client.
- It is usually based on some **shared information** between client and server (e.g. Username-password pair) that must be exchanged securely with a pre-defined protocol.

# General HTTP authentication framework

- HTTP authentication framework can be used
  - ▶ by a server to challenge a client request
  - ▶ by a client to provide authentication information.
- The general message flow above is the same for most (if not all) authentication schemes.
- The actual information in the headers and the way it is encoded does change!

# Challenge and response flow



# Authentication schemes

- The general HTTP authentication framework is the base for a number of authentication schemes.
- IANA<sup>1</sup> maintains a list of authentication schemes, but there are other schemes offered by host services (e.g. Amazon AWS).
- Some common authentication schemes include Basic, Bearer, Digest, HOBA, Mutual, Negotiate / NTLM, VAPID, SCRAM, AWS4-HMAC-SHA256...
- Schemes can differ in security strength and in their availability in client or server software.

---

<sup>1</sup>The Internet Assigned Numbers Authority (IANA) is a standards organization that oversees global IP address allocation, autonomous system number allocation, root zone management in the Domain Name System (DNS), media types, and other Internet Protocol-related symbols and Internet numbers.

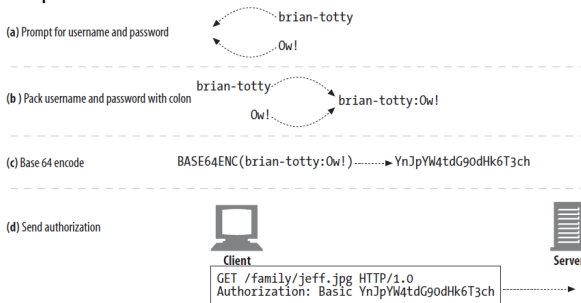
## General HTTP authentication framework

- In this lecture, we will look into Basic and Digest authentications.
- Both are based on a challenge-response framework, but differ in the way in which the information is encoded and exchanged.



# Basic authentication

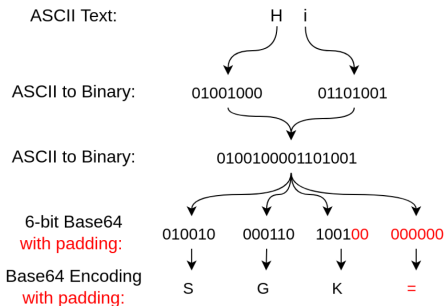
- Basic authentication is originally described in HTTP/1.0.
- It is widely supported, but offers very poor security.
- How basic authentication works
  - a. A server may challenge the client to send a username:password pair.
  - b. User provides the username:password pair.
  - c. username:password pair is encoded with Base64.
  - d. Encoded pair is sent in HTTP header.



- If it is correct, the resource is sent in the next transaction. If not, the challenge is repeated.

# How Base64 works

- Fundamentally, Base64 is used to encode binary data as printable text.
- Consider the sentence Hi.
- Obtain binary representation of each character (see ASCII-to-binary conversion table).
- ASCII uses 8 bits to represent individual characters, but Base64 uses 6 bits (in 24-bit sequences). So the binary needs to be broken up into 6-bit chunks.
- These 6-bit values can be converted into the appropriate printable character by using a Base64 table.
- Since Base64 uses 24-bit sequences, padding is needed when the original binary cannot be divided into a 24-bit sequence.



## Basic authentication problems

- Base64 has same security level of sending user/password with no encryption at all.
- User credentials can be easily decoded by anyone.
- Basic authentication is insecure...

# Digest authentication

## Rationale of digest authentication

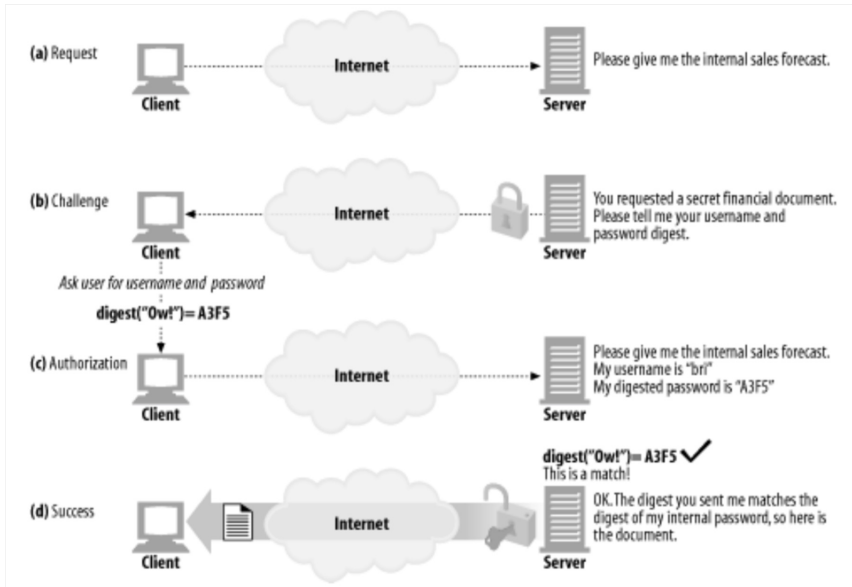
- Rationale of digest authentication is based on the fact that a server does not necessarily need to receive the password from the client.
- In other words, a **digest** is sufficient to prove that a client actually knows the correct password.
- So the password is not sent in cleartext

# Digest authentication

## Digest and hash function

- A **digest** is a small value generated by a hash function from a whole message.
- Ideally, a digest is quick to calculate, irreversible, and unpredictable, and therefore indicates whether someone has tampered with a given message.
- A **hash function** (or cryptographic hash function, digest function) can convert any message (string) to a fixed-length sequence of (random-like) bytes (e.g. MD5, SHA-256).
- To be used for cryptography, a hash function must be
  - ▶ quick to compute (since they are generated frequently)
  - ▶ not invertible (only brute-force can generate a message that leads to a given digest)
  - ▶ tamper-resistant (any change to a message leads to a different digest)
  - ▶ collision-resistant (it should be impossible to find two different messages that produce the same digest)

# Digest authentication



## Digest authentication problems

- With a hash function, we can avoid sending the password directly, but...
- Since a given password always generates the same digest, anyone eavesdropping the channel can steal the digest.
- In web security, this is called a **replay attack**.
- A replay attack happens when an attacker intercepts a previously-sent message and **resends it later** to get the same credentials as the original message, **potentially with a different instruction**.

## Preventing replay attacks

- Replay attacks can be prevented by including a unique, single-use identifier with each message that the receiver can use to verify the authenticity of the transmission.
- This identifier can take the form of a session token or "number used only once" (i.e. nonce).
- The client creates a digest of the pair ( $\langle \text{password} \rangle, \langle \text{nonce} \rangle$ ) so that digests are always different (i.e. they depend on the nonce) and thus can be used only once.



# Cookies in Flask

## Setting and getting cookies in Flask

- Using `set_cookie()` method, we can generate cookies in any application code.
- The syntax for this method is  

```
Response.set_cookie(key, value = '', max_age = None,
expires = None, path = '/', domain = None, secure = None)
```
- Parameters are
  - ▶ `key`: Name of the cookie to be set.
  - ▶ `value`: Value of the cookie to be set.
  - ▶ `max_age`: Usually a few seconds. Default is client's browser session.
  - ▶ `expires`: should be a datetime object or UNIX timestamp.
  - ▶ `domain`: To set a cross-domain cookie.
  - ▶ `path`: limits the cookie to given path. Default is the whole domain.
  - ▶ `secure`: If True, sends cookie only over HTTPS. If False or none, sends over HTTP or HTTPS.
- The `cookies.get( )` method retrieves the cookie value stored from the user's web browser through the request object.

## Simple example for setting and getting cookies 1/4

```
project—directory
|--- app.py
|--- templates
 |--- index.html
 |--- readcookie.html
```

index.html

```
<html>
```

```
<body>
```

```
 <h3>Enter userID</h3>
```

```
 <form action="/setcookie" method="POST">
```

```
 <input type='text' name='nm' />
```

```
 <input type='submit' value='Login' />
```

```
 </form>
```

```
</body>
```

```
</html>
```

## Simple example for setting and getting cookies 2/4

readcookie.html

```
<html>
<body>
 <form action="/getcookie" method="GET">
 <p><input type='submit' value='Show Cookie' /></p>
 </form>
</body>
</html>
```

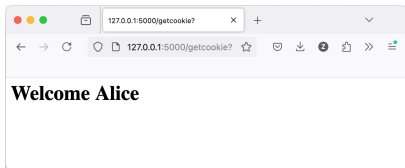
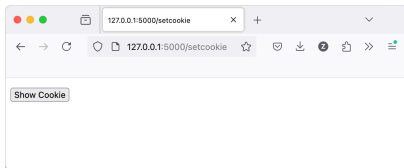
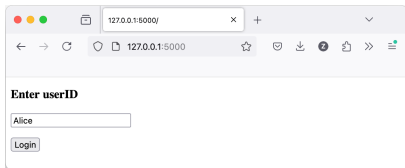
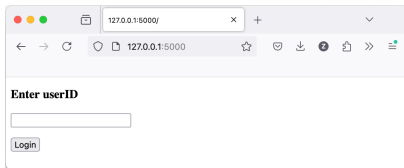
## Simple example for setting and getting cookies 3/4

app.py

```
from flask import Flask, render_template, request, make_response
app = Flask(__name__)
@app.route('/')
def index():
 return render_template('index.html')
@app.route('/setcookie', methods=['POST', 'GET'])
def setcookie():
 if request.method == 'POST':
 user = request.form['nm']
 resp = make_response(render_template('readcookie.html'))
 resp.set_cookie('userID', user)
 return resp
 else:
 return render_template('index.html')
@app.route('/getcookie')
def getcookie():
 name = request.cookies.get('userID')
 return '<h1>Welcome ' + name + '</h1>'
if __name__ == '__main__':
 app.run(debug=True)
```

## Simple example for setting and getting cookies 4/4

Under project directory, run `flask --debug --app app.py run`  
Go to `http://127.0.0.1:5000` on your browser.



## Another example for counting number of visits 1/2

project—directory

|--- app.py

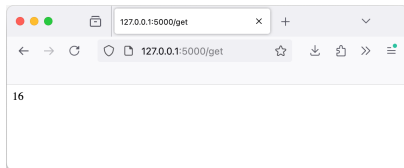
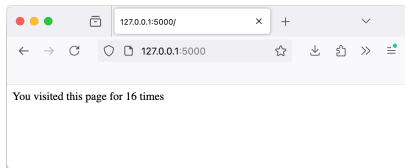
```
from flask import Flask, request, make_response
app = Flask(__name__)
app.config['DEBUG'] = True
@app.route('/')
def visitors_count():
 count = int(request.cookies.get('visitorsCount', 0))
 count = count+1
 output = 'You visited this page for '+str(count) + ' times'
 resp = make_response(output)
 resp.set_cookie('visitorsCount', str(count))
 return resp
@app.route('/get')
def get_visitors_count():
 count = request.cookies.get('visitorsCount')
 return count
if __name__ == '__main__':
 app.run(debug=True)
```

## Another example for counting number of visits 2/2

Under project directory, run `flask --debug --app app.py run`

Go to `http://127.0.0.1:5000` on your browser.

Refresh (CTRL+R) to see the counter increasing.



## Security in WWW era

- The security of Web Applications involves different layers and components
  - ▶ Security of HTTP Protocol
  - ▶ Security of Client
  - ▶ Security of Application Server
  - ▶ ...



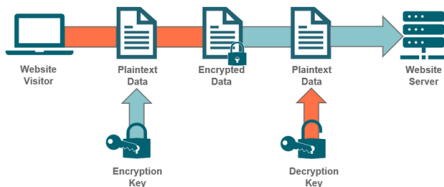
# HyperText Transfer Protocol Secure (HTTPS)

- HTTP request and response are in **plaintext**!
- Data contained in HTTP request and response are easy to read or modify by possible attackers.

How Insecure Website Communications Work (HTTP)



How Secure Website Communications Work (HTTPS)



- HTTPS is designed to **provide a secure and encrypted connection**, when exchanging data between a user's web browser and the website they are connected to.

# HTTP over SSL/TLS = HTTPS

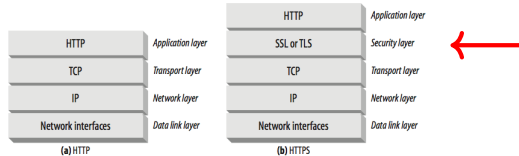
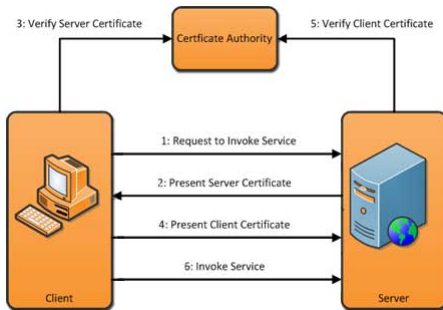


Figure: Physical layer at the bottom is omitted.

- HTTP is a protocol for application layer.
- HTTP is built upon **TCP**, which is a transport-layer protocol providing **reliable, ordered and error-checked transportation of data**.
- To make HTTP more secure, a security layer is added between HTTP and TCP, i.e. **SSL or TLS**.
- SSL stands for "Secure Sockets Layer," and TLS stands for "Transport Layer Security."
- These are cryptographic protocols that provide secure communication over a computer network.

## Two-way SSL authentication

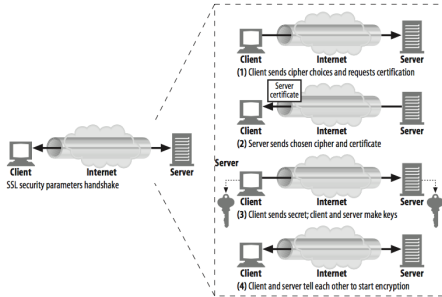


- Two-way SSL authentication using certificates means that 2 parties authenticate each other by verifying the presented certificate.
- Certificates are issued by a Certificate Authority (CA).
- They contain information about the CA, so the other party can check with the CA to determine whether the certificate is valid.
- To do this you have to trust the CA who has issued the certificate.

# Asymmetric Cryptography

- These certificates consist of a private part and a public part.
- The private part is used to identify yourself (this is secured by a password, and should be handled with care)
- The public part is used by the other party for verification purposes.
- If you want to communicate with someone, you hand them the public part and keep the private part for yourself.

# SSL handshake



- The browser receives a digital certificate from the server and runs some checks on the certificate.
- If the browser does not trust the certificate, then it will show a warning message and let the user decide whether to move on.
- The browser extracts a public key from the certificate, then generates a random secret key and encrypts it using the public key, and sends it back to the server.
- This is how the browser and server establish a secret key.

# Example of a certificate

On Firefox, you may click at the left side of the address bar  
Lock symbol » Connection is secure » More information » View  
certificate.

## Certificate

www.google.com		WR2	GTS Root R1
Subject Name			
Common Name	www.google.com		
Issuer Name			
Country	US		
Organization	Google Trust Services		
Common Name	WR2		
Validity			
Not Before	Mon, 24 Jun 2024 07:42:34 GMT		
Not After	Mon, 16 Sep 2024 07:42:33 GMT		
Subject Alt Names			
DNS Name	www.google.com		
Public Key Info			
Algorithm	Elliptic Curve		
Key Size	256		
Public Value	04:C9:F6:99:D5:49:CA:C7:30:96:B3:71:64:F8:57:EF:DF:2C:F3:23:5D:32:2...		

## Recap of Cookies

- Cookies are widely used method to identify users and allow persistent sessions.
- Cookies are key-value pairs that a server asks the client browser to store for future usages.
- They are sent in HTTP headers so are invisible to the user.
- They can be disabled, so implicitly cookies become a user responsibility.
- There exists the conviction that simple data "left by the server" on our browsers cannot be harmful...
- But they can actually be dangerous for our privacy and security!

# Security Problems of Cookies

- Cookies that contain sensitive information such as usernames, passwords, and session identifiers can be captured by an attacker.
- Three types of threats to cookies have been identified<sup>2</sup> as
  - ▶ Network threats
  - ▶ End-system threats
  - ▶ Cookie-harvesting threats

---

<sup>2</sup>Joon Park, Ravi Sandhu, Secure Cookies on the Web, IEEE Internet Computing, July-August, 2000, pp. 36-44.



## Cookie Attacks: Network threats

- Cookies that are sent over **unencrypted channels** can be subject to eavesdropping, i.e. the contents of the **cookie can be read by the attacker**.
- These types of threats can be prevented by the use of **Secure Sockets Layer or SSL protocol** in servers and Internet browsers although this works only if the cookies are on the network.
- One might also use cookies with only the sensitive information encrypted (instead of the entire data payload that is exchanged.)

## Cookie Attacks: End-system threats

- Cookies can be **stolen or copied from the user**, which could either reveal the information in the cookies or allow the attacker to **edit the contents of the cookies** and **impersonate the users**.
- This happens when a cookie, which is in the browser's end system and stored in the **local drive or memory** in clear text, is **altered or copied from one computer** to another with or without the knowledge of the user.
- When the attacker accesses the browser or the proxy cache, he could also obtain the cookie content. This kind of attack is called **cache sniffing**.

## Cookie Attacks: Cookie harvesting

- The attacker can try to impersonate a website.
- The users who believe that they are communicating with a legitimate Web server send cookies to the impersonating site.
- The attacker accepts these cookies and can use these harvested cookies for websites that accept third-party cookies.
- An attacker, for instance, could embed cross-site scripting (also abbreviated as XSS) in a URL he has posted in a discussion forum, message board, or email, which is then activated when the target opens the hyperlink.

## Secure cookies

- Secure cookies are a type of HTTP cookie that have Secure attribute set, which limits the scope of the cookie to "secure" channels (where "secure" is defined by the user agent, typically web browser).
- When a cookie has the "Secure" attribute, the user agent will include the cookie in an HTTP request only if the request is transmitted over a secure channel (typically HTTPS).
- "Secure" attribute is specified in RFC2965 and is optional of the header.

# Lab of WEB Technologies

## Lecture # 14

### Introduction to databases and SQL

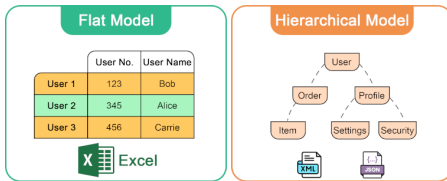
Zeynep Yücel

Ca' Foscari University of Venice  
zeynep.yucel@unive.it  
yucelzeynep.github.io

# Introduction

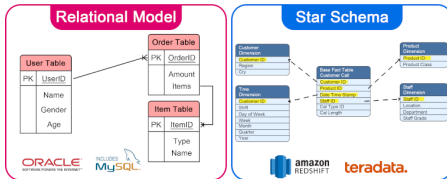
- In many situations, you will need to store, access, and possibly share a set of **structured data**.
  - ▶ Structured data: data with a fixed, structured organization which generally corresponds to a real-world system (i.e. a warehouse, a classroom, user data in a system).
- While data can be stored in many ways (e.g. as CSV files), the peculiarities of a DBMS are key to **provide consistency** of the data with the system.
- This seems a trivial task which you can achieve with a simple file...
- But if you have data of **different kinds** (binary, text, etc.), with complex structure (**many relations** between items), possibly large data sets (e.g. Facebook database) and **concurrent** read/write access, then a Database Management System (DBMS) is needed.

# Some common database models 1/3



- Flat Model:
  - ▶ It consists of a single table where each row is a record and each column is an attribute, making it simple and easy to implement.
  - ▶ However, it is limited in handling complex relationships efficiently due to its lack of structure beyond a single table.
- Hierarchical Model:
  - ▶ It organizes data into a tree-like structure with parent-child relationships, making it efficient for hierarchical data.
  - ▶ But, it is inflexible for many-to-many relationships and can become complex to manage as the data structure grows.

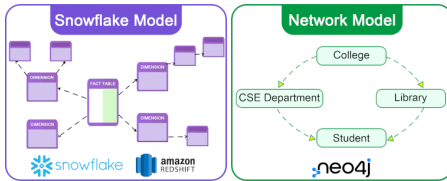
# Some common database models 2/3



- **Relational Model:**
  - ▶ It organizes data into tables with rows and columns, facilitating easy data access and management.
  - ▶ Its use of keys and normalization ensures data integrity and reduces redundancy, while SQL provides a simple and flexible query language.
- **Star Schema:**
  - ▶ The star schema is a data modeling approach used in data warehouses, featuring a central fact table linked to multiple dimension tables.
  - ▶ It is designed to optimize query performance in analytical processing by reducing joins and enabling fast data retrieval.



# Some common database models 3/3



- **Snowflake Model**

- ▶ The snowflake model is a variation of the star schema that normalizes dimension tables into multiple related tables, reducing redundancy and enhancing data integrity.
- ▶ Although it results in more complex queries due to additional joins, it offers better storage efficiency and is beneficial when dimension tables are large or frequently updated.

- **Network Model**

- ▶ The network model enables each record to have multiple parent and child records, creating a graph structure that captures complex relationships between data entities.
- ▶ It addresses the limitations of hierarchical models by effectively managing many-to-many relationships, allowing for more flexible data representation.

# Outline

We will look at:

- Simple data representation with a **symbolic model**, to understand how data are organized in an abstract way.
- How this is **translated into tables**, relations and keys (the basis of the relational databases).
- A **language** that is used to access structured information in many DBMS (SQL).
- **Python bindings** to use it in your applications.

# Relational database

## Symbolic models

- To describe a real system into a computer, you have to **build a model** that represents it.
- This conceptual part of the design **influences all the code** to be realized afterwards.
- In a relational database, there are (at least) **two kinds of knowledge** that you have to organize:
  - ▶ Concrete knowledge
  - ▶ Abstract knowledge

# Relational database

## Concrete knowledge

- Relational databases assume that you can model your systems into **entities**, **properties** and **relationships**.
- We take as an example a database to store students and exam results.
  - ▶ **Entity**: is something that you need to record facts about.
    - ▶ The student is an entity of our model, as well as the exam.
  - ▶ **Property**: is an attribute of an entity, which is not interesting itself, but it becomes interesting as it describes an entity.
    - ▶ The student name, id number, and email are properties of a student; the exam date is a property of the exam.
  - ▶ **Relationships**: are links between entities.
    - ▶ A student may have done many exams.

# Relational database

## Abstract knowledge

- Abstract knowledge **imposes formal restrictions** on the admissible values of concrete knowledge:
  - ▶ Properties may be limited to some kind.
    - ▶ The name of a student is a string limited to maximum 256 chars.
  - ▶ Relationships may be unary or n-ary.
    - ▶ Each student has only one study plan (unary relationship).
    - ▶ Each study plan refers to many exams (n-ary relationships).

# Relational database

Concrete and Abstract Knowledge should not change

- Concrete and abstract knowledge describe the entities of the system.
- If your ground truth does not change, **concrete and abstract knowledge should not change** either.
- The **definition of an entity** should be **general enough** to capture all the data that you need at first place.
- Similarly, relationships should not change.
- **If reality changes**, concrete knowledge will change accordingly.
  - ▶ For example, at some point each student receives a Moodle ID, and the student entity will be updated with a new property.
- **Importance of Models**
  - ▶ It is hard to change a model, when a DB is in use.
  - ▶ Keeping a wrong model may make the DB inefficient.
  - ▶ It is really important to properly design your DB!

# Relational database

Disambiguation between concrete and abstract knowledge and data

- Concrete and abstract knowledge refer to the organization of the data, not to data itself.
- These abstractions should not change often.
  - ▶ The real data about students instead will be constantly added and removed, together with relationships
  - ▶ For example, a student can be added, an exam is passed by a student etc.

# From knowledge to tables

## Tables, columns

- A relational database uses its own terminology and data structures to represent concrete and abstract knowledge:
  - ▶ **tables** (entities), **columns** (properties), **keys and foreign keys** (relationships).
    - ▶ A **table** represents **an entity**.
    - ▶ A **row of the table** represents an **instance of the entity**.
    - ▶ In **each column** of the table, there is **property**.
- Students table:

name	surname	student_ID	birthdate	email
Albert	Einstein	0001	1879-03-14	albert@gmail.com
Alan	Turing	0002	1912-06-23	alan@yahoo.com
Francesco	Totti	0003	1976-09-27	capitano@asroma.it



# Tables

## Keys

name	surname	student_ID	birthdate	email
Albert	Einstein	0001	1879-03-14	albert@gmail.com
Alan	Turing	0002	1912-06-23	alan@yahoo.com
Francesco	Totti	0003	1976-09-27	capitano@asroma.it

- A **key** is a minimal subset of **columns that uniquely identifies a row**.
- In the **simplest case**, it is the the value of only **one column**.
- In **many cases**, you may need **more than one column** to uniquely identify every row.
  - ▶ The **student ID is a key** on the students table, because there can not be two students with the same ID.

# Tables

## Primary Keys

- There could be **more than one key**.
- For instance, in the student tables both the **student\_ID** and the **email** uniquely represent one student.
- In this case, it is a good practice for the DB designer to **choose one as the primary key**.
- Databases generally automatically **create an index using the primary key**.
- Search for data is faster with the primary key.

# Tables

## Foreign Keys

- A **foreign key** is a key that identifies a row in a different table.
- **Foreign keys implement relationships** between entities.
- Observe the below exam table, which represents the exam grades that were registered for each student.

course_title	course_ID	date	student_ID	grade
Software Development	ET7018	2018-12-18	0001	31
Computer Security	ET7016	2019-10-12	0001	28
Computer Security	ET7016	2019-10-12	0003	30

- Here, **student\_ID** is the **foreign key** pointing to rows to the students table.
- What would be a good **primary key** for the exam table?
- (course\_ID, student\_ID) is the primary key for this table.

# Tables

## Importance of the table organization

- The fact that a **key** is given by a certain subset of columns is **directly derived by the concrete and abstract** knowledge of the system.
- The **DB designer needs to decide** this based on knowledge of the system.
- The choice of the relationships **impacts performance**.

# From Data to DBMS

## Constraints and DBs

- We started from an abstract representation of data.
- We observed that a table is a suitable tool to represent data of that kind.
- We now move to **how tables are managed by a DBMS**.

# Database

## Definition and properties

- Definition of DB:
  - ▶ Database is a **collection of persistent data**, partitioned into two:
    - ▶ The schema is a set of rules that defines how data should be organized and what values are allowed.
    - ▶ These rules do not change over time and include both the data structure and the restrictions to ensure data is valid.
    - ▶ The **data**, a **time-variant** representation of specific **facts**.

# Database

## Definition and properties

- Some properties of data in a DB
  - ▶ Once created, the **data** continue to exist until being explicitly deleted (**persistent**).
  - ▶ They are **accessed by** an atomic work unit (called **transaction**).
    - ▶ Here, "atomic" refers to the fact that a transaction either saves all the changes to the database or none at all.
  - ▶ They can be **protected** both from unauthorized users and from hardware and software failures.
  - ▶ They can be **shared concurrently** by several users.

# DBMS

## Definition

- Definition of DBMS
  - ▶ A Database Management System (DBMS) is a **software system** that helps you organize and manage data.
  - ▶ It allows you to **define** your database schema, choose the best way to store the data, and easily **access** and **retrieve** the information you need.
  - ▶ You can use special commands called queries or programs to access and work with the data.



# DBMS

## Properties

- Properties
  - ▶ A DBMS generally has one or more **administrators**.
  - ▶ The administrators of a DBMS define its **schema**, which is the logic how information is recorded.
  - ▶ The **schema** of a DBMS represents the **concrete and abstract** knowledge.
  - ▶ A DBMS has also a **physical representation of the data**.
    - ▶ This means that data may be saved in one file, in several files, tables can be stored as hash tables or lists.
  - ▶ This pertains to the way the specific software works, and the **administrator** generally does **not interfere** with it.
  - ▶ Many DBMS provide a complex way of performing access control on the data, i.e. they allow the **administrator to create users** with specific privileges.

# DBMS Languages

- A DBMS provides a language to access and modify the data.
- The Structured Query Language (SQL) is the most common.
- SQL is informally subdivided in "sublanguages":
  - ▶ The data definition language (DDL)
  - ▶ The data query language (DQL)
  - ▶ The data manipulation language (DML)
  - ▶ The data control language (DCL)

# Non-procedural Languages

- A first important point of SQL is that it is a **declarative language**.
  - ▶ Most the languages that you studied so far are procedural languages.
- Procedural vs Declarative
  - ▶ **Procedural** languages are defined as **sequences of steps** that the process will do, one after the other.
    - ▶ They contain control switches loops and all the other features you are familiar with.
    - ▶ Python is a procedural language.
  - ▶ **Declarative** languages are **"set oriented"**.
    - ▶ They provide a **set of records** that meet a certain condition.
    - ▶ The user describes what data they want, and the system takes care of finding it.

## SQL as a declarative language

- With SQL, you do **not** write a program that does a **sequence of actions** to retrieve the data.
- Instead, you just **"declare"** what are the **characteristics** of the data you want, and the DBMS returns the correct data.
- It is better **suited for a DB**, but it has many **limits for other applications**.
- This is why you **generally use a procedural** language (like Python) to **create your application**.
  - ▶ Inside your application, you use proper libraries that use SQL to retrieve/modify the data from the DB.
- Note, as an **exception**, that SQL uses also some procedural code, that you can run to make some tasks easier.

# Data definition sublanguage (DDL)

- The DDL is used to **define the schema** of the database.
- The common commands you use in the DDL are:
  - ▶ CREATE TABLE
  - ▶ ALTER TABLE
  - ▶ DROP TABLE

# Data definition sublanguage (DDL)

## Creating tables

```
CREATE TABLE student (
 name CHAR(256) NOT NULL,
 surname CHAR(256) NOT NULL,
 student_ID CHAR(8) NOT NULL,
 birthdate CHAR (10) NOT NULL,
 email CHAR(256) NOT NULL,
 PRIMARY KEY (student_ID),
 CHECK (email LIKE ('%@%')));
```

- **No procedures:** just declare the way you want your tables.
- Each **column** has a **name**, a **type** and a **size**.
- You can insert **constraints** on the values like:
  - ▶ **NOT NULL:** must not be empty
  - ▶ **CHECK(...):** email string must contain "@"
- SQL always **terminates with ';'.**
- **Indentation** helps, but it is **not mandatory**.

# Data definition sublanguage (DDL)

## Creating tables

```
CREATE TABLE student (
 name CHAR(256) NOT NULL,
 surname CHAR(256) NOT NULL,
 student_ID CHAR(8) NOT NULL,
 birthdate CHAR (10) NOT NULL,
 email CHAR(256) NOT NULL,
 PRIMARY KEY (student_ID),
 CHECK (email LIKE ('%@%')));
```

- You can define a **primary** key.
- By convention (not mandatory), SQL keywords are **all-CAPS**.
- You can define a **primary** key **on multiple columns**.
- You can define a **foreign** key.
- The **foreign** key **must exist!**
  - ▶ When you define a foreign key, the value you insert into the foreign key column(s) must correspond to an existing primary key (or unique key) value in the referenced table.

# Data definition sublanguage (DDL)

## Removing and Modifying Tables

- You can **remove** a table with:

```
DROP TABLE exam
```

- You can modify it with:

```
ALTER TABLE exam
```

```
ADD COLUMN 'exam_type' CHAR(10) NOT NULL;
```

- The ALTER TABLE command supports also DROP and MODIFY



# Data definition sublanguage (DDL)

## DDL and Knowledge

- DDL creates the database schema, i.e. the representation in an SQL database of the concrete and abstract knowledge.
- This includes tables, properties, and relationships (foreign keys)
  - ▶ In SQLite, you can access the schema with the .schema command.

## Data manipulation sublanguage (DML)

```
INSERT INTO student (name, surname, student_ID, birthdate, email)
VALUES('Albert', 'Einstein', '0001', '1879-03-14', 'albert@gmail.com');
```

```
INSERT INTO exam
VALUES ('Software Development', 'ET7018', '2018-12-18', '0001', 31);
```

- Repeating column names is not mandatory, if you fill all the columns.

# Data query sublanguage (DQL)

## Retrieving data

```
SELECT name, surname FROM student;
```

```
Albert|Einstein
```

```
Alan|Turing
```

```
Francesco|Totti
```

- SELECT returns data.
- You can select data from all columns (using '\*') or specify the columns you want.
- SELECT returns all lines from a table
- Fields are separated by a "|"

```
SELECT * FROM student;
```

```
Albert|Einstein|0001|1879-03-14|albert@gmail.com
```

```
Alan|Turing|0002|1912-06-23|alan@yahoo.com
```

```
Francesco|Totti|0003|1976-09-27|capitano@asroma.it
```

# Data query sublanguage (DQL)

## Retrieving data

```
SELECT name, surname FROM student
WHERE student_ID = '0001';
```

Albert|Einstein

```
SELECT * FROM exam WHERE grade > 30;
```

Software Development|ET7018|2018-12-18|0001|31

```
SELECT name, surname FROM student
WHERE student_ID = '0001' OR
student_ID = '0002';
```

Albert|Einstein  
Alan|Turing

- WHERE returns only the lines that match a specific pattern.
- You can use multiple conditions with the OR or AND operators.

## Matching data from many tables

- When **relationships** exists, it is often needed to select data from **more than one table**.
- In our example, how to retrieve the full name of the students that got a grade of 30 at some exam?
- Data are spread on two tables:
  - ▶ name, surname are on table `student`.
  - ▶ grade is on table `exam`.
- The **wrong way** is to first retrieve all data from one table, and then look inside the data for the one you need.
- The **right way** is using a `JOIN` statement.

# SQL JOIN

- JOIN gives you the **Cartesian product** of tables A and B: i.e. the combination of all rows from table A with all rows from table B.
  - ▶ If you have two tables with 3 rows, it will output 9 rows.
- You can imagine JOIN as a command that temporarily creates a new table, that you can access as the existing ones.

# SQL Join

```
SELECT * from student JOIN exam;
```

Albert|Einstein|0001|1879-03-14|albert@gmail.com|Software Development|ET7018|2018-12-18|0001|31

Albert|Einstein|0001|1879-03-14|albert@gmail.com|Computer Security|ET7016|2019-10-12|0001|28

Albert|Einstein|0001|1879-03-14|albert@gmail.com|Computer Security|ET7016|2019-10-12|0003|30

Alan|Turing|0002|1912-06-23|alan@yahoo.com|Software Development|ET7018|2018-12-18|0001|31

Alan|Turing|0002|1912-06-23|alan@yahoo.com|Computer Security|ET7016|2019-10-12|0001|28

Alan|Turing|0002|1912-06-23|alan@yahoo.com|Computer Security|ET7016|2019-10-12|0003|30

Francesco|Totti|0003|1976-09-27|capitano@asroma.it|Software Development|ET7018|2018-12-18|0001|31

Francesco|Totti|0003|1976-09-27|capitano@asroma.it|Computer Security|ET7016|2019-10-12|0001|28

Francesco|Totti|0003|1976-09-27|capitano@asroma.it|Computer Security|ET7016|2019-10-12|0003|30

- Note that every row in student is combined with every row on exam, so there are **mismatched values** (Albert appears to have done Computer Security twice).
- We just want **the rows** in which the student\_ID column in the student table **matches** the student\_ID column in the exam table.

# SQL Join

```
SELECT student.name, student.surname from student JOIN exam
WHERE student.student_ID = exam.student_ID;
```

Albert|Einstein|0001|1879-03-14|albert@gmail.com|Software Development|ET7018|2018-12-18|0001|31

Albert|Einstein|0001|1879-03-14|albert@gmail.com|Computer Security|ET7016|2019-10-12|0001|28

Francesco|Totti|0003|1976-09-27|capitano@asroma.it|Computer Security|ET7016|2019-10-12|0003|30

- Now we matched the rows in table students with the corresponding row on table exam.
- We still need one more condition: the grade must be 30.



# SQL Join

- We solve this with an AND condition.

```
SELECT student.name, student.surname from student JOIN exam
WHERE student.student_ID = exam.student_ID AND
exam.grade = 30;
```

Francesco|Totti

- Note that when you use JOIN you must specify the table name when you match a column: `SELECT name` becomes `SELECT student.name`.
- This is mandatory only when columns names are repeated, but makes the query easier to read

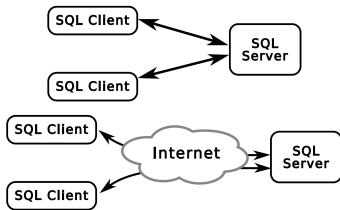
## SQL takeaway

- Now, you know how to create tables, insert data and query data from one table
- You also know how to make JOIN queries.
- We need to know how to use SQL inside a Python program.

# Phyton SQLite

## SQLite: Serverless DBMS

- The most used SQL servers are **reachable using a TCP** socket.
- They are servers that **open a TCP port**.
- The programs that use the DB communicate using **a TCP connection**.
- Such connections may be local or go through the Internet.



- The **Server** handles the **concurrency**.
  - ▶ It is the server that orders the queries and make it possible to have more than one client accessing the data at the same time.

# SQLite

- SQLite is a different kind of SQL Server, it relies on a **local process and a file**.
- You can **access an SQLite DB**, only if you reside on the **same host**.
  - ▶ Using an SQLite DB is **similar to opening/closing a file**.
  - ▶ SQLite is often used to store local configurations.
  - ▶ For **large-scale databases** accessed with a network connection you need other **more complex** software.
- Yet, **the way you code** for SQLite is exactly the **same** you would code for any other SQL DBMS.
- SQLite is supported as a **standard Python module** and requires no configuration, that's why we use it as an example.

## Prepared statements

- A prepared statement is a **query** that is passed to the DBMS **with fixed placeholders** (i.e. **?**) for the variables.
- `cursor.execute()` is the correct way to use prepared statements in `PySQLite`:

```
rows = conn.execute (" SELECT * FROM user
 WHERE username =? and password =?", (username , digest))
```

- This line is telling to `SQLite` that a query with 2 placeholders will be made and the values for the placeholders will follow.
- The DBMS **receives separately** the query and the value of the variables.
- For every placeholder, the DBMS expects one value.
- Only one query is executed at a time.

# Hash functions

- A **hash function** is a function that **converts an input** of any length into a **fixed-length string**.

- The conversion is irreversible:

```
import hashlib
```

```
plaintext = """A text of any length, of any size, of any kind,
or even a binary file""" # Triple quotes for multi-line string
```

```
digest = hashlib.sha256(plaintext.encode('utf-8')).hexdigest()
print(digest)
```

```
>> 44a5509918a58fbf5f5c21b01d81f17b9aea3b66ddb0e34d1f38764c10eb5a11
```

- A password DB does **not store passwords**, it stores **hashes** of passwords.
- When the user enters the password again, the **password will be hashed and compared to the one in the DB**.

# Hash Salts

- Hashes should not be repeated.
  - ▶ If the user stores the same password in two different databases, the hash should be different.
- To achieve this, the hash must be computed **on a random number (a salt)** that is saved together with user data.

```
import hashlib
import random
password = " the password typed by the user "
salt = str(random . random ())
concat = salt + password
digest = hashlib.sha256(concat.encode('utf -8')).hexdigest ()
print (digest)
```

```
>> 52793290377fd8e5038e140b0b60e266a6b9fb26ce2980a2aace83f2d257df2c
```

- The **salt is unique for each user**, and it is saved into the user table.
  - ▶ You need it to check the password when the user logs-in.

# References

- Features

<https://www.sqlite.org/features.html>

<https://www.sqlite.org/about.html>

- Python SQLite module

<https://docs.python.org/3/library/sqlite3.html>

- The theory of DBMS is taken from the short book of Antonio Albano (UniPi) available on his course page

<http://didawiki.cli.di.unipi.it/lib/exe/fetch.php/bdd-infuma/bookdb.pdf>

- The SQLitetutorial website contains several tutorials you can follow to practice with SQL commands

<https://www.sqlitetutorial.net/>



# Lab of WEB Technologies

## Lecture # 15 Database exercise

Zeynep Yücel

Ca' Foscari University of Venice  
zeynep.yucel@unive.it  
yucelzeynep.github.io

## Scenario and application description

- Build a web application called the "Cookbook App" with the following features:
  - ▶ User registration, login, logout.
  - ▶ Add recipes with dish names, dish types, descriptions, and dietary considerations, i.e. undesired ingredients.
  - ▶ Search recipes based on:
    - ▶ Dish type
    - ▶ Exclusion of specific undesired ingredients
  - ▶ Find recipes of a certain dish type that do not contain certain undesired ingredients.

# Features and functional requirements

User authentication, landing page and cookbook page

- Main page [index.html](#):
  - ▶ Presents two separate forms:
  - ▶ Login Form: For users to sign in.
  - ▶ Registration Form: For new users to create an account.
  - ▶ After successful login, users are redirected to the main "Cookbook" page.

# Features and functional requirements:

## Adding and searching recipes and styling

- Cookbook Page [index.html](#):
  - ▶ Accessible only to authenticated users.
  - ▶ Contains links to:
    - ▶ A page to add a recipe into the database
    - ▶ A page to search recipes in the database
    - ▶ Preferences
    - ▶ Logout
- Adding a New Recipe [add\\_recipe.html](#): Users can specify
  - ▶ Dish Type: A dropdown menu with type options.
  - ▶ Dish Name: Text input.
  - ▶ Description: Multi-line text area for cooking instructions.
  - ▶ Dietary considerations: A set of checkboxes representing common allergens, religious or ethical restrictions.
  - ▶ Upon submitting, the recipe is saved in the database.

# Features and functional requirements:

## Adding and searching recipes and styling

- Searching for Recipes `search_recipe.html`: User can search based on selected criteria.
  - ▶ Dish Type: Dropdown menu with same options as in adding recipes.
  - ▶ Dietary considerations: Checkboxes with same options as in adding recipes.
  - ▶ The system displays all recipes matching the selected dish type that do not contain any of the undesired ingredients.
- Design and styling `styles.css`:
  - ▶ The interface should have a clean, simple layout.
  - ▶ You may a CSS file to style elements such as body font, form spacing, button appearance, and warning messages.
  - ▶ Flash messages should be styled to stand out (e.g. in red).

# Technical Details and Implementation Guidelines:

## Backend

- Use Flask as backend.
- Implement user registration, login, and session management.
- Store user credentials securely using password hashing (e.g. werkzeug.security).
- Protect routes so only logged-in users can access recipe management and search features.
- Handle form submissions for login, registration, adding recipes, and searching recipes.

# Technical Details and implementation guidelines

## Frontend

- Use Jinja2 templates with inheritance:
- `layout.html` as the base layout, including header, flash message display, and linked styles.
- Specific pages (`index.html`, `add_recipe.html`, `search_recipe.html`, `preferences.html`) extend the base layout.
- Forms should be method POST and include necessary input fields.
- Links should use Flask's `url_for` to generate URLs dynamically.

# Technical Details and Implementation Guidelines:

## Database

- Use SQLite with two tables:
  - ▶ users (for authentication)
  - ▶ recipes (for recipe details)
- Initialize database schema, if not already created.
- Design the database with proper keys.



## Deliverables overview

project—directory

- |--- app.py
- |--- cookbook.db
- |--- database.py
- |--- static
  - |--- styles.css
- |--- templates
  - |--- add\_recipe.html
  - |--- index.html
  - |--- layout.html
  - |--- preferences.html
  - |--- search\_recipe.html
- |--- venv (optional but recommended)
  - |--- bin
  - |--- include
  - |--- lib